CIE-1 STUDY GUIDE · 5TH SEMESTER · THEORY

Software Engineering & Modelling

COURSE CODE CS51



Everything you need for **CIE-1**, explained from zero, with every slide point covered, diagrams you can reproduce in the exam, highlighted key points, tips and a question bank for each chapter.

- 70+ diagrams
- Key points highlighted
- Exam tips & memory tricks
- One-page recaps
- 85 likely questions + how to answer

UNIT 1

Process & Agile

- Introduction & ethics
- Software processes
- Agile development

UNIT 2

Requirements & Design

- Requirements engineering
- Design concepts
- The design model

UNIT 3 · PART

Architecture

- Architectural design
- Design decisions
- Architectural views (4+1)

Built from **your lecture slides**, **Sommerville, Software Engineering (10th ed)** and **Pressman, Software Engineering: A Practitioner's Approach**.

Course coordinators
Mr. Pradeep Kumar D · Dr Shilpa S Chaudhari

Start here: how to use this guide

This guide covers **exactly the CIE-1 portion** of Software Engineering & Modelling (CS51): all of Unit 1, all of Unit 2, and Unit 3 up to **architectural views**. It assumes you know nothing yet. Every point from your lecture slides is here, explained in plain English first, then in the textbook wording you should use in answers.

Unit	Syllabus topics (CIE-1)	Where in this guide
Unit 1	**Introduction:** professional software development, software engineering ethics	Chapter 1 · Introduction
Unit 1	**Software processes:** process models, process activities, coping with change, process improvement	Chapter 2 · Software Processes
Unit 1	**Agile software development:** agile methods, agile development techniques, agile project management	Chapter 3 · Agile Development
Unit 2	**Requirements engineering:** functional & non-functional requirements, RE processes, elicitation, specification, validation, change	Chapter 4 · Requirements Engineering
Unit 2	**Design concepts:** design within the context of SE, design concepts, the design model	Chapter 5 · Design Concepts
Unit 3	**Architectural design:** architectural design decisions, architectural views (patterns onwards are after CIE-1)	Chapter 6 · Architectural Design

How each chapter works

1. **Opener page**: what the chapter covers, how long it takes and what you will be able to do.
2. **Sections**: each idea is explained simply, then given in exam wording, with an example and a diagram where one helps.
3. **Chapter in one page**: a dense revision sheet. Re-read it the night before the exam.
4. **Likely exam questions**: each one says what to write and which diagram to draw.

What the coloured boxes mean

DEFINITION — LEARN THIS LINE
A definition to learn **word for word**. Write it as the first line of your answer.

★ KEY POINT
A fact you must remember. These are the things examiners look for.

IN SIMPLE WORDS
The idea in everyday language. Read this first if a topic feels confusing.

EXAMPLE
A worked example, usually from the textbook case studies.

✎ EXAM TIP
How to write the answer: structure, diagram, keywords.

⚠ WATCH OUT
A common confusion or mistake that loses marks.

MEMORY TRICK
A trick for remembering a list.

? QUICK CHECK
Test yourself. Cover the answer and try it first.

Highlighted phrases are the exact keywords to use in your answers.

Priority badges

★★★ MUST KNOW

Asked often: definitions, models, diagrams, comparisons. Study these first and practise the diagrams.

★★ IMPORTANT

Commonly asked as short questions or parts of long answers. Understand them well.

★ READ ONCE

Background or lower-priority material. Read it once so nothing surprises you.

Study plans

If you have a week

- **Day 1**: Ch 1 + start Ch 2
- **Day 2**: finish Ch 2 and draw every model
- **Day 3**: Ch 3
- **Day 4**: Ch 4
- **Day 5**: Ch 5 + Ch 6
- **Day 6**: all question banks
- **Day 7**: one-page recaps + diagrams

If you have 3 days

- **Day 1**: Ch 1–3, focusing on ★★★ MUST KNOW
- **Day 2**: Ch 4–6, focusing on ★★★ MUST KNOW
- **Day 3**: question banks, then recaps; redraw every diagram once

If it's the night before

- Read all six **one-page recaps**
- Redraw the must-know diagrams from memory
- Skim each question bank and say the answer outline out loud
- Sleep; recall is better when you're rested

✎ GOLDEN RULE FOR SE ANSWERS

For anything worth more than 2 marks: **definition → diagram → explanation in points → example → advantages/disadvantages (or comparison)**. A neat, labelled diagram earns marks even when your text is short.

⚠ NOT IN CIE-1, SO SKIP THESE FOR NOW

Unit 3 from **architectural patterns** onwards (MVC, layered, repository, client–server, pipe-and-filter, application architectures), object-oriented design using UML, design patterns and implementation issues. Also all of **Unit 4** (testing, evolution) and **Unit 5** (project management, planning, quality). They come after CIE-1.

Contents

UNIT 1 **Software Process & Agile Development** 5

CH 1 **Introduction to Software Engineering** 6

1.1 Why software engineering matters 6
1.2 Professional software development 7
1.3 Software products and product specification 8
1.4 Essential attributes of good software 9
1.5 What is software engineering? 10
1.6 General issues that affect software 12
1.7 Software engineering diversity and application types 12
1.8 Internet (web-based) software engineering 14
1.9 Software engineering ethics 14
1.10 Case studies used throughout the course 16
Chapter 1 in one page 20
Likely exam questions 21

CH 2 **Software Processes** 22

2.1 The software process 22
2.2 Software process models (Sommerville) 24
2.3 More process models (from Textbook 2, Pressman) 28
2.4 Process activities 32
2.5 Coping with change 36
2.6 Process improvement 39
2.7 Comparing all the process models 41
2.8 Comparative analysis on the case studies (lab) 42
Chapter 2 in one page 44
Likely exam questions 45

CH 3 **Agile Software Development** 46

3.1 Rapid and agile development 46
3.2 Agile methods 48
3.3 Agile techniques: Extreme Programming 50
3.4 User stories for requirements 52
3.5 Refactoring 53
3.6 Test-first development 54
3.7 Pair programming 56
3.8 Agile project management and Scrum 56
3.9 Scaling agile & agile vs plan-driven in practice 59
Chapter 3 in one page 62
Likely exam questions 62

UNIT 2 Requirements Engineering & Design Concepts 64

CH 4 Requirements Engineering 65

4.1 Requirements, readers and stakeholders 65
4.2 Functional and non-functional requirements 69
4.3 Requirements engineering processes 76
4.4 Requirements elicitation 78
4.5 Requirements specification 85
4.6 Requirements validation 92
4.7 Requirements change 93
4.8 Lab: analysing a real-time case study 96
Chapter 4 in one page 98
Likely exam questions 99

CH 5 Design Concepts 101

5.1 Design within the context of software engineering 101
5.2 The design process 104
5.3 Design concepts 105
5.4 Design concepts at a glance 116
5.5 The design model 117
Chapter 5 in one page 122
Likely exam questions 123

UNIT 3 Architectural Design (CIE–1 part) 124

CH 6 Architectural Design: Decisions and Views 125

6.1 What is architectural design? 125
6.2 Example: a packing robot control system 127
6.3 Architectural abstraction: small and large 127
6.4 Advantages of explicit architecture 128
6.5 Architectural representations and their use 128
6.6 Architectural design decisions 130
6.7 Architecture reuse 131
6.8 Architecture and system characteristics 131
6.9 Architectural views 132
6.10 Representing architectural views 134
Chapter 6 in one page 135
Likely exam questions 136

EXAM Exam-Day Playbook 137

CIE-1 · UNIT 1

Unit 1: Software Process & Agile Development

1

What software engineering is, how software is built step by step (process models), and how modern teams build it in short iterations (agile, XP, Scrum).

SYLLABUS, WORD FOR WORD

Introduction: Professional software development, Software engineering ethics.

Software processes: Software process models, Process activities, Coping with change, Process improvement.

Agile Software Development: Agile methods, Agile development techniques, Agile project management.

CHAPTERS IN THIS UNIT

CHAPTER 1 — **Introduction to Software Engineering**
Professional software development · Software engineering ethics · Case studies — page 6

CHAPTER 2 — **Software Processes**
Software process models · Process activities · Coping with change · Process improvement — page 22

CHAPTER 3 — **Agile Software Development**
Agile methods · Extreme Programming & agile techniques · Agile project management (Scrum) · Scaling agile — page 46

COURSE OUTCOME

CO1: Explain professional software development practices, process models, ethical aspects, and apply agile principles and project management techniques in software engineering.

LAB COMPONENT

Comparative analysis of software process models on the real-time case study application.

CIE-1 SCOPE

Whole unit is in CIE-1.

UNIT 1 · CHAPTER 1

Introduction to Software Engineering

Professional software development · Software engineering ethics · Case studies

STUDY TIME
About 2.5 hours

SOURCES
Your slides (Ch 1, 57 slides) + Sommerville Ch 1

EXAM WEIGHT
High: definitions, attributes of good software, application types and the code of ethics are classic 2–10M questions

AFTER THIS CHAPTER YOU CAN

- Define software and software engineering
- Compare generic and customized products
- Explain the four essential attributes of good software
- Name the four software process activities
- Differentiate SE, computer science and system engineering
- Explain the issues that affect software and the 8 application types
- Describe how the web changed software engineering
- Explain professional responsibility and the 8 ACM/IEEE principles

1.1 Why software engineering matters ★★ IMPORTANT

Banks, power grids, phones, cars, hospitals, films and games all run on software. Because so much depends on it, building software is a serious professional job, and that job is **software engineering**.

- The **economies of ALL developed nations are dependent on software**. Infrastructure, utilities, manufacturing and the financial system are all computer-controlled.
- **More and more systems are software controlled**: most electrical products now contain a computer and controlling software.
- Software engineering is concerned with theories, methods and tools for professional software development.
- **Expenditure on software** is a significant fraction of **GNP** (Gross National Product, the total value a country produces) in all developed countries.

Software costs

- Software costs often **dominate computer system costs**. The software on a PC often costs more than the hardware.
- Software **costs more to maintain than it does to develop**. For systems with a long life, maintenance costs may be *several times* development costs.
- So software engineering is concerned with cost-effective software development.



Fig 1.1 The cost "iceberg": most cost comes after delivery (not to scale). Drawn for this guide

Why do software projects fail? ★★★ MUST KNOW

Many software projects still go wrong. Sommerville traces most "software failures" to two factors:

1. **Increasing system complexity**: new software engineering techniques let us build larger, more complex systems, and then the demands change. Systems have to be built and delivered more quickly, larger and even more complex systems are required, and systems need new capabilities that were previously thought impossible.
2. **Failure to use software engineering methods**: it is fairly easy to write programs without software engineering methods. Many companies have *drifted* into software development as their products and services evolved, and they do not use these methods in their everyday work. Consequently, their software is often **more expensive and less reliable** than it should be.

1.2 Professional software development ★★★ MUST KNOW

Lots of people write programs for themselves, but most software development is a **professional activity**: software built for business purposes, for inclusion in other devices, or as software products.

> ★ KEY POINT
> Professional software is intended for use by someone other than its developer, is usually built by **teams** rather than individuals, and is **maintained and changed throughout its life**. Software engineering supports this kind of development (not individual programming) with techniques for specification, design and evolution.

What is "software"? ★★★ MUST KNOW

> DEFINITION — LEARN THIS LINE
> **Software**: computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.

So software is **not just programs**. It also includes the **documentation, libraries, support websites and configuration data** that make them useful: separate programs, configuration files, **system documentation** (the system's structure), **user documentation** (how to use it) and product websites. A personal program needs none of this, because nobody else will use or change it.

Frequently asked questions about software engineering ★★★ MUST KNOW

This table (Sommerville Fig 1.1, slides 7–8) summarises the subject in ten answers. Each row is a ready-made 2-mark answer.

Question	Answer
What is software?	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
What are the attributes of good software?	Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
What is software engineering?	An engineering discipline that is concerned with all aspects of software production (textbook adds: "from initial conception to operation and maintenance").
What are the fundamental software engineering activities?	Software specification, software development, software validation and software evolution.
Software engineering vs computer science?	Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
Software engineering vs system engineering?	System engineering is concerned with all aspects of computer-based systems development, including hardware, software and process engineering. Software engineering is part of this more general process.
What are the key challenges facing software engineering?	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
What are the costs of software engineering?	Roughly 60% of software costs are development costs and 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are the best software engineering techniques and methods?	All projects must be professionally managed and developed, but different techniques suit different types of system. For example, games should always be developed using a series of prototypes, whereas safety-critical control systems require a complete and analyzable specification. You cannot say that one method is better than another.
What differences has the web made to software engineering?	It has led to the availability of software services and the possibility of developing highly distributed, service-based systems. Web-based development has led to important advances in programming languages and software reuse. (The textbook adds that the Internet created an "app" industry for mobile devices.)

1.3 Software products and product specification ★★★ MUST KNOW

Software engineers are concerned with developing **software products**, which is software that can be sold to a customer. There are two kinds:

	Generic products	Customized (bespoke) products
What	Stand-alone systems that are marketed and sold to **any customer** who wishes to buy them.	Software that is **commissioned by a specific customer** to meet their own needs; a contractor builds it especially for them.

	Generic products	Customized (bespoke) products
Examples	PC software such as graphics programs and project management tools; CAD software; software for specific markets such as appointments systems for dentists; mobile apps.	Embedded control systems, air traffic control software, traffic monitoring systems; systems supporting one particular business process.
Product specification	The specification of what the software should do is owned by the software developer, and decisions on software change are made by the developer.	The specification is owned by the customer for the software, and they make decisions on the software changes that are required.

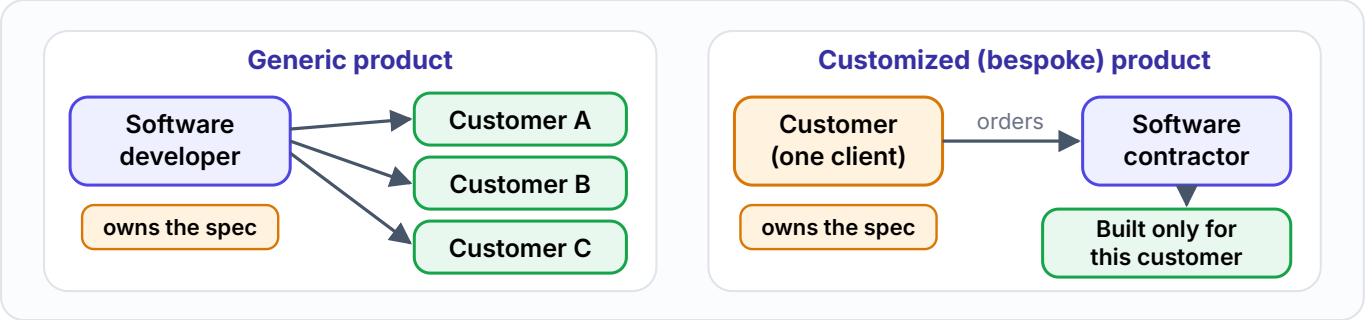


Fig 1.2 Generic vs customized products: the key difference is who owns the specification, and so who decides on changes. Drawn for this guide

Generic software is like a ready-made shirt: the maker decides the design, and can rethink it if development runs into problems. Customized software is like a tailor-made suit: the customer supplies the measurements (the specification) and the developers must work to them. The line is becoming **blurred**: many systems start from a generic product adapted for one customer, e.g. **ERP (Enterprise Resource Planning)** systems from SAP and Oracle, configured with a company's business rules, processes and reports.

1.4 Essential attributes of good software ★★★ MUST KNOW

Good software must first deliver the **required functionality and performance**. Beyond that, quality covers how it *behaves while running* and how its code and documents are *structured*. These are its **quality** or **non-functional attributes**, e.g. response time or how understandable the code is. The ones that matter most depend on the application: an aircraft control system must be **safe**, a game **responsive**, a telephone switch **reliable**. In general there are four:

Product characteristic	Description
Maintainability	Software should be written in such a way that it can **evolve to meet the changing needs of customers**. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics, including **reliability, security and safety**. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make **wasteful use of system resources** such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.

Product characteristic	Description
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be **understandable, usable and compatible** with other systems that they use.

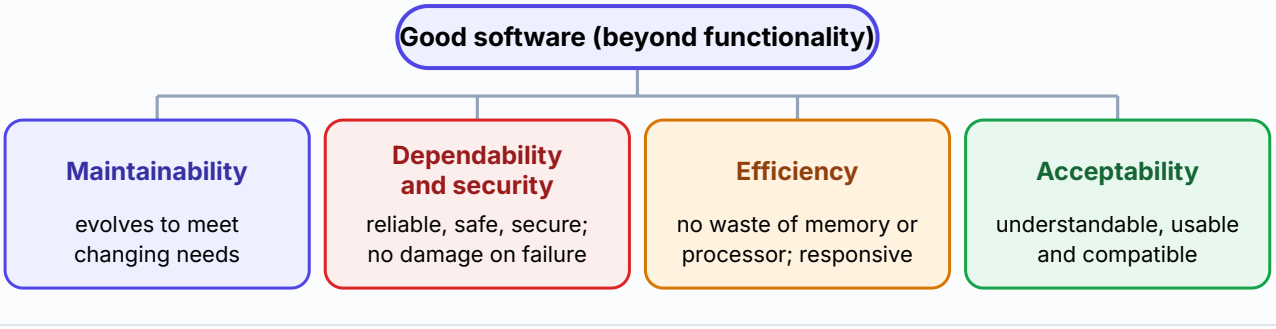


Fig 1.3 The four essential attributes of good software. Based on Sommerville Fig 1.2

MEMORY TRICK

M-D-E-A: "My Dog Eats Apples": **M**aintainability, **D**ependability & security, **E**fficiency, **A**cceptability. Example: for the **insulin pump**, dependability (safety, reliability) matters most; for **Mentcare**, security of patient records and acceptability to busy clinicians.

⚠ WATCH OUT

The FAQ says "maintainable, dependable and usable", but the expected list has **four names**: Maintainability, Dependability and security, Efficiency, Acceptability. Dependability covers reliability, security and safety.

1.5 What is software engineering? ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software engineering: an engineering discipline that is concerned with all aspects of software production, from the early stages of system specification through to maintaining the system after it has gone into use.

Key phrase 1: Engineering discipline

Using **appropriate theories and methods to solve problems**, bearing in mind **organisational and financial constraints**.

Key phrase 2: All aspects of software production

Not just the **technical process of development**. It also includes **project management** and developing **tools, methods and theories** to support software production.

Engineering means getting results of the **required quality within schedule and budget**, which often needs compromises. A systematic, organised approach is usually best, but a more flexible process suits web-based systems and mobile apps.

Importance of software engineering ★★★ MUST KNOW

1. **Society depends on software**: more and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
2. **It is cheaper in the long run** to use software engineering methods and techniques than to write programs as if it was a personal programming project. For most types of system, the majority of costs are the costs of **changing the software after it has gone into use**.

Software process activities ★★★ MUST KNOW

A **software process** is *a sequence of activities that leads to the production of a software product*. Four fundamental activities are common to all software processes:

1. **Software specification**: customers and engineers define the software that is to be produced and the constraints on its operation.
2. **Software development**: the software is designed and programmed.
3. **Software validation**: the software is checked to ensure that it is what the customer requires.
4. **Software evolution**: the software is modified to reflect changing customer and market requirements.

Systems organise these activities differently: aircraft software is **completely specified before development begins**, while e-commerce specifications and programs are **developed together** (Chapter 2).

> **MEMORY TRICK**
> **S-D-V-E: "Some Developers Value Evolution"**: Specification, Development, Validation, Evolution.

Software engineering vs computer science vs system engineering ★★★ MUST KNOW

- **Computer science** is concerned with the theories and methods that underlie computers and software (theory and fundamentals). It is essential background, as physics is for electrical engineers, but its theories rarely fit large, complex problems.
- **Software engineering** is concerned with the practical problems of producing software, i.e. developing and delivering useful software.
- **System engineering** is concerned with all aspects of developing and evolving complex systems where software plays a major role: hardware development, policy and process design, system deployment *and* software engineering. System engineers specify the system, define its overall architecture and integrate the parts. Software engineering is part of system engineering.

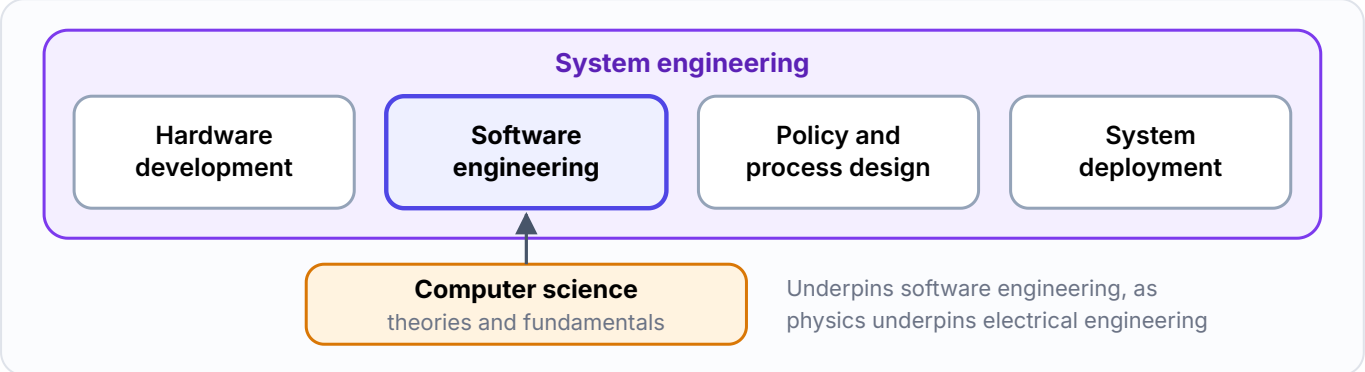


Fig 1.4 Software engineering is one part of system engineering and builds on computer science. Drawn for this guide from Sommerville §1.1.1

1.6 General issues that affect software ★★★ MUST KNOW

No method suits every type of software, but **four related issues** affect many different types:

Issue	Meaning, and the challenge it creates
Heterogeneity	Increasingly, systems are required to operate as **distributed systems across networks** that include different types of computer and mobile devices, and must often integrate with older **legacy systems** in other languages. *Challenge:* dependable software flexible enough to cope.
Business and social change	Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to **change their existing software** and **rapidly develop new software**. *Challenge:* traditional techniques are slow and must deliver value sooner.
Security and trust	As software is intertwined with all aspects of our lives, it is essential that we can **trust** that software, especially remote systems used through web pages or web services. *Challenge:* stop malicious attacks and keep information secure.
Scale	Software has to be developed across a very wide range of scales, from **very small embedded systems** in portable or wearable devices to **Internet-scale, cloud-based systems** that serve a global community.

MEMORY TRICK

"Hungry Bears Steal Snacks": **H**eterogeneity, **B**usiness & social change, **S**ecurity & trust, **S**cale. They match the FAQ's **key challenges**: coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.

1.7 Software engineering diversity and application types

★★★ MUST KNOW

- There are many different types of software system, and there is no universal set of software techniques that is applicable to all of them.
- The methods and tools used depend on the **type of application** being developed, the **requirements of the customer** and the **background of the development team**. The type of application is the most significant factor.

The eight application types ★★★ MUST KNOW

Type	What it is	Examples
1. Stand-alone applications	Run on a local computer such as a PC (or as mobile apps). They include all necessary functionality and **do not need a network**.	Office software, CAD programs, photo editors
2. Interactive transaction-based applications	Execute on a **remote computer** and are accessed by users from their own PCs, phones or terminals; often update a large data store in each transaction.	Web applications such as e-commerce; cloud mail
3. Embedded control systems	Software that **controls and manages hardware devices**. Numerically, there are probably more embedded systems than any other type.	Phone software, car anti-lock brakes, microwave oven, insulin pump

Type	What it is	Examples
4. Batch processing systems	Business systems that **process data in large batches**: large numbers of individual inputs create corresponding outputs.	Phone billing, salary payment
5. Entertainment systems	Primarily for **personal use**, intended to entertain the user. Quality of user interaction matters most.	Computer and console games
6. Systems for modelling and simulation	Developed by scientists and engineers to **model physical processes or situations** with many separate, interacting objects; often need high-performance parallel computers.	Scientific simulations of physical processes
7. Data collection systems	**Collect data from their environment using sensors** and send it to other systems for processing; often in hostile places (inside an engine, remote areas).	Wilderness weather station
8. Systems of systems	Systems **composed of a number of other software systems**, some generic (e.g. ERP), some specially written; used by large organisations.	A company-wide system built from an ERP package plus custom systems

MEMORY TRICK

"Some Indian Engineers Build Excellent Models During Summer": **S**tand-alone, **I**nteractive transaction-based, **E**mbedded control, **B**atch, **E**ntertainment, **M**odelling & simulation, **D**ata collection, **S**ystems of systems.

The boundaries are **blurred** (a phone game faces the same power limits as phone software), but each type needs **its own techniques**. A car's safety-critical **embedded system** is burned into ROM, so it needs extensive verification before sale; an **interactive web app** suits iterative delivery from reusable components; a **system of systems** needs its interactions specified in detail in advance.

Software engineering fundamentals ★★★ MUST KNOW

Some fundamental principles apply to **all types of software system**, whatever development techniques are used:

1. **Managed process**: systems should be developed using a managed and understood development process (different processes suit different types of software).
2. **Dependability and performance** are important for all types of system: no failures, available when needed, safe, secure and not wasteful.
3. **Requirements**: understanding and managing the software specification and requirements (what the software should do) are important.
4. **Reuse**: where appropriate, reuse software that has already been developed rather than write new software.

MEMORY TRICK

"Proper Design Requires Reuse": Process, Dependability (& performance), Requirements, Reuse.

1.8 Internet (web-based) software engineering ★★ IMPORTANT

- The **Web is now a platform for running applications**, and organisations are increasingly developing web-based systems rather than local systems.
- **Web services** allow application functionality to be accessed over the web.
- **Cloud computing** is an approach to providing computer services where applications run remotely on the "cloud" (a huge number of linked computers shared by many users). Users do not buy software but pay according to use.

Before the web, business software was **monolithic** (single programs on single computers); now it is deployed on web servers, delivered as **software as a service** (e.g. Google Apps, Office 365), highly distributed and built by reuse. Four changes followed:

Change	Meaning
Software reuse	Reuse is the **dominant approach** for constructing web-based systems: you think about how to assemble them from pre-existing software components and systems.
Incremental and agile development	Web-based systems should be developed and delivered **incrementally**; it is impractical to specify all their requirements in advance.
Service-oriented systems	Software may be built using service-oriented software engineering, where the components are **stand-alone web services**.
Rich interfaces	Interface technologies such as **AJAX and HTML5** support rich interfaces within a web browser.

Web-based systems are complex distributed systems, but the fundamental ideas of software engineering apply to them in the same way as to any other type of system. **Memory trick:** "Reuse It, Serve it Rich" = **R**euse, **I**ncremental, **S**ervice-oriented, **R**ich interfaces.

1.9 Software engineering ethics ★★★ MUST KNOW

- Software engineering involves **wider responsibilities** than simply the application of technical skills.
- Software engineers must behave in an **honest and ethically responsible** way if they are to be respected as professionals.
- Ethical behaviour is more than simply upholding the law; it involves following a set of principles that are morally correct.

Issues of professional responsibility ★★★ MUST KNOW

In these areas, acceptable behaviour is set by **professional responsibility** rather than by laws:

Issue	What the engineer should do	Example
Confidentiality	Respect the confidentiality of employers or clients, **whether or not a formal confidentiality agreement** has been signed.	Not discussing a client's unreleased product
Competence	Do not misrepresent your level of competence or **knowingly accept work outside your competence**.	Not posing as a security expert to win a contract
Intellectual property rights	Know the local laws on **patents, copyright, etc.** and protect the intellectual property of employers and clients.	Not reusing one client's code for another

Issue	What the engineer should do	Example
Computer misuse	Do not use technical skills to misuse other people's computers. Misuse ranges from **trivial** (game playing on an employer's machine) to **extremely serious** (dissemination of viruses).	Not reading others' files with admin access

MEMORY TRICK

"Careful Coders Inspect Code": **C**onfidentiality, **C**ompetence, **I**ntellectual property rights, **C**omputer misuse.

The ACM/IEEE Code of Ethics ★★★ MUST KNOW

- The professional societies in the US (**ACM and IEEE**) have cooperated to produce a code of ethical practice: the *Software Engineering Code of Ethics and Professional Practice*. Other bodies, such as the British Computer Society, publish similar codes.
- Members of these organisations **sign up to the code** of practice when they join.
- The Code contains **eight Principles** related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students.

Rationale: computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society. Software engineers take part (directly or by teaching) in analysing, specifying, designing, developing, certifying, maintaining and testing software, so they have significant opportunities to **do good or cause harm**, or to enable or influence others to. To ensure their efforts are used for good, they must commit to making software engineering a beneficial and respected profession.

Preamble: the **short version** states **aspirations** at a high level; the full version's **clauses** give details. Without aspirations, details become legalistic and tedious; without details, aspirations become high-sounding but empty; together they form a cohesive code. Committed to the **health, safety and welfare of the public**, software engineers shall adhere to eight principles:

Principle	Exact wording (learn this)	In plain words
1. PUBLIC	Software engineers shall act consistently with the public interest.	Public safety first; never hide a harmful defect
2. CLIENT AND EMPLOYER	Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.	Be loyal to them, but not at the public's expense
3. PRODUCT	Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.	Test properly; never ship poor work knowingly
4. JUDGMENT	Software engineers shall maintain integrity and independence in their professional judgment.	Honest opinions; don't sign what you haven't checked
5. MANAGEMENT	Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.	Realistic plans; no pressure to cut ethical corners
6. PROFESSION	Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.	Act so people trust software engineers

Principle	Exact wording (learn this)	In plain words
7. COLLEAGUES	Software engineers shall be fair to and supportive of their colleagues.	Credit others' work; help newcomers
8. SELF	Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.	Keep learning; practise ethics yourself

MEMORY TRICK

Take the principles in pairs: **PC · PJ · MP · CS**, "A **PC** in **PJ**s meets an **MP** from **CS**". That gives **P**ublic–**C**lient, **P**roduct–**J**udgment, **M**anagement–**P**rofession, **C**olleagues–**S**elf. Each pair is one verb: whom you **serve**, what you **deliver**, how you **lead**, how you **grow**.

Ethical dilemmas ★★ IMPORTANT

An **ethical dilemma** is a situation with no clearly right choice, common when people have different views and objectives. The slides give three examples:

1. **Disagreement in principle with the policies of senior management.** Argue your case from within, or resign on principle? When should you report project problems? Too early may be overreacting; too late may make them impossible to fix.
2. **Your employer acts unethically and releases a safety-critical system without finishing the testing** (e.g. falsifies safety validation records). Keep confidentiality, or alert the customer or the public? There are **no absolutes in safety**: disclosure may harm the employer and colleagues, silence may harm others. Weigh the potential damage, its extent and who is affected; go public only if it is very dangerous, and first try to resolve it while respecting the employer's rights.
3. **Participation in the development of military weapons systems or nuclear systems.** Views differ, so **employers and employees should make their views known to each other in advance**.

✎ EXAM TIP

For a 10M "code of ethics" answer: what the code is and who it covers → rationale (2–3 lines) → preamble idea (aspirations + details) → all **eight principles** in their wording → one dilemma. If the question says "professional responsibility", add confidentiality, competence, IPR and computer misuse.

1.10 Case studies used throughout the course ★★ IMPORTANT

Sommerville uses **four different types of system** as examples, because software engineering practice depends on the type of system. Slide 2 says three, but slide 36 introduces four, and later chapters keep returning to all of them.

1. Insulin pump control system ★★ IMPORTANT

An **embedded system** in an insulin pump used by diabetics to maintain blood glucose control. In **diabetes** the pancreas cannot make enough **insulin**; the pump **simulates the pancreas**:

- It **collects data from a blood sugar sensor** and calculates the amount of insulin required to be injected.
- The calculation is based on the **rate of change of blood sugar levels**.
- It **sends signals to a micro-pump** to deliver the correct dose (one pulse = one unit of insulin).
- It is safety-critical: **low** blood sugar can lead to brain malfunctioning, coma and death; **high** blood sugar has long-term consequences such as eye and kidney damage.

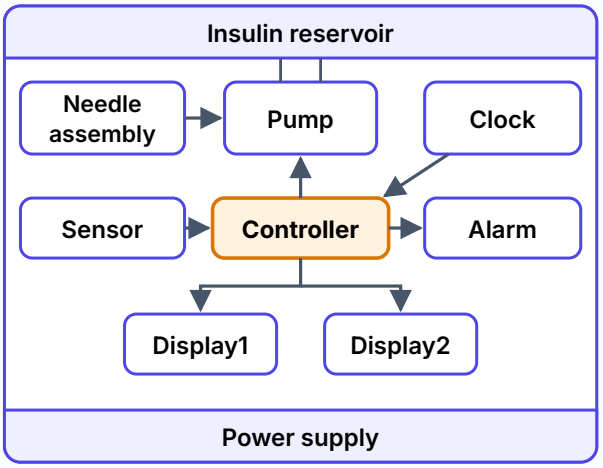


Fig 1.5 Insulin pump hardware architecture; the controller runs the software. Sommerville Fig 1.4 · Your slides, slide 38

A sensor measures a blood parameter (electrical conductivity) that depends on the sugar level; the **controller** works out the dose and pulses the pump. The UML **activity model** shows how the software turns a sensor reading into pump commands:

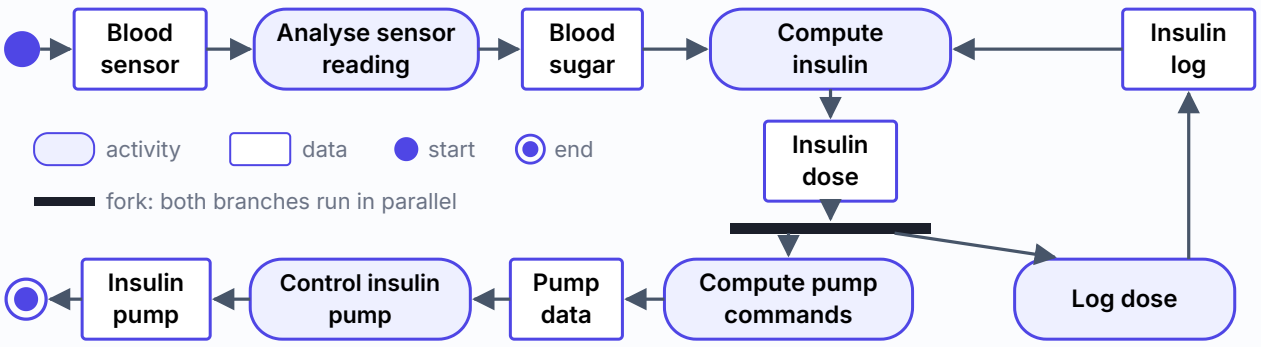


Fig 1.6 Activity model of the insulin pump (UML): sensor → sugar → dose → pump commands, while the dose is also logged. Sommerville Fig 1.5 · Your slides, slide 39

Essential high-level requirements: (1) the system shall be **available** to deliver insulin when required; (2) the system shall perform **reliably** and deliver the correct amount of insulin to counteract the current level of blood sugar. The system must therefore be designed and implemented to ensure that it always meets these requirements.

2. Mentcare: a patient information system for mental health care ★★ IMPORTANT

- A **medical information system** that maintains information about patients suffering from mental health problems and the treatments they have received.
- Most patients do not need hospital treatment but regularly attend **specialist clinics**, which are also held in **local medical practices or community centres**.
- Intended for use in clinics, it uses a **centralised database** of patient information but can also run on a PC at sites without secure network connectivity.
- With secure network access it uses the database; when disconnected it can **download and use local copies** of patient records.

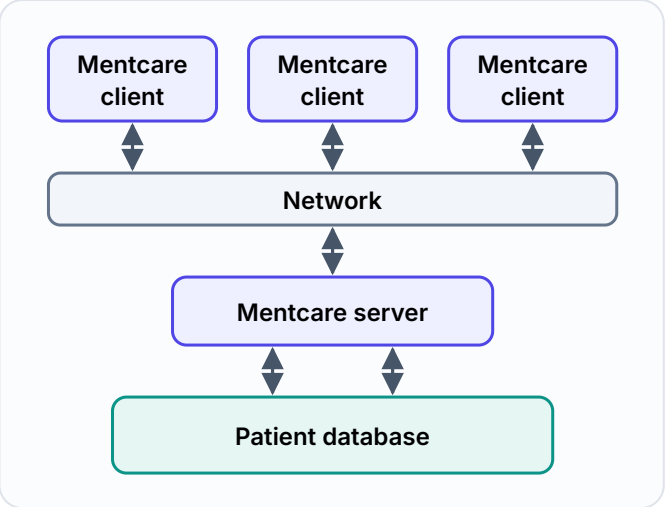


Fig 1.7 Organisation of the Mentcare system. Sommerville Fig 1.6 · Your slides, slide 44

Goals: (1) to generate **management information** that allows health service managers to assess performance against local and government targets; (2) to provide **medical staff** with timely information to support the treatment of patients.

Key features: individual care management: clinicians create, edit and view patient records and history, and data summaries let doctors quickly learn the key problems and prescribed treatments. **Patient monitoring**: the system monitors the records of patients in treatment and issues warnings if possible problems are detected. **Administrative reporting**: monthly management reports on patients treated at each clinic, patients entering and leaving the care system, patients *sectioned* (confined to a secure hospital), and drugs prescribed and their costs.

Concerns: privacy: patient information must be confidential and never disclosed to anyone apart from authorised medical staff and the patient. **Safety**: some patients may become suicidal or a danger to others, so the system should warn medical staff wherever possible. It must also be **available** when needed, or safety may be compromised and correct medication may be impossible to prescribe.

3. Wilderness weather station ★★ IMPORTANT

- A **data collection system** that collects data about weather conditions in remote areas. The government of a country with large areas of wilderness deploys **several hundred** such stations.
- The instruments measure **temperature and pressure, sunshine, rainfall, wind speed and wind direction** (ground and air temperature, barometric pressure, 24-hour rainfall).
- Each instrument is controlled by software that **takes parameter readings periodically** and manages the data collected.

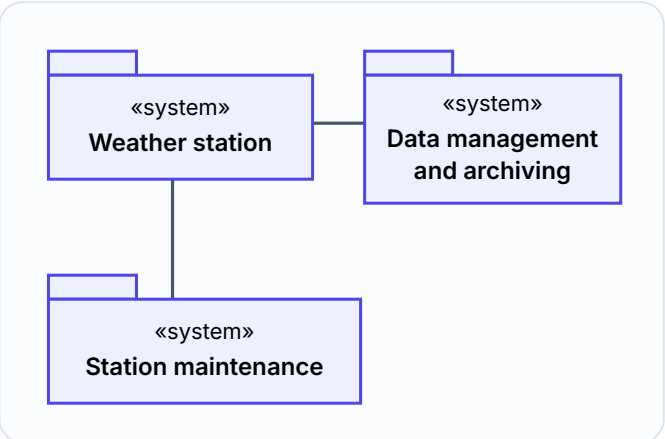


Fig 1.8 The weather station's environment (UML packages with the «system» stereotype). Sommerville Fig 1.7 · Your slides, slide 48

The station belongs to a larger **weather information system** of three systems that exchange information (Fig 1.8). **The weather station system** collects weather data, carries out some initial data processing and transmits it to the data management system. **The data management and archiving system** collects the data from all of the wilderness weather stations, carries out data processing and analysis, and archives the data. **The station maintenance system** communicates by satellite with all wilderness weather stations to monitor their health and provide reports of problems.

Stations are unattended and battery-powered, so the software also needs **additional functionality**: (1) **monitor** the instruments, power and communication hardware and report faults to the management system; (2) **manage the system power**, charging batteries whenever environmental conditions permit but shutting down generators in potentially damaging weather such as high wind; (3) support **dynamic reconfiguration**, where parts of the software are replaced with new versions and backup instruments are switched in after a system failure.

4. iLearn: a digital learning environment ★★ IMPORTANT

- A system to **support learning in schools**: a framework in which general-purpose and specially designed learning tools may be embedded, plus applications geared to the learners' needs.
- The tools are **chosen by teachers and learners**: general applications such as spreadsheets, learning management applications such as a **Virtual Learning Environment (VLE)** for homework submission and assessment, games and simulations.
- It is a service-oriented system: every component is a **replaceable service**. So it can be **updated incrementally** as new services become available, and **rapidly configured** for groups such as very young children who cannot read, or senior students.
- **Services: utility services** give basic application-independent functionality used by other services; **application services** give specific applications (email, conferencing, photo sharing) and educational content (scientific films, historical resources); **configuration services** adapt the environment with a set of application services and define how services are shared between students, teachers and parents.

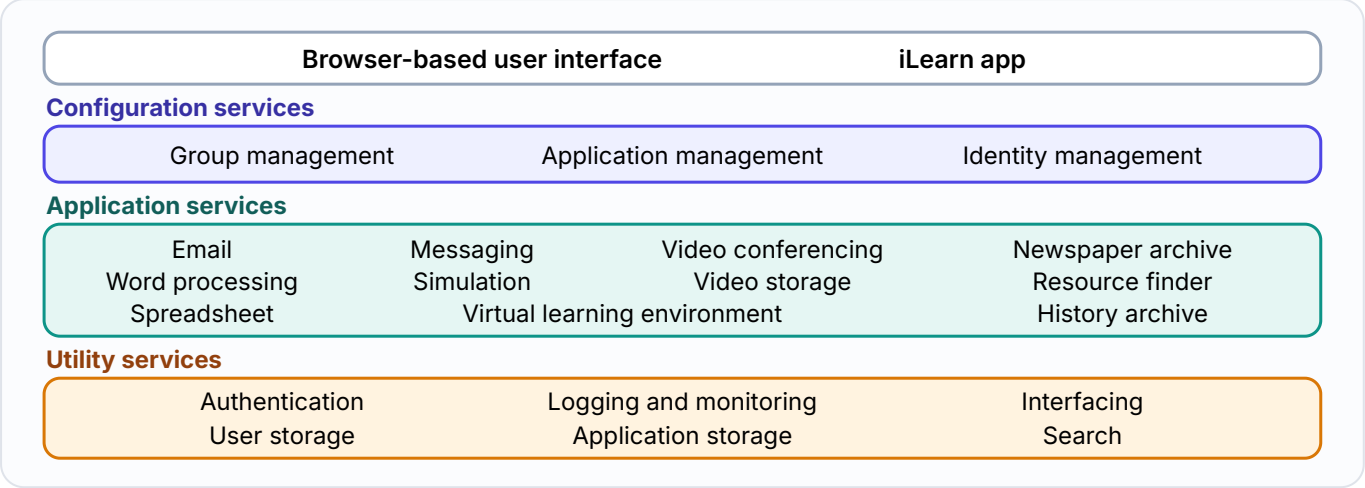


Fig 1.9 The iLearn architecture: user interface, then configuration, application and utility services. Sommerville Fig 1.8 · Your slides, slide 54

Service integration happens at two levels. **Integrated services** offer an **API** (application programming interface) that other services can call, so **direct service-to-service communication** is possible (e.g. authentication). **Independent services** are simply accessed through a **browser interface** and operate independently; information is shared only through user actions such as **copy and paste**, and **re-authentication** may be needed for each one.

Chapter 1 in one page

Definitions

- **Software**: computer programs and associated documentation (plus libraries, support websites, configuration data).
- **Software engineering**: an engineering discipline concerned with all aspects of software production, from early system specification to maintaining the system in use. Key phrases: *engineering discipline* and *all aspects of software production*.
- **Software process**: a sequence of activities that leads to the production of a software product.
- **Generic product**: sold to anyone; developer owns the spec. **Customized (bespoke)**: commissioned by one customer, who owns the spec.

Why SE matters

- Economies depend on software; maintenance costs more than development.
- Failures come from **increasing complexity** and **not using SE methods**.
- Important because society relies on software, and SE is **cheaper in the long run**.

SE vs CS vs system engineering

- CS = theory and fundamentals. SE = practicalities of producing useful software. System engineering = hardware + software + policy/process design + deployment; SE is part of it.

Diagrams to practise

- Insulin pump hardware (Fig 1.5) and activity model (Fig 1.6)
- Mentcare organisation, weather station environment, iLearn architecture (Figs 1.7–1.9)

Lists to learn (with memory tricks)

- **4 attributes** (My Dog Eats Apples): Maintainability, Dependability & security, Efficiency, Acceptability.
- **4 process activities** (S-D-V-E): Specification, Development, Validation, Evolution.
- **4 general issues** (Hungry Bears Steal Snacks): Heterogeneity, Business & social change, Security & trust, Scale.
- **8 application types** (Some Indian Engineers Build Excellent Models During Summer): Stand-alone, Interactive transaction-based, Embedded control, Batch processing, Entertainment, Modelling & simulation, Data collection, Systems of systems.
- **4 fundamentals** (Proper Design Requires Reuse): managed Process, Dependability & performance, Requirements, Reuse.
- **4 web changes** (Reuse It, Serve it Rich): Reuse, Incremental & agile, Service-oriented, Rich interfaces (AJAX, HTML5).
- **4 responsibility issues** (Careful Coders Inspect Code): Confidentiality, Competence, Intellectual property rights, Computer misuse.
- **8 ACM/IEEE principles** (PC · PJ · MP · CS): Public, Client & employer, Product, Judgment, Management, Profession, Colleagues, Self.
- **3 ethical dilemmas**: disagreeing with senior management; employer releasing an untested safety-critical system; military or nuclear work.
- **4 case studies**: insulin pump (embedded), Mentcare (information system), weather station (data collection), iLearn (service-oriented learning environment).

Likely exam questions

2M **Define software and software engineering. State the two key phrases of the definition.**

How to answer → Both definitions word for word (§1.2, §1.5) → "engineering discipline" and "all aspects of software production", one line each.

4M **Differentiate between generic and customized software products, with examples.**

How to answer → Table from §1.3: what, examples, who owns the specification and decides on changes; mention ERP blurring.

6M **Explain the essential attributes of good software.**

How to answer → "Required functionality and performance" first → the four attributes with textbook descriptions → Fig 1.3 → an example each.

4M **What are the fundamental software process activities? Explain each.**

How to answer → Define software process → specification, development, validation, evolution, one line each.

5M **Differentiate software engineering from computer science and from system engineering.**

How to answer → Three short definitions + Fig 1.4: CS = theory; SE = practicalities; SE is part of system engineering.

4M **Why is software engineering important? Why do software projects fail?**

How to answer → Two reasons for importance + two failure causes (increasing complexity; not using SE methods).

6M **Explain the general issues that affect most types of software.**

How to answer → Heterogeneity, business and social change, security and trust, scale: meaning + challenge for each (§1.6).

10M **Explain the types of software applications with examples. Why do they need different techniques?**

How to answer → All eight types with an example → blurred boundaries → car vs web app vs system of systems → the four fundamentals.

6M **How has the web changed software engineering?**

How to answer → Web platform, web services, cloud (pay per use) → reuse, incremental/agile, service-oriented, rich interfaces.

10M **Explain the ACM/IEEE Code of Ethics and the issues of professional responsibility. Give an ethical dilemma.**

How to answer → Four responsibility issues → code: who, rationale, preamble → eight principles in their wording → one dilemma (§1.9).

8M **Describe the insulin pump system with its hardware architecture and activity model. State its essential requirements.**

How to answer → Purpose + why safety-critical → Fig 1.5 → Fig 1.6 → the two high-level requirements.

6M **Explain the Mentcare system (or the weather station, or iLearn) as a case study.**

How to answer → Mentcare: Fig 1.7, goals, features, concerns. Weather station: Fig 1.8, three systems, extra functions. iLearn: services, Fig 1.9, integration.

UNIT 1 · CHAPTER 2

Software Processes

Software process models · Process activities · Coping with change · Process improvement

STUDY TIME
About 4 hours (2 reading + 2 practising the diagrams)

SOURCES
Your Ch 2 slides (58) + Sommerville Ch 2 + Pressman Ch 2 (process models)

EXAM WEIGHT
Very high: "explain with a neat diagram" and "compare the models" are standard questions

AFTER THIS CHAPTER YOU CAN

- Define a software process and its four basic activities
- Compare plan-driven and agile processes
- Draw and explain waterfall, incremental, reuse, V, prototyping and spiral models
- Draw the requirements, design, testing and evolution diagrams
- Explain prototyping and incremental delivery as ways to cope with change
- Tell incremental development and incremental delivery apart
- Explain the improvement cycle and the five CMM levels
- Choose and justify a process model for a case study

2.1 The software process ★★★ MUST KNOW

Every piece of software is produced by people following some sequence of steps: find out what is needed, decide how to build it, build it, check it, and keep changing it afterwards. That organised sequence of steps is the **software process**.

> **DEFINITION — LEARN THIS LINE**
>
> **Software process** — a structured set of activities required to develop a software system. (Textbook: a set of related activities that leads to the production of a software system.)

The four fundamental activities ★★★ MUST KNOW

There are many different software processes, but all of them involve these four activities:

Activity	What it means (slide wording)
1. Specification	Defining what the system should do (its functionality and the constraints on its operation).
2. Design and implementation	Defining the organisation of the system and implementing the system (textbook: *software development*).
3. Validation	Checking that it does what the customer wants.
4. Evolution	Changing the system in response to changing customer needs.

Each activity has sub-activities (e.g. requirements validation, architectural design, unit testing), and processes also contain supporting activities such as configuration management and project planning.

MEMORY TRICK

S-D-V-E — "**S**ome **D**evelopers **V**alue **E**volution": Specification, Design & implementation, Validation, Evolution.

Software process models and process descriptions ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software process model — an abstract representation of a process. It presents a description of a process from some particular perspective.

A process model is also called a **Software Development Life Cycle (SDLC) model**; the very general models are called **process paradigms**. Being a simplified view from one perspective, a model shows only *part* of the process (e.g. an activity model shows activities and their order, but not who does them). Companies adapt these general frameworks into their own processes.

When we describe a process we usually talk about its **activities** (e.g. specifying a data model, designing a user interface) and the **ordering** of these activities. A process description may also include:

Element	Meaning	Example
Products	The outcomes of a process activity.	Architectural design produces a model of the software architecture.
Roles	The responsibilities of the people involved in the process.	Project manager, configuration manager, programmer.
Pre- and post-conditions	Statements that are true before and after a process activity has been enacted or a product produced.	Pre: customer has approved all requirements. Post: the architecture models have been reviewed.

Plan-driven and agile processes ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Plan-driven processes — processes where all of the process activities are planned in advance and progress is measured against this plan.

Agile processes — planning is incremental, and it is easier to change the process to reflect changing customer requirements.

Point	Plan-driven	Agile
Planning	All activities planned in advance; progress measured against the plan	Incremental and continual as the software is developed
Change	Costly: plans and documents must be redone	Easy to change the process to follow the customer
Requirements	Complete specification before design starts	Specified informally for each increment just before it is built

Point	Plan-driven	Agile
Documents & testing	Detailed design documents; testing driven by test plans and an independent test team	Minimum documentation; design kept in the code; tests written with the requirements
Suits	Safety-critical, embedded, large multi-site systems (e.g. waterfall, V-model)	Business systems and products with rapidly changing requirements (Chapter 3)

★ KEY POINT

There are no right or wrong software processes — no single process suits every system. The choice depends on the type of software, the customer's requirements and the developers' skills: a safety-critical system needs a very structured process with detailed records; a business system with fast-changing requirements suits a flexible agile process. In practice, most practical processes include elements of both plan-driven and agile approaches.

2.2 Software process models (Sommerville) ★★★ MUST KNOW

Model	Key idea (slide wording)	Plan-driven or agile?
The waterfall model	Separate and distinct phases of specification and development.	Plan-driven
Incremental development	Specification, development and validation are interleaved.	May be plan-driven or agile
Integration and configuration	The system is assembled from existing configurable components.	May be plan-driven or agile

In practice, **most large systems are developed using a process that incorporates elements from all of these models**: well-understood parts are built waterfall-style or bought and configured, hard-to-specify parts are developed incrementally, and components are reused throughout (the architecture, though, must be planned up front). Typically, safety-critical software uses waterfall, software products are incremental, and business systems are configured from existing systems. Read once: the **Rational Unified Process (RUP)** tries to combine all the general models; its phases are inception, elaboration, construction and transition.

The waterfall model ★★★ MUST KNOW

IN SIMPLE WORDS

Water in a waterfall falls from one ledge to the next and never flows back up. Each phase "cascades" into the next: finish and sign off the requirements, then design, then code, then test, then deliver.

The first published process model (Royce, 1970), taken from large military systems engineering; also called the **software life cycle**. It is **plan-driven**: you plan and schedule all activities before starting development.

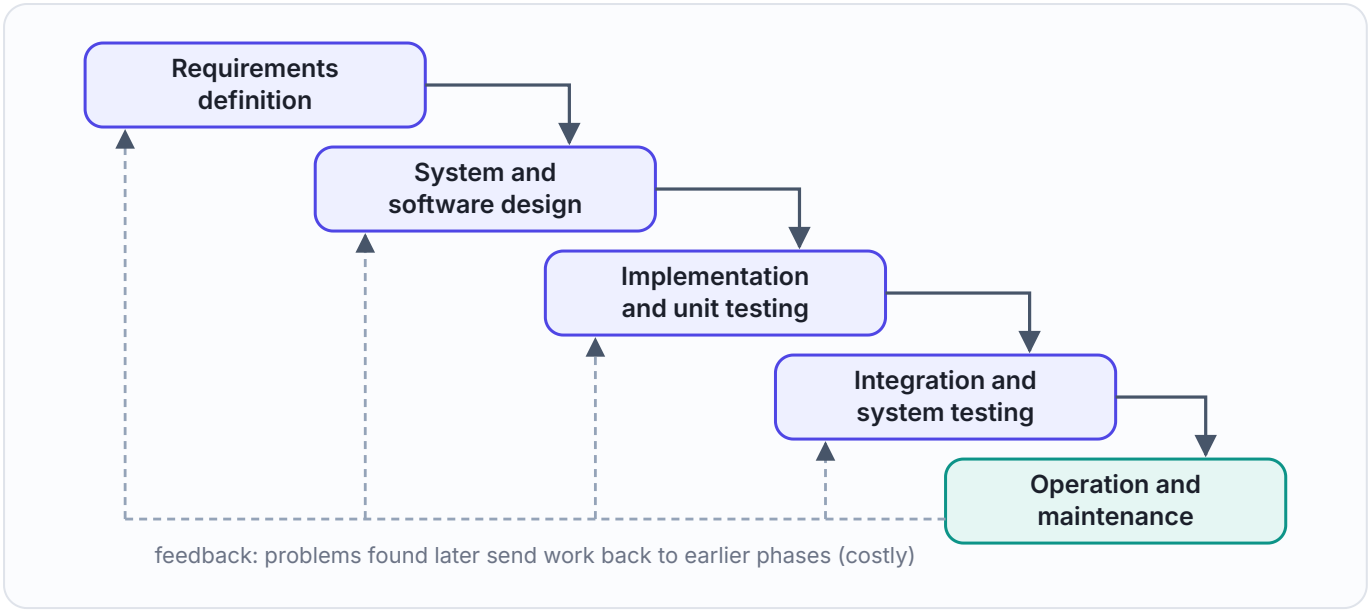


Fig 2.1 The waterfall model. Each phase must finish (and be signed off) before the next begins. Sommerville Fig 2.1 · Your slides, slide 8

1. **Requirements analysis and definition** — the system's services, constraints and goals are established with users and defined in detail as the system specification.
2. **System and software design** — requirements are allocated to hardware or software and an overall architecture is established; software design identifies the fundamental software abstractions and their relationships.
3. **Implementation and unit testing** — the design is realised as programs or program units; each unit is verified against its specification.
4. **Integration and system testing** — units are integrated and tested as a complete system against the requirements, then delivered to the customer.
5. **Operation and maintenance** — normally the *longest* phase: the system is used; maintenance corrects errors not found earlier, improves the implementation and adds new services.

The result of each phase is one or more approved ("*signed-off*") documents, and a phase has to be complete before moving onto the next. In real software, phases overlap and feed information back (dashed lines in Fig 2.1). The main drawback is the difficulty of accommodating change after the process is underway.

> **MEMORY TRICK**
>
> **R-S-I-I-O** — "**R**ain **S**lides **I**nto **I**cy **O**ceans": Requirements, System design, Implementation, Integration, Operation.

When to use it: embedded systems (hardware is inflexible, so software decisions can't be delayed); critical systems (need safety and security analysis of a complete specification and design); and **large systems engineering projects developed at several sites** or by partner companies, where its plan-driven nature helps coordinate the work. Avoid it when requirements change quickly and the team can communicate informally. Read once: **formal system development** (e.g. the B method) is a waterfall variant that transforms a mathematical specification into code; used only for very critical systems.

✎ EXAM TIP

10 marks: diagram with feedback lines → one line per phase → "plan-driven, signed-off documents" → 3 advantages, 3 problems → when appropriate, with an example (insulin pump).

ADVANTAGES

- ✓ Simple, systematic; easy to plan and manage
- ✓ Signed-off documents make progress visible
- ✓ Plan-driven nature coordinates work at several sites
- ✓ Complete specification allows safety/security analysis

PROBLEMS

- ✗ Inflexible partitioning into distinct stages makes it difficult to respond to changing customer requirements
- ✗ Only appropriate when requirements are well understood and changes limited — few business systems have stable requirements
- ✗ Requirements get frozen too early, so the system may not do what users want
- ✗ Pressman: customers can't state all requirements up front; no working version until late; team members wait in "blocking states"

Incremental development ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Incremental development — develop an initial implementation, get feedback from users, and evolve the software through several versions until the required system is developed. Specification, development and validation are interleaved, with rapid feedback across activities.

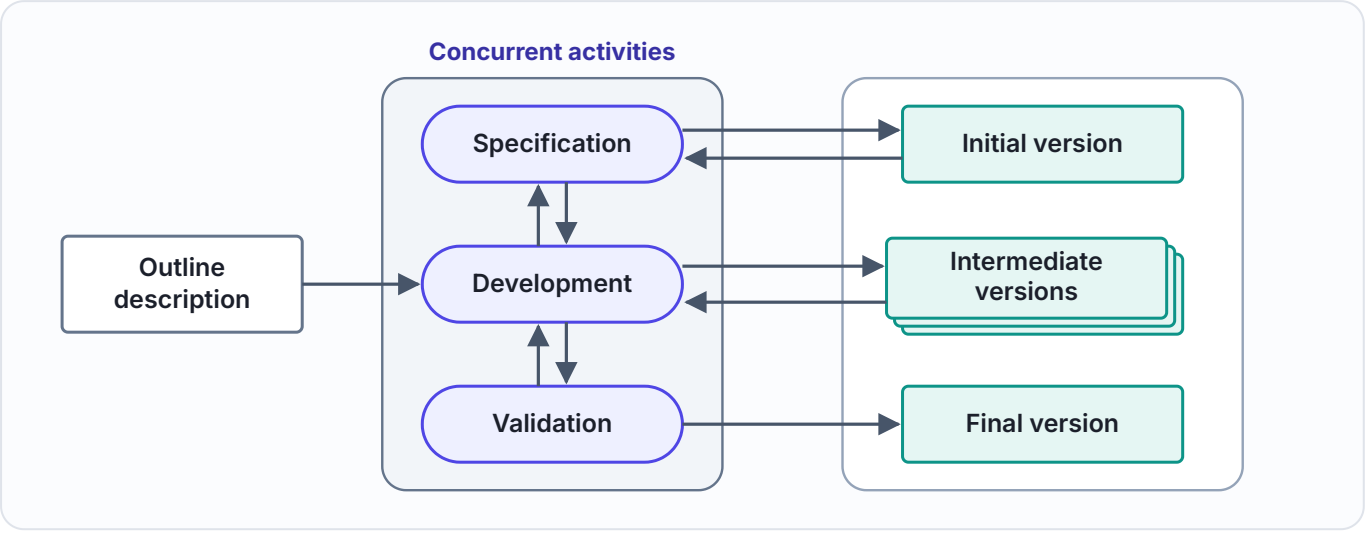


Fig 2.2 Incremental development: activities run concurrently and produce a series of versions. Sommerville Fig 2.2 · Your slides, slide 11

Now the **most common approach** for application systems and software products, and the basis of agile methods. It can be plan-driven (increments identified in advance), agile (later increments depend on progress and priorities) or a mix. Early increments hold the **most important or urgent functionality**, so the customer evaluates the system early; if it is wrong, only the current increment changes.

IN SIMPLE WORDS

Like writing an essay in drafts: write a rough version, show it, improve it, show it again — each draft (increment) adds to the last.

BENEFITS

- ✓ **Cost of accommodating changing requirements is reduced** — far less analysis and documentation to redo than with waterfall.
- ✓ **Easier to get customer feedback** — customers comment on demonstrations and see how much has been implemented.
- ✓ **More rapid delivery and deployment** of useful software — customers gain value earlier than with waterfall.

PROBLEMS

- ✗ **The process is not visible** — managers need regular deliverables to measure progress; documenting every quickly-built version is not cost-effective.
- ✗ **System structure tends to degrade** as increments are added. Unless time and money is spent on *refactoring*, regular change corrupts the structure and further changes become difficult and costly.

These problems are worst for large, long-lifetime systems built by several teams, which need a stable architecture planned in advance.

⚠ WATCH OUT

Incremental **development** does not mean every increment is delivered to the customer — versions may only be *shown* for comment. Deploying increments for real use is **incremental delivery** (Section 2.5.2).

Integration and configuration (reuse-oriented SE) ★★★ MUST KNOW

IN SIMPLE WORDS

Like furnishing a room with ready-made furniture you assemble and adjust, instead of making every piece from raw wood.

- Based on **software reuse**: systems are integrated from existing components or application systems, sometimes called **COTS (commercial-off-the-shelf)** systems.
- Reused elements may be **configured** to adapt their behaviour and functionality to a user's requirements.
- Reuse is now the **standard approach for building many types of business system** (covered in depth in Sommerville Chapter 15).

Types of reusable software: (1) **stand-alone application systems** (COTS) configured for a particular environment; (2) **collections of objects** developed as a package to be integrated with a component framework such as .NET or J2EE (textbook: Java Spring); (3) **web services** developed to service standards and available for remote invocation.

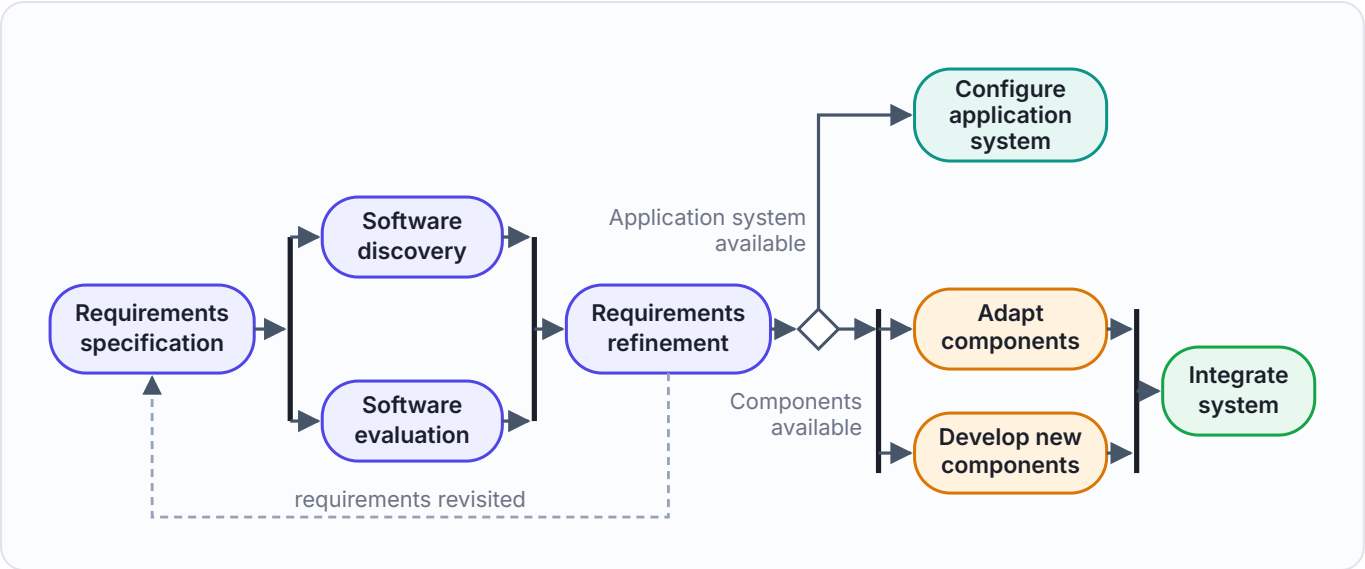


Fig 2.3 Reuse-oriented software engineering. Thick bars split or join parallel paths; the diamond is a decision. Sommerville Fig 2.3 · Your slides, slide 16

1. **Requirements specification** — brief initial requirements: essential requirements and desirable features.
2. **Software discovery and evaluation** — search for components/systems providing the functionality and evaluate whether they meet the essential requirements.
3. **Requirements refinement** — modify the requirements to fit the components found; if impossible, search again (so requirements engineering is repeated).
4. **Application system configuration** — if an off-the-shelf system meets the requirements, configure it to create the new system.
5. **Component adaptation and integration** — otherwise, modify reusable components, develop new ones, and integrate them.

ADVANTAGES

- ✓ Reduced costs and risks as less software is developed from scratch
- ✓ Faster delivery and deployment of the system

DISADVANTAGES

- ✗ Requirements compromises are inevitable, so the system may not meet the real needs of users
- ✗ Loss of control over evolution of reused system elements

EXAMPLE

iLearn (digital learning environment for schools) is built this way: its application services (email, conferencing, spreadsheets, a virtual learning environment) are bought or freely available, and configuration services adapt it to each school.

2.3 More process models (from Textbook 2, Pressman)

★★★ MUST KNOW

These models are not in Sommerville's chapter or your slides (except the V-model, slide 30) but are commonly examined. Pressman calls them **prescriptive process models**, and labels his diagrams with five generic **framework activities**: **Communication** (requirements gathering), **Planning** (estimating, scheduling,

tracking, risk analysis), **Modeling** (analysis, design), **Construction** (code, test) and **Deployment** (delivery, support, feedback). His waterfall ("classic life cycle") is these five in sequence.

The V-model ★★★ MUST KNOW

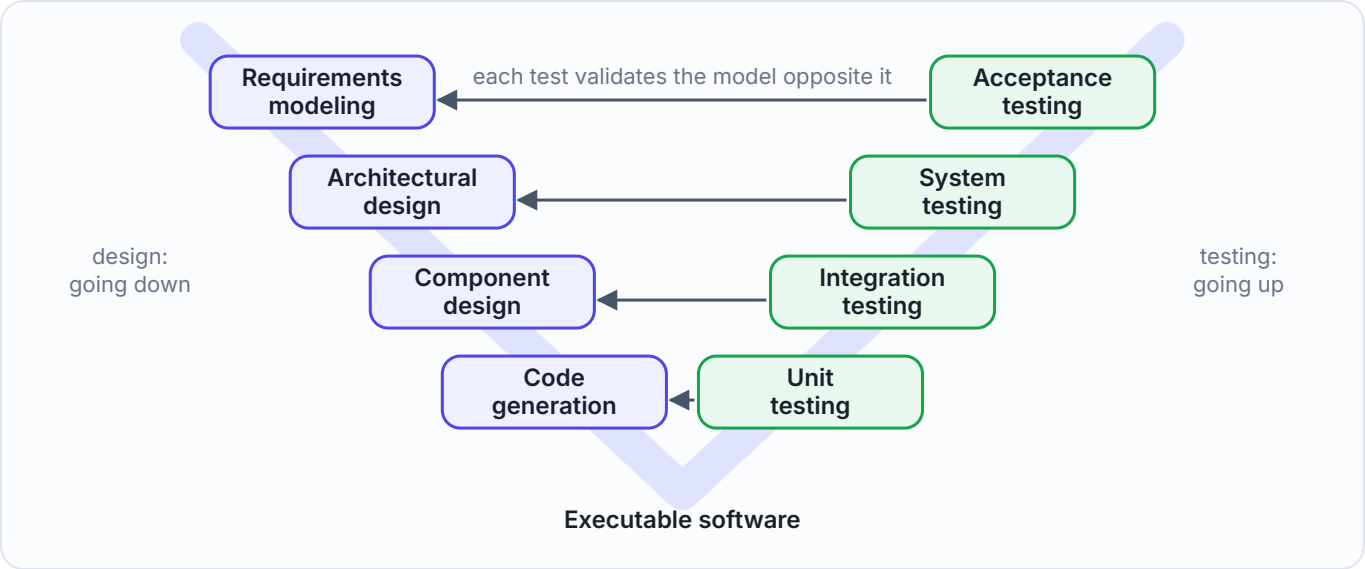


Fig 2.4 The V-model as drawn by Pressman. Pressman Fig 2.4 (From Textbook 2)

- A **variation of the waterfall model** showing how quality-assurance (verification and validation) actions relate to the earlier communication, modeling and construction actions.
- Going **down the left side**, requirements are refined into more detailed technical representations; after code is generated, the team goes **up the right side**, running tests that validate each model made on the way down.
- There is *no fundamental difference* between the classic life cycle and the V-model; the V simply visualises where V&V is applied. Pros: tests planned early, clear stage-to-test link. Cons: as inflexible as waterfall; working software late.
- Sommerville's version (your slide 30) adds the **test plans** — Fig 2.10 in Section 2.4.3.

Pressman's incremental model (read once) applies linear sequences in a staggered way over calendar time; each delivers an operational increment, the first usually a *core product* (e.g. a word processor: basic editing first, then advanced editing, then spell-checking, then page layout). Because each increment is delivered to the customer, it matches Sommerville's **incremental delivery**.

Evolutionary model 1: the prototyping paradigm ★★★ MUST KNOW

Evolutionary models are iterative: each iteration produces a more complete version. Use **prototyping** when the customer has general objectives but no detailed requirements, or the developer is unsure of an algorithm's efficiency, an operating system's adaptability or the form of the user interaction. It can be a stand-alone model but is more often a *technique* used inside other models.

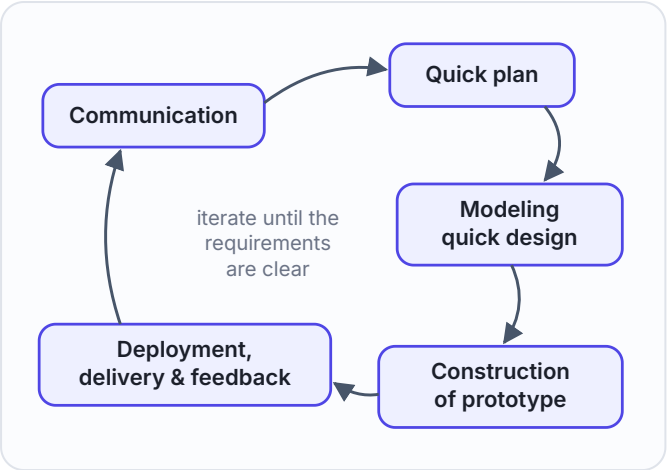


Fig 2.5 The prototyping paradigm. Pressman Fig 2.6 (From Textbook 2)

1. **Communication** — define overall objectives, known requirements and areas needing definition.
2. **Quick plan** — plan an iteration quickly.
3. **Modeling (quick design)** — of what users will see: screen layouts, output formats.
4. **Construction of prototype** — build quickly, reusing fragments and tools.
5. **Deployment, delivery & feedback** — stakeholders evaluate; feedback refines the requirements. Iterate.

A **throwaway** prototype is discarded after use (Brooks: "plan to throw one away"); an **evolutionary** prototype slowly grows into the real system. **Strengths:** clarifies fuzzy requirements; users get an early feel for the system; developers build something at once. **Problems:** (1) stakeholders see an apparently working version, unaware it is held together haphazardly, and demand "a few fixes" to ship it; (2) quick implementation compromises (wrong language, inefficient algorithm) become a permanent part of the system.

> ★ KEY POINT
>
> Define **the rules of the game at the beginning**: everyone agrees the prototype only defines requirements; it is then discarded (at least in part) and the real software is engineered for quality. Sommerville's prototyping (slides 37–41) is in Section 2.5.1.

Evolutionary model 2: the spiral model (Boehm) ★★★ MUST KNOW

> IN SIMPLE WORDS
>
> Like climbing a spiral staircase: on each round you pass the same points again (communicate, plan and analyse risks, design, build, deliver), but each round takes you higher — to a more complete system — and you check for risks every time.

Proposed by **Barry Boehm (1988)**: an evolutionary, **risk-driven** model that couples the *iterative nature of prototyping* with the *controlled, systematic aspects of the waterfall model*. Its two distinguishing features: (1) a **cyclic approach** that grows the system's definition and implementation while **decreasing its degree of risk**; (2) **anchor point milestones** that secure stakeholder commitment to feasible, mutually satisfactory solutions.

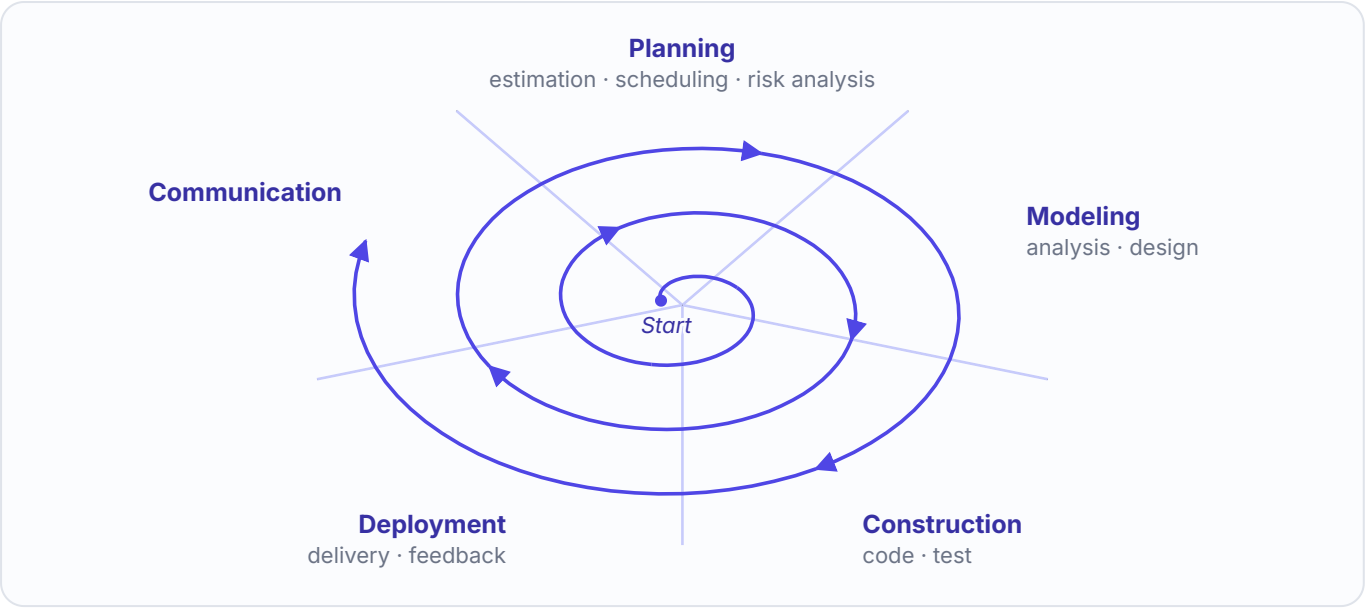


Fig 2.6 A typical spiral model. Start at the centre and move clockwise; each circuit yields a more complete version, and risk is analysed on every circuit. Pressman Fig 2.7 (From Textbook 2)

- The team moves **clockwise from the centre** through the framework activities; **risk is considered on each revolution** and anchor point milestones are noted on each pass.
- Early circuits produce a *specification* or *prototype*; later circuits produce progressively more complete versions. Each pass through planning adjusts cost, schedule and the number of iterations.
- Sommerville: **each loop represents a phase** (innermost — feasibility, next — requirements definition, next — system design, ...). It combines change avoidance with change tolerance and has **explicit risk management**.
- It can cover the whole life of the software (concept development → new product development → product enhancement) until the software is retired.

Also seen in exam answers: Boehm's original drawing (used in older Sommerville editions) splits each loop into four sectors — objective setting; risk assessment and reduction; development and validation; planning. Either drawing earns marks if labelled.

ADVANTAGES

- ✓ Realistic approach for large-scale systems
- ✓ Developer and customer understand and react to risks at each level
- ✓ Prototyping used as risk reduction at any stage
- ✓ Systematic stepwise approach inside an iterative framework

DISADVANTAGES

- ✗ Hard to convince customers (in contracts) it is controllable
- ✗ Demands considerable risk-assessment expertise
- ✗ If a major risk is missed, problems will occur
- ✗ Poor fit for fixed budgets: cost revised every circuit

Read once — the concurrent model (concurrent engineering): all activities exist at the same time, each in a *state* (inactive, under development, awaiting changes, under review, under revision, baselined, done); *events* trigger transitions, e.g. a requirements error found during design moves requirements analysis from "done" to "awaiting changes". It gives an accurate picture of a project's current state.

2.4 Process activities ★★★ MUST KNOW

Real software processes are **inter-leaved sequences of technical, collaborative and managerial activities** with the overall goal of specifying, designing, implementing and testing a software system (usually with tool support).

★ KEY POINT

The four basic activities — specification, development, validation and evolution — are **organised differently in different processes**: in the waterfall model they are **in sequence**, whereas in incremental development they are **interleaved**.

2.4.1 Software specification ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software specification — the process of establishing what services are required and the constraints on the system's operation and development. This is **requirements engineering**: the process of developing a software specification.

It is a **critical stage**: mistakes here cause later problems in design and implementation. A short, cheap **feasibility study** may come first (is there a need, and is it technically and financially realistic?). The goal is an agreed **requirements document** at two levels: *user requirements* for customers and end-users, and detailed *system requirements* for developers.

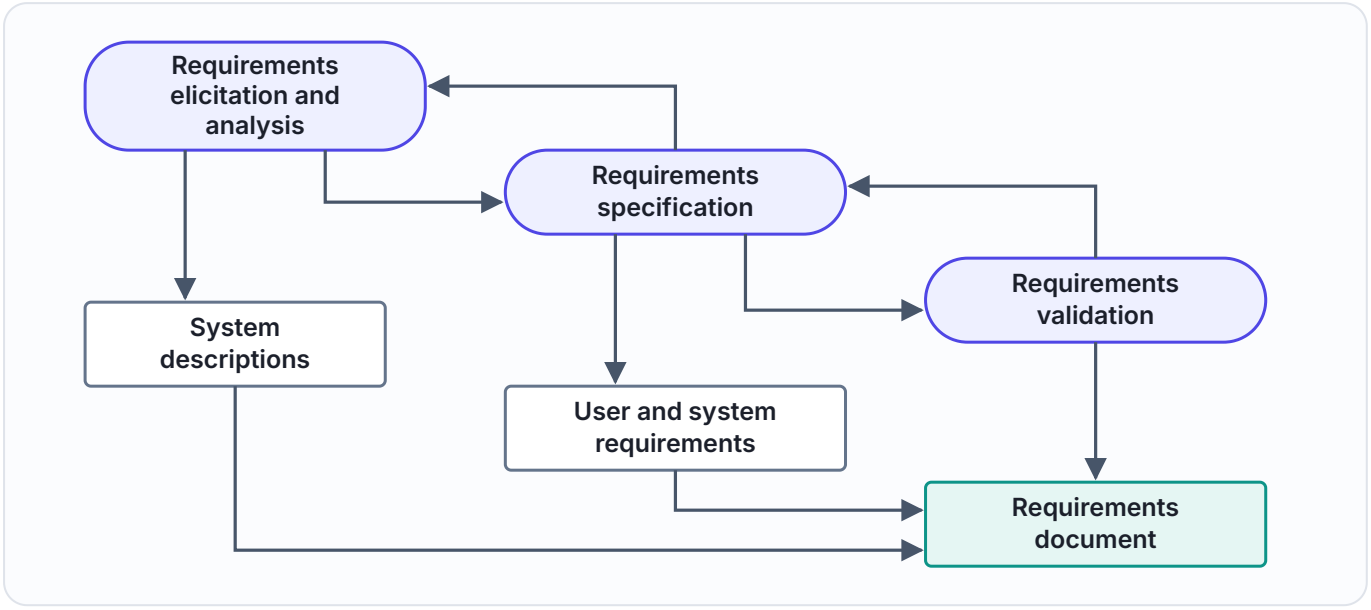


Fig 2.7 The requirements engineering process. Rounded boxes are activities; square boxes are products (documents).
Sommerville Fig 2.4 · Your slides, slide 21

Activity	Slide point	Textbook detail
Requirements elicitation and analysis	What do the system stakeholders require or expect from the system?	Observe existing systems, talk to users and procurers, task analysis; may use models and prototypes.
Requirements specification	Defining the requirements in detail.	Write the gathered information as user requirements and system requirements.

Activity	Slide point	Textbook detail
Requirements validation	Checking the validity of the requirements.	Check realism, consistency and completeness; correct the errors found.

The activities are **interleaved** (back-arrows in Fig 2.7). In **agile** methods specification is not separate: requirements are written informally for each increment just before it is built. *Example (insulin pump):* "The system shall be available to deliver insulin when required" is a high-level user requirement.

2.4.2 Software design and implementation ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Design and implementation** — the process of converting the system specification into an executable system.

Software design: design a software structure that realises the specification (structure, data, interfaces between components, sometimes algorithms). **Implementation**: translate this structure into an executable program. The two are **closely related and may be inter-leaved**; in agile development design is informal (whiteboards) with no formal design documents.

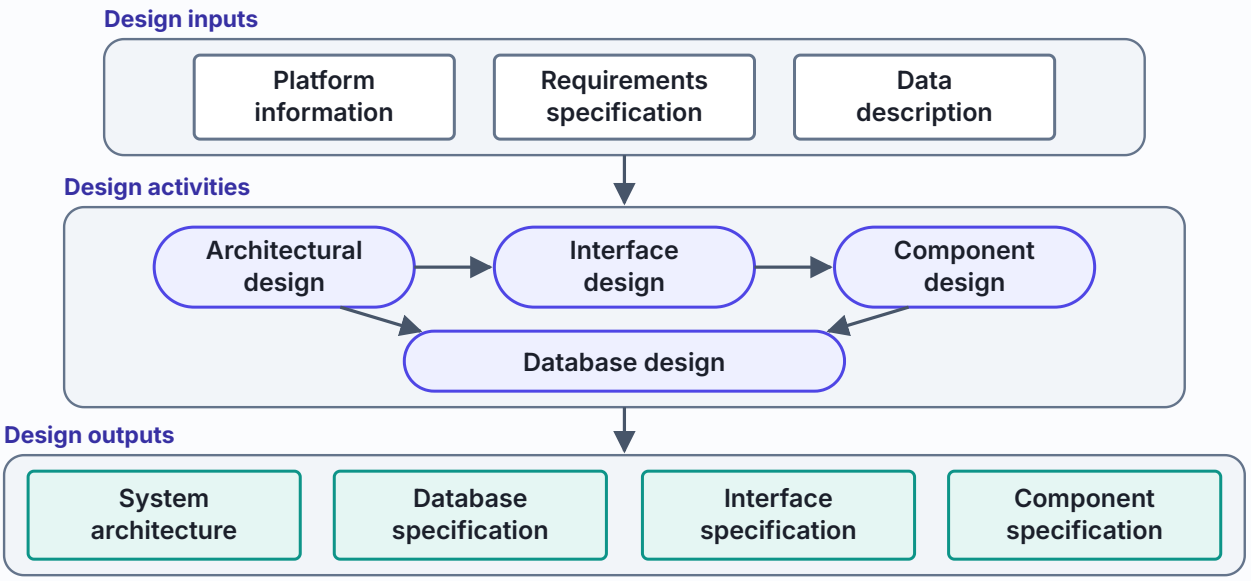


Fig 2.8 A general model of the design process. Sommerville Fig 2.5 · Your slides, slide 24

Inputs: platform information (the operating system, database, middleware and other systems the software runs with), the requirements specification, and a data description. **Outputs:** detailed documents for critical systems, design diagrams in model-driven development, or just the code in agile development.

Design activity	What you do
Architectural design	Identify the overall structure of the system, the principal components (sub-systems or modules), their relationships and how they are distributed.
Database design	Design the system data structures and how these are to be represented in a database.
Interface design	Define the interfaces between system components — unambiguously, so a component can be used without knowing its implementation.

Design activity	What you do
Component selection and design	Search for reusable components; if unavailable, design how the new component will operate.

The activities are interleaved and interdependent, so design rework is inevitable; they also vary (a real-time system adds *timing design* and may have no database).

System implementation ★★ IMPORTANT

- The software is implemented either by **developing a program or programs** or by **configuring an application system**. Design and implementation are interleaved for most systems.
- **Programming is an individual activity with no standard process** (some programmers start with familiar components, others leave them till last).
- **Debugging** is the activity of finding program faults and correcting these faults — by forming and testing hypotheses about the program's behaviour, often with interactive debuggers.

> ⚠ WATCH OUT
>
> **Testing ≠ debugging.** Testing establishes the *existence* of defects; debugging *locates and corrects* them.

2.4.3 Software validation ★★★ MUST KNOW

> DEFINITION — LEARN THIS LINE
>
> **Verification and validation (V & V)** — is intended to show that a system conforms to its specification and meets the requirements of the system customer.

In short, validation checks that the system conforms to its specification *and* meets the real needs of its users. V & V involves **checking and review processes** and **system testing**. System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system. **Testing is the most commonly used V & V activity.** Except for small programs, systems are tested in stages, not as one unit.

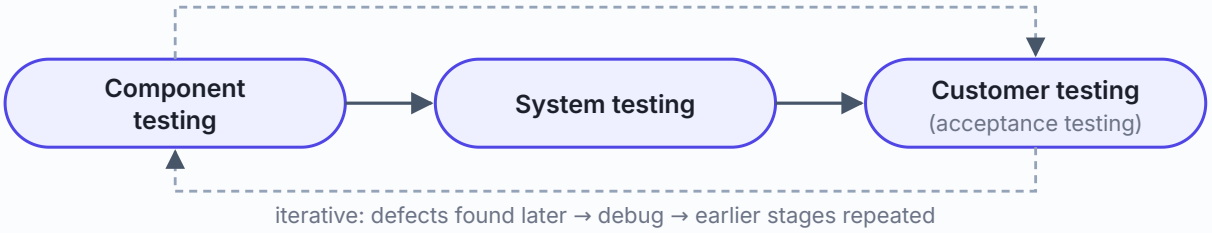


Fig 2.9 Stages of testing (slide 28 labels the last stage "Acceptance testing"). Sommerville Fig 2.6 · Your slides, slide 28

Stage	Slide points	Textbook detail
Component testing	Individual components are tested independently; components may be functions or objects or coherent groupings of these.	Done by the developers; tools such as JUnit rerun tests automatically.

Stage	Slide points	Textbook detail
System testing	Testing of the system as a whole. Testing of emergent properties is particularly important.	Finds interaction and interface errors; checks functional and non-functional requirements.
Customer testing	Testing with customer data to check that the system meets the customer's needs.	Last stage before use; may reveal requirements problems. For products: **beta testing** by selected users.

Emergent properties belong to the whole system, not to any one part (e.g. overall reliability, performance) — so they can only be tested after integration.

Testing phases in a plan-driven software process (V-model) ★★★ MUST KNOW

In a plan-driven process (e.g. for critical systems), an independent team of testers works from **test plans** developed from the system specification and design. The test plans are the **link between development and testing**; the V-model shows the validation activity that corresponds to each stage of the waterfall model.

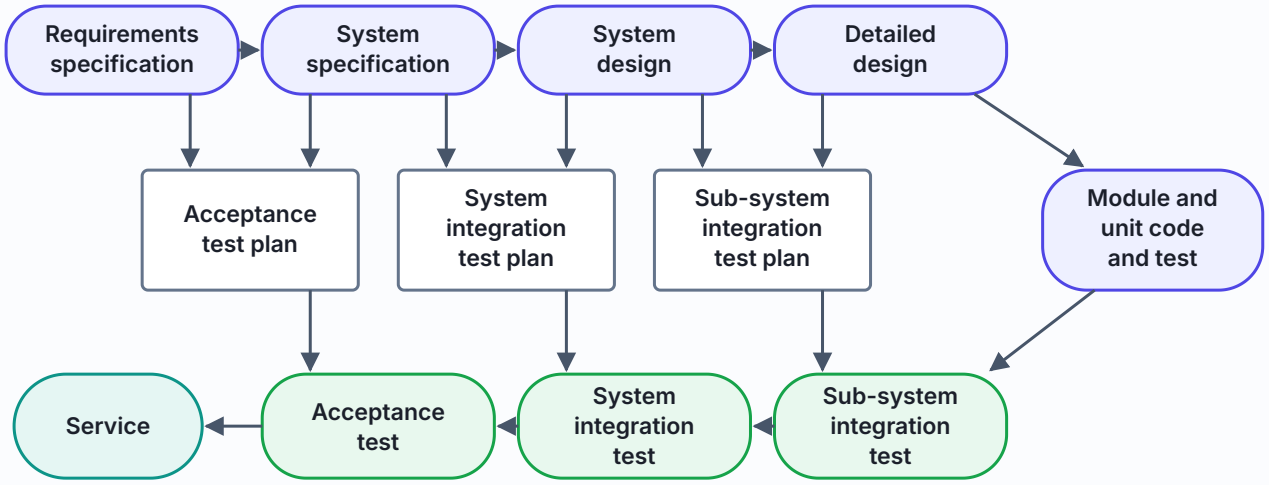


Fig 2.10 Testing phases in a plan-driven software process (V-model). Square boxes are test plans written during development and used later in testing. Your slides, slide 30 · Sommerville Fig 2.7

Read it as a V lying on its side: development runs along the top to *module and unit code and test*; testing returns along the bottom to *service*. Requirements + system specification → **acceptance test plan**; system specification + system design → **system integration test plan**; system design + detailed design → **sub-system integration test plan.** (The 10th-edition textbook says "customer test plan", "component design" and "component code and test" — either labelling is correct.)

2.4.4 Software evolution ★★★ MUST KNOW

- **Software is inherently flexible and can change** — far more cheaply than hardware, even after delivery.
- As requirements change through changing business circumstances, the software that supports the business must also evolve and change — **it must evolve to remain useful**.
- Although there has been a demarcation between development and evolution (maintenance), this is **increasingly irrelevant as fewer and fewer systems are completely new**; development and maintenance form one continuum.

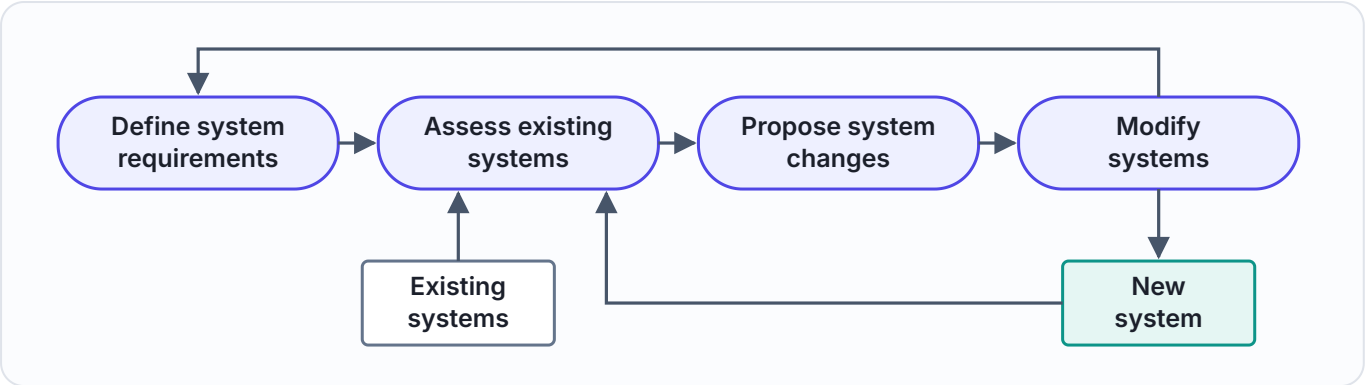


Fig 2.11 System evolution: the new system becomes an existing system that will itself be changed. Sommerville Fig 2.8 · Your slides, slide 32

2.5 Coping with change ★★★ MUST KNOW

Change is inevitable in all large software projects: business changes lead to new and changed system requirements; new technologies open up new possibilities for improving implementations; changing platforms require application changes. Change leads to **rework** (redoing completed work), so the costs of change include both rework (e.g. re-analysing requirements) and the costs of implementing new functionality.

	Change anticipation	Change tolerance
Meaning	The process includes activities that can anticipate possible changes **before significant rework is required**.	The process is designed so that changes can be accommodated at **relatively low cost**.
Example	A prototype system shows key features to customers, who refine their requirements before costly development.	Incremental development: changes go into increments not yet developed; if impossible, only a single increment (a small part of the system) is altered.
Technique	**System prototyping**	**Incremental delivery** (also gives change *avoidance*); refactoring

So there are two ways of coping with changing requirements: **system prototyping** — a version of the system or part of it is developed quickly to check the customer's requirements and the feasibility of design decisions (supports change anticipation); and **incremental delivery** — increments are delivered to the customer for comment and experimentation (supports both change avoidance and change tolerance). Processes structured for iterative development and delivery let changes be made without disrupting the system as a whole.

2.5.1 Software prototyping ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
>
> **Prototype** — an initial version of a system used to demonstrate concepts and try out design options.

A prototype can be used in

1. **Requirements engineering** — to help with requirements elicitation and validation.
2. **Design** — to explore options and develop a UI design (rapid prototyping with users is the only sensible way to develop user interfaces).
3. **Testing** — to run *back-to-back tests*: the same tests run on the prototype and the system, and the outputs are compared.

Benefits of prototyping

1. Improved system usability
2. A closer match to users' real needs
3. Improved design quality
4. Improved maintainability
5. Reduced development effort

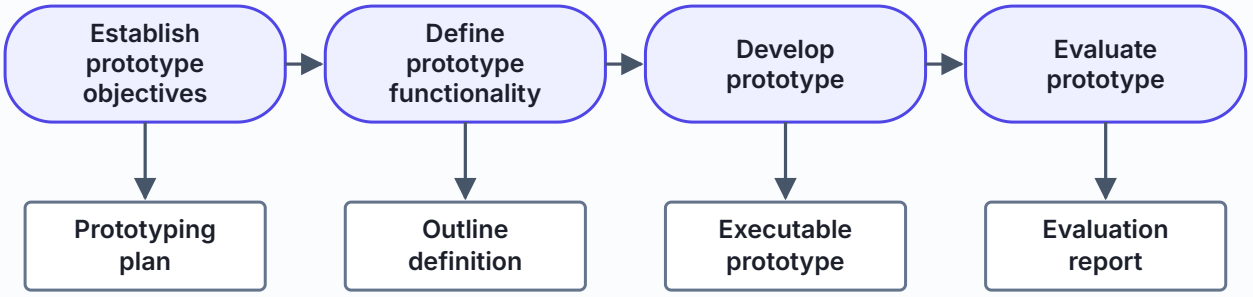


Fig 2.12 The process of prototype development: four activities, each with its output. Sommerville Fig 2.9 · Your slides, slide 39

Objectives must be explicit (e.g. develop the UI, validate functional requirements, or demo to managers) — one prototype rarely meets all of them. **Functionality**: decide what to leave out. **Evaluation**: train users and let them settle into normal use; note that users may not use a prototype the way they will use the final system. **Prototype development** (slide 40):

- May be based on rapid prototyping languages or tools.
- May involve leaving out functionality: focus on areas of the product that are **not well understood**; error checking and recovery may not be included; focus on **functional** rather than non-functional requirements such as reliability and security.

Throw-away prototypes

Prototypes should be **discarded after development**, as they are not a good basis for a production system:

1. It may be impossible to tune the system to meet non-functional requirements.
2. Prototypes are normally undocumented.
3. The prototype structure is usually degraded through rapid change.
4. The prototype probably will not meet normal organisational quality standards.

? QUICK CHECK

Your manager wants to ship your impressive prototype as the production system. What do you say?

Answer: Give the four throw-away reasons: it cannot be tuned for performance, security or reliability; it has no documentation; its structure is degraded, so it is hard to maintain; it fails quality standards. Use it as a specification and build the real system properly.

2.5.2 Incremental delivery ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Incremental delivery — rather than deliver the system as a single delivery, the development and delivery is broken down into increments, with each increment delivering part of the required functionality.

User requirements are **prioritised** and the highest priority requirements are included in early increments. Once the development of an increment is started, **its requirements are frozen**, though requirements for later increments can continue to evolve. Each finished increment is installed in the customer's working environment and integrated with earlier ones.

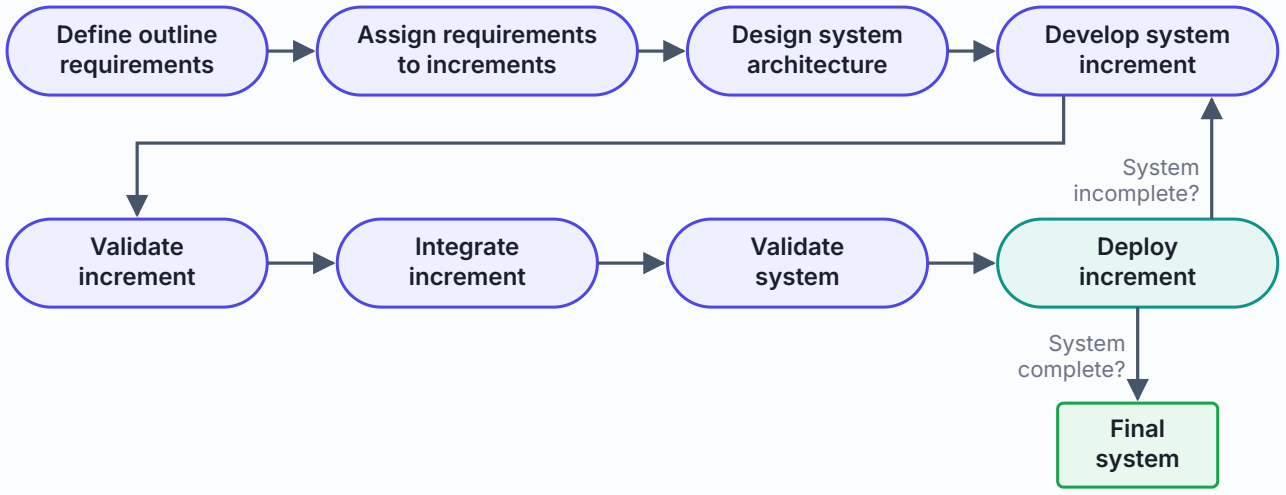


Fig 2.13 Incremental delivery. The "System incomplete?" loop repeats once per increment. Sommerville Fig 2.10 · Your slides, slide 44

ADVANTAGES

- ✓ **Customer value can be delivered with each increment**, so system functionality is available earlier.
- ✓ **Early increments act as a prototype** to help elicit requirements for later increments (and they are part of the real system — no relearning).
- ✓ **Lower risk of overall project failure.**
- ✓ **The highest priority system services tend to receive the most testing.**

PROBLEMS

- ✗ Most systems need **basic facilities used by different parts**; as requirements are not detailed until an increment is implemented, these common facilities are hard to identify.
- ✗ In iterative processes the specification is developed with the software; this **conflicts with the procurement model** of many organisations, where the complete specification is part of the contract.
- ✗ Hard for **replacement systems** — users won't accept less than the old system.

Incremental development vs incremental delivery ★★★ MUST KNOW

Point	Incremental development	Incremental delivery
Each increment	Develop the system in increments and **evaluate each increment** before proceeding to the next	**Deploy** each increment for use by end-users

Point	Incremental development	Incremental delivery
Evaluated by	User or **customer proxy** (someone representing the customer), via demos	Real end-users in their working environment — a **more realistic evaluation** of practical use
Typical use	The **normal approach used in agile methods**	New systems whose users have time to experiment
Main difficulty	Process not visible; structure degrades	**Difficult for replacement systems**, as increments have less functionality than the system being replaced

★ KEY POINT

Incremental **development** is about **how you build** (in pieces, with feedback). Incremental **delivery** is about **how you release** (every piece goes into real use). Delivery implies incremental development, but not the other way round.

2.6 Process improvement ★★★ MUST KNOW

Many software companies have turned to **software process improvement** as a way of enhancing the quality of their software, reducing costs or accelerating their development processes.

DEFINITION — LEARN THIS LINE

Process improvement — understanding existing processes and changing these processes to increase product quality and/or reduce costs and development time.

	Process maturity approach	Agile approach
Focus	Improving process and project management and introducing good software engineering practice	Iterative development and the reduction of overheads in the software process
Key idea	The **level of process maturity** reflects the extent to which good technical and management practice has been adopted in the organisation's processes	The primary characteristics of agile methods are **rapid delivery of functionality** and **responsiveness to changing customer requirements**
Goal / style	Product quality and process predictability; plan-driven, adds "overhead" activities	Best processes have the lowest overheads; minimal formality and documentation

The process improvement cycle ★★★ MUST KNOW

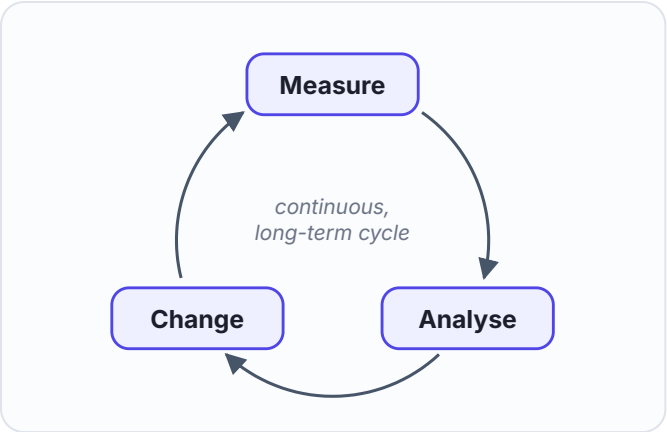


Fig 2.14 The process improvement cycle. Sommerville Fig 2.11 · Your slides, slide 50

1. **Process measurement** — measure one or more attributes of the process or product. These measurements form a **baseline** to decide if improvements have been effective.
2. **Process analysis** — assess the current process; identify **weaknesses and bottlenecks**. Process models (**process maps**) may be developed.
3. **Process change** — propose and introduce changes for the weaknesses; the cycle resumes to collect data on their effectiveness.

Process measurement: wherever possible, **quantitative process data should be collected** — but where organisations do not have clearly defined process standards this is very difficult, as you don't know what to measure; a process may have to be defined first. Measurements should be used to **assess** improvements, but should not **drive** them: the improvement driver should be the organisational objectives.

Process metric	Example
Time taken for process activities to be completed	Calendar time or effort to complete an activity or process
Resources required for processes or activities	Total effort in person-days
Number of occurrences of a particular event	Number of defects discovered

Capability maturity levels — the SEI capability maturity model ★★★ MUST KNOW

In the late 1980s the **Software Engineering Institute (SEI)** proposed a model of process capability maturity (Humphrey, 1988) so that the U.S. Department of Defense could assess defence contractors and limit contracts to mature ones. Its five maturity levels essentially correspond to the use of good software engineering practice. The model later evolved into CMMI (slide 54 names).

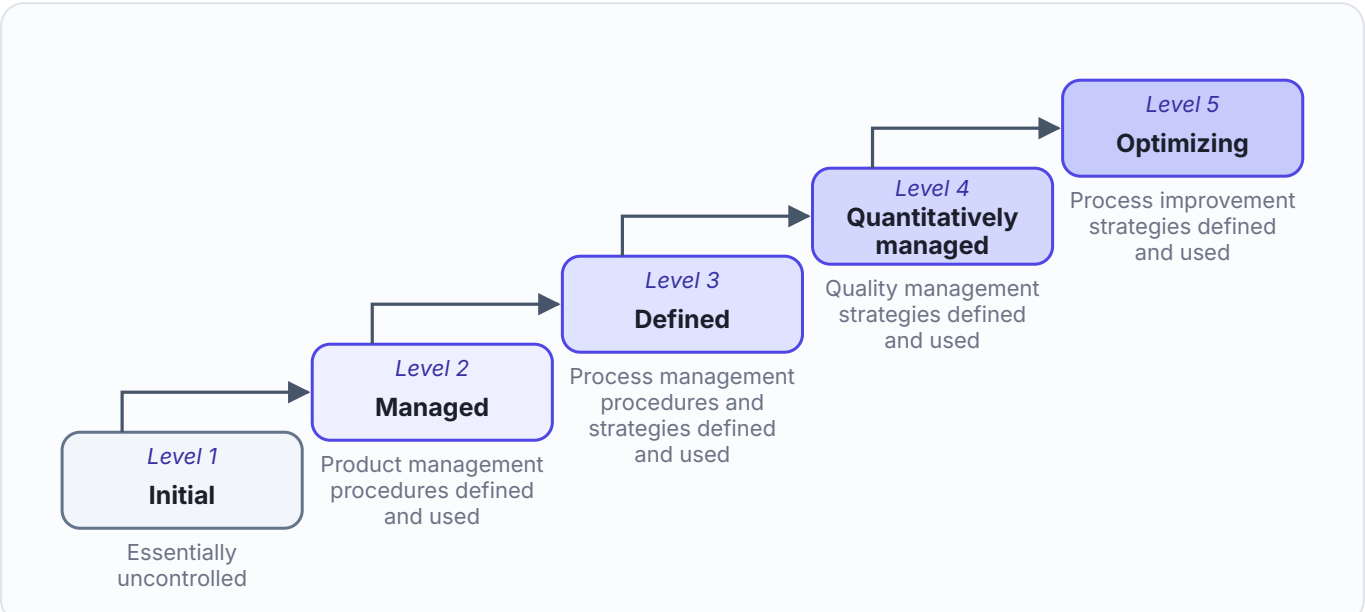


Fig 2.15 Capability maturity levels as a staircase (box names from slide 54; grey meanings from slide 55). Sommerville Fig 2.12 · Your slides, slides 54–55

No.	SEI CMM (slide 55)	CMMI (slide 54)	Meaning (slide 55)	Textbook detail
1	**Initial**	Initial	Essentially uncontrolled	Process goals satisfied; scope of work set out and communicated to the team.
2	**Repeatable**	Managed	Product management procedures defined and used	Policies say when each process is used; documented project plans; resource management and process monitoring.
3	**Defined**	Defined	Process management procedures and strategies defined and used	Standard organisational processes adapted per project; process measurements kept for improvement.
4	**Managed**	Quantitatively managed	Quality management strategies defined and used	Statistical/quantitative methods control sub-processes using process and product measurements.
5	**Optimising**	Optimizing	Process improvement strategies defined and used	Measurements drive improvement; trends analysed; processes adapted to business needs.

MEMORY TRICK

SEI levels bottom to top: **I**nitial, **R**epeatable, **D**efined, **M**anaged, **O**ptimising — "**I R**eally **D**o **M**ean **O**rder". CMMI renames only levels 2 and 4.

Impact and limits: the maturity work focused attention on processes and led to significant improvements in capability, but it has **too much overhead for small companies** and **maturity estimation with agile processes is difficult**, so mainly large companies use it. (Pressman also names CBA IPI, SPICE (ISO/IEC 15504) and ISO 9001:2000 for Software as assessment standards — read once.)

2.7 Comparing all the process models ★★★ MUST KNOW

Model	Core idea	Best when	Strengths	Weaknesses	Example system
Waterfall	Separate, sequential, signed-off phases; plan-driven	Stable, well-understood requirements; embedded, critical, multi-site projects	Easy to manage; documents at every phase; supports safety analysis	Inflexible to change; working software late; requirements frozen early	Insulin pump; antilock braking system
V-model	Waterfall bent into a V; each design stage has a test plan and a test level	Critical, plan-driven systems with independent testers	Tests planned early; clear stage-to-test traceability	As inflexible as waterfall; little early working software	Insulin pump; avionics control software
Incremental development	Spec, development and validation interleaved; series of versions	Changing requirements; business systems and products	Cheaper change; easy feedback; early useful software	Process not visible; structure degrades	iLearn; Mentcare; a mobile app

Model	Core idea	Best when	Strengths	Weaknesses	Example system
Incremental delivery	Prioritised increments deployed to users one by one	New system; early value needed; users can experiment	Early value; increments act as prototypes; lower risk; key services tested most	Common facilities hard to find; clashes with fixed contracts; bad for replacements	Mentcare clinics; Pressman's word processor
Integration & configuration	Configure and integrate COTS systems, components, web services	Suitable reusable software exists; business systems	Lower cost and risk; faster delivery	Requirements compromises; no control over evolution of reused parts	iLearn services; an ERP or accounting system
Prototyping	Quick version to learn requirements or try designs, then thrown away	Fuzzy requirements; critical UI; technical doubt	Usability; closer match to real needs; fewer late changes	Pressure to ship the prototype; shortcuts stay	Mentcare user interface; a VR system
Spiral (Boehm)	Risk-driven loops; each circuit a more complete version	Large, complex, high-risk, long-lived systems	Explicit risk management; realistic for large systems	Needs risk expertise; hard to fix by contract; heavy for small jobs	A national weather information system

⚠ **WATCH OUT**

Never write "waterfall is bad, agile is good". Every model suits some systems. Marks come from linking the choice to the **system's characteristics**: requirements stability, criticality, hardware dependence, size and spread of teams, available reusable software, and risk.

2.8 Comparative analysis on the case studies (lab) ★★★ MUST KNOW

For the lab (and the "suggest a model" exam question) use five steps: **(1)** characterise the system (type, criticality, hardware, users, size, replacing an old system?); **(2)** list deciding criteria; **(3)** rate each model; **(4)** recommend one model with reasons tied to the characteristics; **(5)** say what you borrow from other models — real processes are hybrids.

Case study	Waterfall	V-model	Incr. dev.	Incr. delivery	Reuse	Prototyping	Spiral
Insulin pump	✓✓	✓✓	✗	✗	✗	✓ (requirements)	✓
Mentcare	✗	✗	✓✓	✓	✓	✓✓ (UI)	✓
Weather station	✓✓	✓	✗	✗	✓ (data store)	✓	✓✓
iLearn	✗	✗	✓✓	✓✓	✓✓	✓ (UI)	✗

✓✓ strong fit · ✓ useful inside a hybrid · ✗ poor fit. These are reasoned suggestions; another choice also scores if justified from the system's characteristics.

System and characteristics	Recommended	Why (tie each reason to a characteristic)	Not suitable / watch out
Insulin pump — embedded medical device; sensor + pump hardware; safety-critical	Waterfall with V-model test plans (formal methods for the most critical parts)	Embedded: hardware is fixed early, so software decisions can't wait. Critical: a complete specification and design are needed for safety analysis; regulators need signed-off documents; independent testing against test plans.	Incremental delivery (no "half a pump"); agile's light documentation. Use a prototype only to explore requirements.
Mentcare — patient information system for mental-health clinics; many user types; privacy, security, legal duties	Incremental development + UI prototyping	Many users' needs are hard to fix up front and will change; clinicians give feedback per increment (care management first, then patient monitoring, then reports); prototyping gives usability and a closer match to real needs.	Specify privacy and legal requirements carefully up front. If it replaces an old system, incremental delivery is hard.
Wilderness weather station — unattended, battery-powered, satellite link, hostile environment; part of a larger weather information system	Plan-driven (waterfall/V) for station software, inside a spiral for the whole system	Software depends on instrument, power and comms hardware fixed early; several sub-systems and partners to coordinate; high risks (power, communication, damage) suit the spiral's explicit risk analysis.	Pure agile/incremental delivery. Reuse existing database software for data archiving.
iLearn — service-based learning environment for ages 3–18; bought or free application services; replaceable services	Integration and configuration + incremental (agile) delivery	Most functionality exists as services, so reuse cuts cost and time; configuration services adapt each school's version; needs vary and evolve; teachers give quick feedback; not safety-critical.	Requirements compromises and loss of control over external services' evolution.

? QUICK CHECK

Practice (Sommerville Exercise 2.1): choose a model for (a) an antilock braking system, (b) a virtual-reality system to support software maintenance, (c) a university accounting system replacing an existing one, (d) an interactive travel planning system.

Answer: (a) Waterfall/V-model — embedded and safety-critical. (b) Incremental development with prototyping — uncertain requirements and interface. (c) Plan-driven, or configure an accounting package — requirements known from the old system; incremental delivery is hard for replacements. (d) Incremental development — user-facing, requirements change with feedback.

Chapter 2 in one page

Definitions

- **Software process**: structured set of activities required to develop a software system.
- **Process model**: abstract representation of a process from a particular perspective.
- **Plan-driven**: all activities planned in advance; progress measured against the plan. **Agile**: incremental planning; easy to change.
- **Prototype**: initial version used to demonstrate concepts and try out design options.
- **Process improvement**: understand and change processes to raise quality and/or reduce cost and time.

Lists to write in full

- **Activities**: specification, design & implementation, validation, evolution.
- **Process description**: products, roles, pre-/post-conditions.
- **Waterfall**: requirements; system & software design; implementation & unit test; integration & system test; operation & maintenance.
- **Reusable software**: COTS apps; object collections (.NET/J2EE); web services.
- **Reuse stages**: spec; discovery & evaluation; refinement; configuration; adaptation & integration.
- **RE**: elicitation & analysis; specification; validation.
- **Design**: architectural, database, interface, component selection & design.
- **Testing**: component, system, customer (acceptance).
- **Prototyping** uses: RE, design, back-to-back testing. Benefits: usability, real needs, design quality, maintainability, less effort.

Comparisons

- **Waterfall vs incremental**: sequential vs interleaved; change cost; feedback; visibility.
- **Incr. development vs delivery**: evaluate with proxy vs deploy to real users.
- **Anticipation vs tolerance**: prototyping vs incremental.
- **Maturity vs agile improvement**: good practice vs low overhead.
- **Testing vs debugging**: find defects vs locate and fix them.

Numbers and names

- Incremental development: **3 benefits, 2 problems**. Incremental delivery: **4 advantages, 2 problems**.
- Throw-away reasons: NFRs untunable; undocumented; degraded structure; below standards.
- Improvement cycle: **measure → analyse → change**. Metrics: **time, resources, events**.
- SEI CMM: **Initial, Repeatable, Defined, Managed, Optimising**.
- Spiral: Boehm 1988, risk-driven. Waterfall: Royce 1970.

Diagrams to practise

- Waterfall · incremental · reuse · both V-models · prototyping paradigm · spiral
- RE process · design process · testing stages · evolution
- Prototype development · incremental delivery · improvement cycle · CMM staircase

Case studies

- Pump → waterfall + V · Mentcare → incremental + prototyping · Weather station → plan-driven in a spiral · iLearn → reuse + incremental.

Likely exam questions

4M **Define software process and software process model. List the four fundamental activities and what process descriptions include.**

How to answer → Slide definitions word for word; four activities; products, roles, pre-/post-conditions (2.1).

5M **Differentiate between plan-driven and agile processes.**

How to answer → Two definitions → 5-row table → "most processes combine both; no right or wrong process".

10M **Explain the waterfall model with a neat diagram. Discuss its problems and when it is appropriate.**

How to answer → Fig 2.1 → 5 phases → main drawback → problems → embedded, critical, multi-site systems.

8M **Explain incremental development with a diagram, with its benefits and problems.**

How to answer → Fig 2.2 → interleaving, versions → 3 benefits, 2 problems with sub-points.

8M **Explain reuse-oriented software engineering (integration and configuration).**

How to answer → COTS, configuration → 3 types of reusable software → Fig 2.3 → 5 stages → pros and cons.

10M **Explain Boehm's spiral model with a neat diagram. List its advantages and disadvantages.**

How to answer → Fig 2.6 → risk-driven circuits, anchor point milestones, each loop a phase → pros, cons.

6M **Explain the V-model. How are test plans linked to development in a plan-driven process?**

How to answer → Fig 2.10 (slide 30) and the three test plans; mention Pressman's V (Fig 2.4) pairs.

8M **With neat diagrams, explain the requirements engineering process and the general model of design.**

How to answer → Fig 2.7 + three RE activities; Fig 2.8 + inputs, four design activities, outputs.

6M **Explain the stages of testing. Distinguish testing from debugging.**

How to answer → V & V definition → Fig 2.9 → three stages → testing vs debugging.

6M **Why is change inevitable? Explain change anticipation and change tolerance.**

How to answer → Three causes → rework → anticipation vs tolerance table → the two ways of coping.

10M **What is a prototype? Explain prototype development with a diagram, and why throw-away prototypes are discarded.**

How to answer → 3 uses, 5 benefits → Fig 2.12 → slide 40 points → 4 reasons (Pressman's Fig 2.5 is a bonus).

8M **Explain incremental delivery with a diagram. How does it differ from incremental development?**

How to answer → Priorities, freezing → Fig 2.13 → 4 advantages, problems → comparison table.

10M **Explain the process improvement cycle and the SEI capability maturity levels.**

How to answer → Two approaches → Fig 2.14 + three activities → Fig 2.15 → five levels with meanings.

10M **Suggest and justify process models for the insulin pump, Mentcare, the weather station and iLearn.**

How to answer → For each: characteristics → model → reasons → what to borrow (2.8).

UNIT 1 · CHAPTER 3

Agile Software Development

Agile methods · Extreme Programming & agile techniques · Agile project management (Scrum) · Scaling agile

STUDY TIME
About 3 hours

SOURCES
Your slides (Ch 3) + Sommerville Ch 3

EXAM WEIGHT
Very high: manifesto, principles, XP practices and the Scrum sprint cycle are standard 5–10 mark questions

AFTER THIS CHAPTER YOU CAN

- Explain why rapid (agile) development is needed
- Differentiate plan-driven and agile development with a diagram
- Quote the Agile Manifesto and explain the five agile principles
- Draw the XP release cycle and list all ten XP practices
- Explain user stories, refactoring, test-first development and pair programming
- Define every Scrum term and explain the Scrum sprint cycle
- State the benefits of Scrum and the needs of distributed Scrum
- Discuss when agile or plan-driven development is the better choice

3.1 Rapid and agile development ★★★ MUST KNOW

Rapid software development ★★★ MUST KNOW

- **Rapid development and delivery** is now often the most important requirement for software systems. Businesses may even accept lower quality or fewer features to get essential software deployed quickly.
- Businesses operate in a **fast-changing environment**, so it is practically **impossible to produce a set of stable software requirements**. Customers cannot predict how a system will affect their work or which operations should be automated; the real requirements often become clear only after users have tried the system.
- Software therefore has to **evolve quickly** to reflect changing business needs.
- **Plan-driven development** (specify everything, then design, build and test) is essential for some types of system, e.g. safety-critical control systems, but it **does not meet these business needs**: each change forces rework, so software arrives long after it was specified.
- **Agile development methods** (e.g. Extreme Programming, Scrum, DSDM) emerged in the **late 1990s**. Their aim was to radically reduce the delivery time for working software systems.

IN SIMPLE WORDS

Ordering a tailor-made suit. **Plan-driven**: you give every measurement on day one and collect the suit months later; if your size or taste changed, alterations are expensive. **Agile**: the tailor gives you a fitting every two weeks and adjusts as you go.

What is agile development? ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Agile methods — incremental development methods that focus on rapid software development, frequent releases of the software, reducing process overheads by minimizing documentation, and producing high-quality code. Customer representatives are involved directly in development.

An **increment** is a small, working piece of new functionality added in each version; a **stakeholder** is anyone with an interest in the system (users, customers, managers, regulators). Agile development is characterized by:

1. **Interleaved specification, design and implementation.** There is no detailed specification; design documentation is minimal or generated automatically.
2. **A series of versions (increments)**, with stakeholders involved in specifying and evaluating each version.
3. **Frequent delivery** of new versions for evaluation, typically every 2–3 weeks.
4. **Extensive tool support**, e.g. automated testing tools, configuration management and integration tools.
5. **Minimal documentation, focus on working code**; informal communication instead of formal documents.

Plan-driven vs agile development ★★★ MUST KNOW

Plan-driven Based around **separate development stages**, with the outputs of each stage **planned in advance**. Not necessarily waterfall: plan-driven *incremental* development is possible. **Iteration occurs within activities.**

Agile Specification, design, implementation and testing are **interleaved**, and the outputs are **decided through negotiation** during development. **Iteration occurs across activities.**

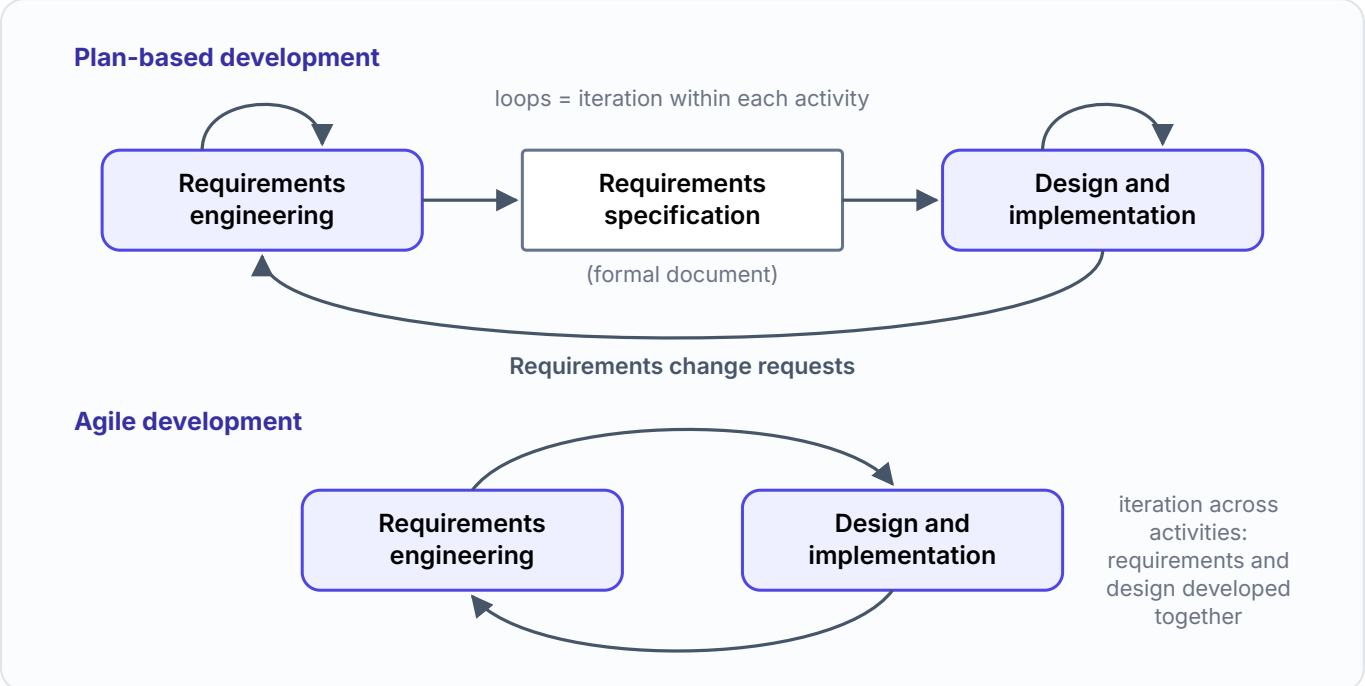


Fig 3.1 Plan-driven and agile development. Plan-based: a formal requirements specification sits between the stages, and a change request must go all the way back. Agile: requirements and design feed each other continuously. Sommerville Fig 3.1 · Your slides, slide 5

Aspect	Plan-driven development	Agile development
Basic idea	Separate stages; outputs planned in advance	Activities interleaved; outputs negotiated during development
Iteration	Within activities	Across activities
Communication	Formal documents between stages	Informal; minimal documentation, working code
Delivery	Often long after the software was specified	New versions every 2–3 weeks
Change	Causes rework and retesting of design and code	Expected; handled in the next increment
Best for	Safety-critical, large, long-life or regulated systems	Small/medium products and business systems with changing requirements

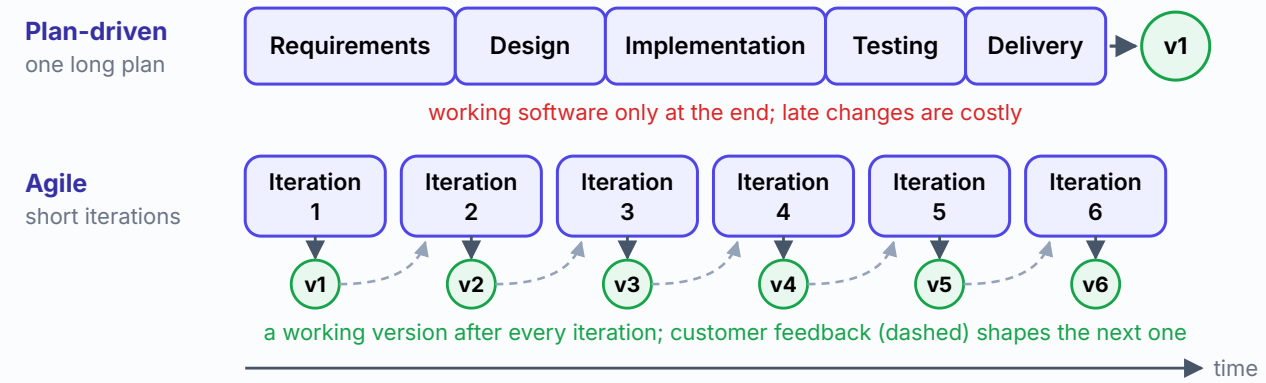


Fig 3.2 One big plan vs many short iterations. Drawn for this guide

⚠ **WATCH OUT**

Plan-driven is not the same as waterfall: plan-driven increments are possible if planned in advance. And agile is not "code only": an iteration may produce models and documentation. Real projects usually mix both (3.9).

✎ **EXAM TIP**

"Differentiate plan-driven and agile": draw Fig 3.1 + 5–6 table rows; say **"iteration within activities" vs "iteration across activities"**.

3.2 Agile methods ★★★ MUST KNOW

The problem agile methods solved ★★ IMPORTANT

In the 1980s and early 1990s, better software was thought to come from **careful project planning, formalized quality assurance, tool-supported analysis and design methods, and rigorous controlled processes**. This suited **large, long-lived systems** (aerospace, government) built by many teams. Applied to **small and medium business systems**, the overhead **dominated**: more time went on *how* to develop than on programming and testing, and every requirements change meant reworking the specification and design.

Dissatisfaction with these overheads led to agile methods, which:

- **focus on the code** rather than the design;
- are based on an **iterative approach** to software development;
- are intended to **deliver working software quickly** and **evolve it quickly** to meet changing requirements.

> ★ KEY POINT
> **Aim of agile methods:** to reduce overheads in the software process (e.g. by limiting documentation) and to be able to **respond quickly to changing requirements without excessive rework**.

The Agile Manifesto ★★★ MUST KNOW

Issued by the leading developers of agile methods (agilemanifesto.org). Learn it word for word:

> *"We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:*
>
> - **Individuals and interactions** over processes and tools
> - **Working software** over comprehensive documentation
> - **Customer collaboration** over contract negotiation
> - **Responding to change** over following a plan
>
> *That is, while there is value in the items on the right, we value the items on the left more."*

In simple words: people talking to each other matter more than fixed procedures or tools; progress is measured by software that runs, not by documents; work with the customer rather than argue over a contract; adapt when requirements change instead of clinging to an old plan.

> ⚠ WATCH OUT
> The manifesto does **not** call processes, documentation, contracts or plans useless; it values the left-hand items *more*. Writing "agile means no documentation" loses marks.

The principles of agile methods ★★★ MUST KNOW

All agile methods share these principles, based on the manifesto (Sommerville Fig 3.2):

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is to provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments, with the customer specifying the requirements to be included in each increment.
People, not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change, and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

MEMORY TRICK

"Can I Pass Every Module?" gives **C**ustomer involvement, **I**ncremental delivery, **P**eople not process, **E**mbrace change, **M**aintain simplicity.

Agile method applicability ★★ IMPORTANT

1. **Product development**: a software company developing a **small or medium-sized product for sale**. Virtually all software products and apps are now developed using an agile approach.
2. **Custom system development within an organization**, where there is a **clear commitment from the customer** to be involved in development and **few external rules and regulations** affect the software.

They fit because the customer can talk to the team continuously, the software is stand-alone, and a small **co-located** team can communicate informally. A safety-critical, regulated system such as the textbook's **insulin pump** needs plan-driven analysis instead.

3.3 Agile techniques: Extreme Programming ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Extreme Programming (XP) — a very influential agile method, developed in the late 1990s, that introduced a range of agile techniques. It takes an **"extreme" approach to iterative development**: new versions may be built several times per day, increments are delivered to customers every 2 weeks, and all tests must be run for every build; the build is only accepted if the tests run successfully.

The XP release cycle ★★★ MUST KNOW

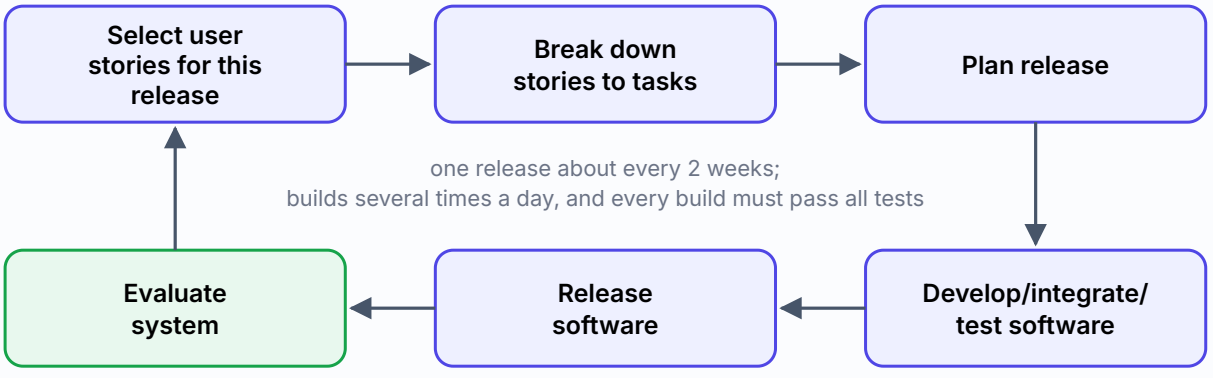


Fig 3.3 The XP release cycle: the process that produces one increment of the system. Sommerville Fig 3.3 · Your slides, slide 14

In words: (1) the customer **selects the user stories** for this release by priority and time available; (2) developers **break the stories into tasks**; (3) tasks are estimated and the **release is planned**; (4) pairs **develop, integrate and test** (tests first, all tests must pass); (5) the **software is released** to the customer; (6) the customer **evaluates the system**, and new or changed stories start the next cycle.

Extreme programming practices ★★★ MUST KNOW

All ten (Sommerville Fig 3.4). A **unit test** checks one small piece of code; **integration** means adding finished code into the whole system.

Principle or practice	Description
1. Incremental planning	Requirements are recorded on **story cards**, and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development "tasks" (see Figs 3.4 and 3.5).
2. Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
3. Simple design	Enough design is carried out to meet the current requirements and no more.
4. Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
5. Refactoring	All developers are expected to refactor the code continuously as soon as potential code improvements are found. This keeps the code simple and maintainable.
6. Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
7. Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
8. Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
9. Sustainable pace	Large amounts of overtime are not considered acceptable, as the net effect is often to reduce code quality and medium-term productivity.
10. On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an XP process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

MEMORY TRICK

Five groups. **Plan**: incremental planning, small releases, on-site customer. **Design**: simple design, refactoring. **Code**: pair programming, collective ownership, continuous integration. **Test**: test-first. **People**: sustainable pace. (3 + 2 + 3 + 1 + 1 = 10.)

XP and agile principles ★★ IMPORTANT

Agile principle	How XP supports it
Incremental development	**Small, frequent system releases**; simple customer stories decide what goes into each increment.
Customer involvement	**Full-time customer engagement** with the team; the customer also defines acceptance tests.
People, not process	**Pair programming, collective ownership** and a process that **avoids long working hours**.
Embrace change	**Regular system releases**, plus test-first development, refactoring and continuous integration.
Maintain simplicity	**Constant refactoring** of code and simple designs that do not anticipate future changes.

Influential XP practices ★★ IMPORTANT

XP has a **technical focus** and is **not easy to integrate with management practice** in most organizations. So although agile development uses XP practices, **the method as originally defined is not widely used**; companies pick the practices they want, often alongside Scrum (3.8). The **key practices** are: **user stories for specification, refactoring, test-first development and pair programming** (sections 3.4–3.7).

3.4 User stories for requirements ★★★ MUST KNOW

Requirements always change, so agile methods have no separate requirements phase; they **combine requirements elicitation** (finding out what users need) **with development**.

> **DEFINITION — LEARN THIS LINE**
>
> **User story** — a scenario of use that might be experienced by a system user, written briefly on a **story card** by the customer and the team. The team aims to implement it in a future release.

- In XP, a **customer or user is part of the XP team** and is responsible for making decisions on requirements.
- User requirements are expressed as **user stories or scenarios**.
- These are written on **cards**, and the development team breaks them down into **implementation tasks**. These tasks are the basis of **schedule and cost estimates**.
- The **customer chooses the stories** for the next release based on their priorities and the schedule estimates (about two weeks of work). Unimplemented stories may change or be discarded as requirements change.

Example: the "prescribing medication" story ★★★ MUST KNOW

From **Mentcare**, the textbook's patient information system for mental health care. A **formulary** is the approved list of drugs and their recommended doses. (The textbook tells the story about Kate, a doctor; the slides use "you".)

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either 'current medication', 'new medication' or 'formulary'.

If you select 'current medication', you will be asked to check the dose. If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose 'new medication', the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose 'formulary', you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click 'OK' or 'Change'. If you click 'OK', your prescription will be recorded on the audit database. If you click 'Change', you reenter the 'Prescribing medication' process.

Fig 3.4 A "prescribing medication" story card. Sommerville Fig 3.5 · Your slides, slide 20

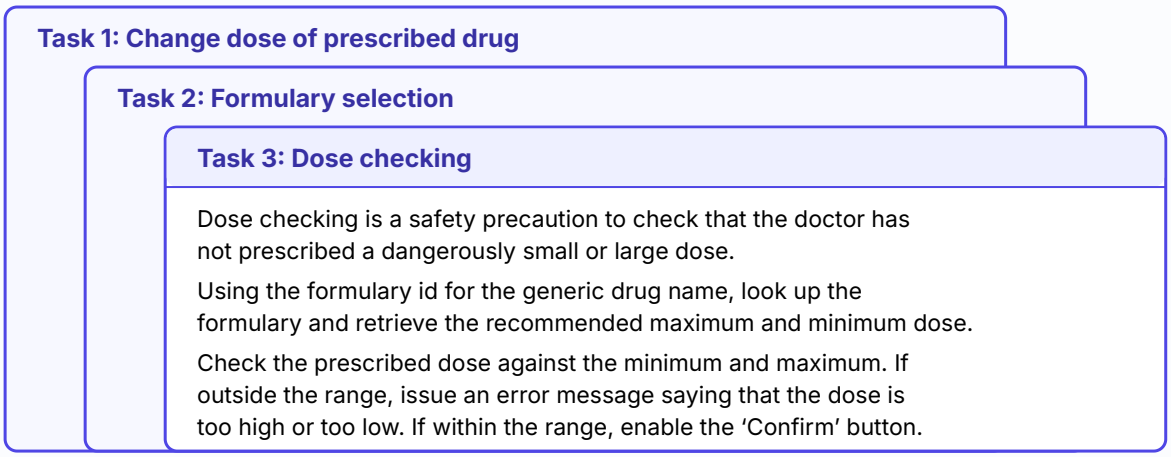


Fig 3.5 Examples of task cards for prescribing medication (Task 3 shown in full). Sommerville Fig 3.6 · Your slides, slide 21

STRENGTHS

- ✓ People relate to stories much more easily than to a requirements document or use cases.
- ✓ They get users involved in suggesting requirements.
- ✓ Tasks give a direct basis for estimates and planning.

PROBLEM: COMPLETENESS

- ✗ Hard to judge if enough stories cover all essential requirements.
- ✗ Hard to judge if one story gives a true picture of an activity.
- ✗ Experienced users leave things out when describing their work.

3.5 Refactoring ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Refactoring — constant code improvement: the programming team look for possible improvements to the software and implement them immediately, even where there is no immediate need for them. In plain words, tidying up code so it is clearer and better structured, without adding features.

	Conventional wisdom: design for change	XP: refactoring
Idea	Spend time and effort anticipating changes, as this reduces costs later in the life cycle	Not worthwhile: **changes cannot be reliably anticipated** (expected ones often never come)
Practice	Add generality to the design up front	**Constant code improvement**, so changes are easy when they have to be implemented

Why refactor? ★★★ MUST KNOW

In incremental development, **local changes degrade the software structure** (workarounds, duplicated code), making each further change harder. Refactoring prevents this:

- The team looks for possible software improvements and makes them **even where there is no immediate need**.
- It improves the **understandability** of the software and so **reduces the need for documentation**.
- **Changes are easier** to make because the code is well-structured and clear.

- **However**, some changes require **architecture refactoring**, and this is **much more expensive**. Delivery pressure can also delay refactoring.

Examples of refactoring ★★★ MUST KNOW

1. **Re-organization of a class hierarchy** to remove duplicate code (Fig 3.6).
2. **Tidying up and renaming attributes and methods** to make them easier to understand.
3. **Replacement of inline code** with calls to methods that have been included in a program library.

Development environments usually provide **refactoring tools** that find dependencies between code sections and make global changes.

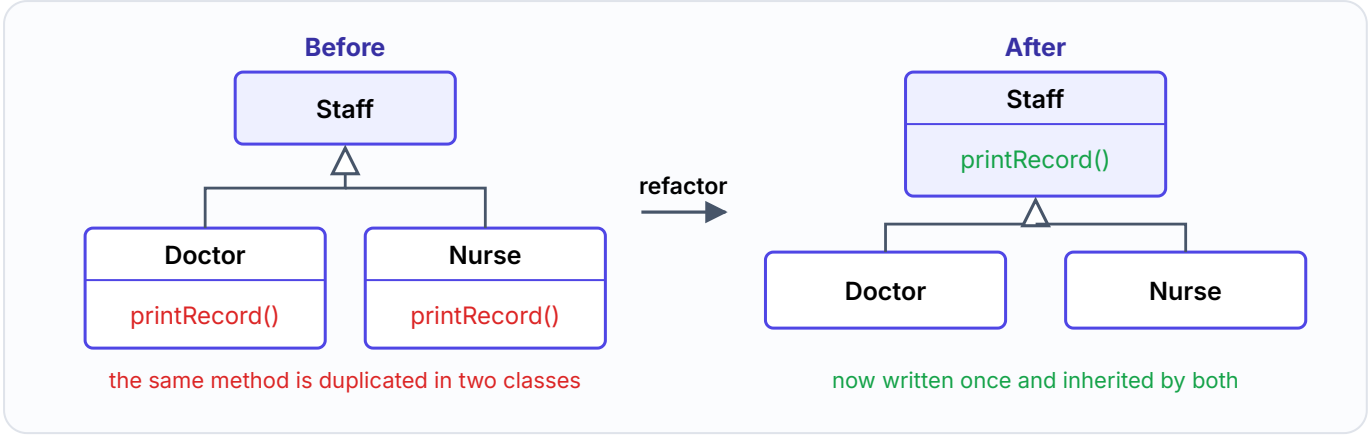


Fig 3.6 Refactoring a class hierarchy to remove duplicate code. Behaviour is unchanged, but *printRecord()* is now changed in one place. Drawn for this guide

3.6 Test-first development ★★★ MUST KNOW

Incremental development has no full specification for an external test team to work from. So in XP, **testing is central**: the program is **tested after every change**, and development cannot proceed until all tests pass. **XP testing features:**

1. **Test-first development**: tests are written before the code.
2. **Incremental test development from scenarios** (user stories and their tasks).
3. **User involvement** in test development and validation.
4. **Automated test harnesses** run all component tests each time a new release is built.

Test-driven development (TDD) ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
>
> **Test-first (test-driven) development** — instead of writing code and then tests for it, you write the tests **before** the code, so they can be run while the code is written and problems are found during development.

- **Writing tests before code clarifies the requirements** to be implemented. A test defines the interface and the expected behaviour, so ambiguities must be cleared up before coding.
- **Tests are written as programs rather than data** so that they can be executed automatically. The test includes a check that it has executed correctly. It usually relies on a testing framework such as **JUnit** (for Java).

- **All previous and new tests are run automatically** when new functionality is added, checking that it has not introduced errors.
- Each task card generates one or more **unit tests**, and testing never falls behind coding ("test-lag" is avoided).

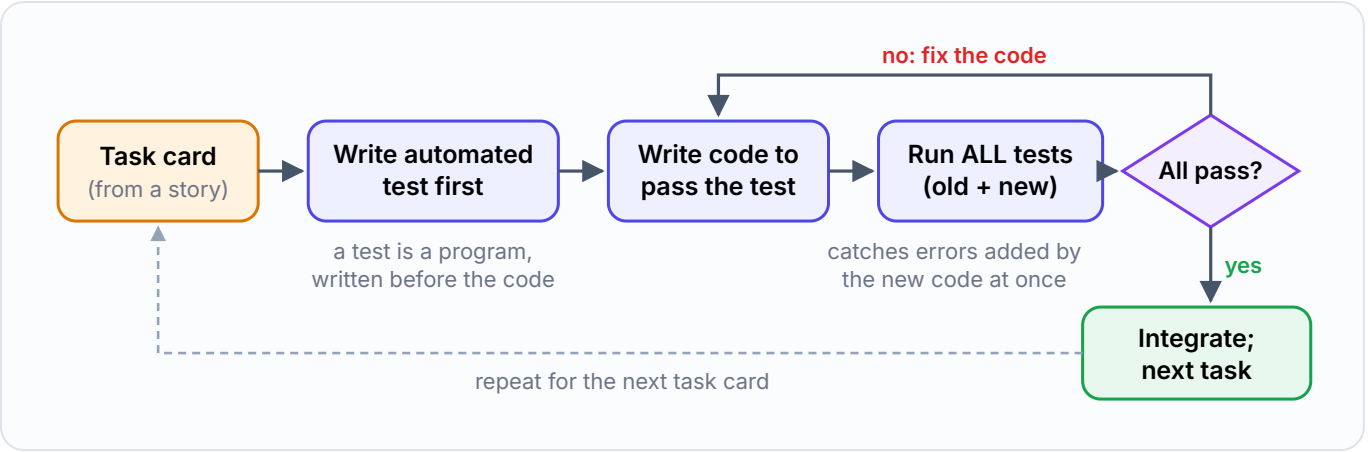


Fig 3.7 The test-first rhythm for one task. Drawn for this guide

Example: test case for dose checking ★★ IMPORTANT

This test checks Task 3 of Fig 3.5. It covers the wrong cases as well as the correct one, and would be written as a JUnit program *before* the dose-checking code.

Test 4: Dose checking

Input:
1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Output:
OK or error message indicating that the dose is outside the safe range.

Tests:
1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Fig 3.8 Test case description for dose checking. Sommerville Fig 3.7 · Your slides, slide 28

Customer involvement

- The customer helps develop **acceptance tests** (tests with customer data that check real needs are met) for the stories in the next release.
- The customer in the team **writes tests as development proceeds**, so all new code is validated against what the customer needs.
- **But** customers have limited time, cannot work full-time with the team, and may feel that giving the requirements was enough, so they may be reluctant to help with testing.

Test automation

- Tests are written as **executable components before the task is implemented**.
- They should be **stand-alone**, **simulate the submission of input**, and **check the result meets the output specification**.
- An **automated test framework** (e.g. JUnit) makes it easy to write executable tests and submit a set of tests for execution.
- There is **always a set of tests that can be quickly run**, so problems introduced by new code are caught immediately.

Problems with test-first development ★★★ MUST KNOW

1. **Programmers prefer programming to testing** and sometimes take short cuts, e.g. incomplete tests that do not check for all possible exceptions.
2. **Some tests are very difficult to write incrementally**, e.g. unit tests for the "display logic" and workflow between screens of a complex user interface.
3. **It is difficult to judge the completeness of a set of tests.** You may have many tests but not complete coverage, so crucial parts stay untested. A big test set can give a false impression that the system is correct.

3.7 Pair programming ★★★ MUST KNOW

> DEFINITION — LEARN THIS LINE
>
> **Pair programming** — programmers work in pairs, sitting together at the same computer, developing the code together. Pairs are created dynamically, so that all team members work with each other during development.

Advantages

1. **Common ownership of code; knowledge spread across the team.** The team as a whole owns the software and shares responsibility for its problems.
2. **Informal review process**: each line of code is looked at by more than one person. This finds fewer errors than formal inspections but is cheaper and easier to organize.
3. **Encourages refactoring**, because the whole team benefits from improving the system code.
4. **Reduces project risk**: the knowledge sharing is very important because it reduces the overall risk to a project when team members leave.

Is it inefficient? ★★ IMPORTANT

Pair programming is **not necessarily inefficient**: there is some evidence that a pair is more efficient than two programmers working separately. Results are mixed, though. With student volunteers (Williams et al., 2000), productivity was **comparable** to two individuals (fewer false starts and less rework); with experienced programmers (Arisholm et al., 2007), it showed a **significant loss**. The knowledge sharing alone may make it worthwhile.

3.8 Agile project management and Scrum ★★★ MUST KNOW

Why agile project management? ★★ IMPORTANT

The principal responsibility of software project managers is to manage the project so that the software is **delivered on time and within the planned budget**. The standard approach is **plan-driven**: managers draw up a plan showing **what** should be delivered, **when**, and **who** will work on the deliverables. Agile project management requires a **different approach**, adapted to incremental development and agile practices, which still gives managers **visibility** of progress.

Scrum ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
>
> **Scrum** — an agile method that focuses on **managing iterative development** rather than on specific agile practices. It provides a project management framework centred round a set of **sprints**, which are fixed time periods when a system increment is developed.

Scrum uses new terms (e.g. *ScrumMaster*, not *project manager*) and does not mandate practices such as pair programming, so it fits existing company practice; it is the **most widely used** agile method. Your slides give **three phases**:

1. **Outline planning phase**: establish the general objectives for the project and design the software architecture.
2. **A series of sprint cycles**, where each cycle develops an increment of the system.
3. **Project closure phase**: wraps up the project, completes required documentation such as system help frames and user manuals, and assesses the lessons learned.

Scrum terminology ★★★ MUST KNOW

Scrum term	Definition (Sommerville Fig 3.8)
Development team	A **self-organizing** group of software developers, which should be **no more than 7 people**. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. "Potentially shippable" means that it is in a **finished state** and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	A list of **"to do" items** which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to **identify product features or requirements, prioritize these** for development and **continuously review the product backlog** to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.
Scrum	A **daily meeting** of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a **short face-to-face meeting** that includes the whole team (the daily Scrum or stand-up).
ScrumMaster	Responsible for **ensuring that the Scrum process is followed** and guides the team in the effective use of Scrum. He or she is responsible for **interfacing with the rest of the company** and for ensuring that the Scrum team is **not diverted by outside interference**. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A **development iteration**. Sprints are usually **2–4 weeks** long.
Velocity	An **estimate of how much product backlog effort that a team can cover in a single sprint**. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

Also used: the **sprint backlog** (the items selected for one sprint) and the **Scrum board** (a shared whiteboard with post-it notes showing the sprint backlog, work done, staff unavailability, etc.).

The Scrum sprint cycle ★★★ MUST KNOW

1. **Product backlog** (Fig 3.9): the starting point for planning; the list of work to be done on the project (features, requirements, user stories, engineering tasks).
2. **Review work and select items**: the Product Owner prioritizes the backlog. In the **selection phase** the **whole project team works with the customer** to select the features and functionality to be developed during the sprint.
3. **Plan sprint**: the team estimates the work using its **velocity**, producing the **sprint backlog**. Once these are agreed, the **team organizes itself** to develop the software.
4. **Sprint**: fixed length, normally **2–4 weeks**. The **team is isolated from the customer and the organization**; all communication goes through the **Scrum master**, whose role is to **protect the team from external distractions**. Daily Scrums track progress.
5. **Review sprint**: at the end of the sprint the work done is **reviewed and presented to stakeholders**. The **next sprint cycle then begins**.

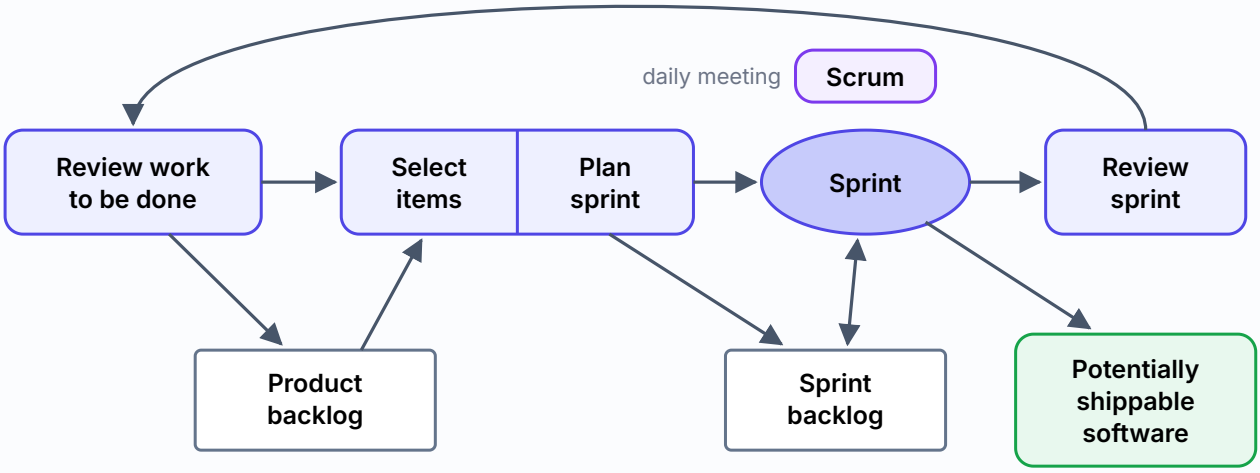


Fig 3.9 The Scrum sprint cycle. Input: the product backlog; output of each cycle: potentially shippable software. Sommerville Fig 3.9 · Your slides, slide 38

⚠ WATCH OUT

Sprints are **never extended** to finish late work; unfinished items go back to the product backlog.



Fig 3.10 One sprint as a timeline. Drawn for this guide

Teamwork in Scrum ★★ IMPORTANT

- The **Scrum master is a facilitator** who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside the team.
- The **whole team attends short daily meetings (Scrums)** to share progress since the last meeting, problems that have arisen and plans for the following day, so everyone knows what is going on and can **re-plan short-term work** to cope with problems.

Scrum benefits ★★★ MUST KNOW

1. The product is broken down into a set of **manageable and understandable chunks**.
2. **Unstable requirements do not hold up progress.**
3. The **whole team has visibility of everything**, so team communication is improved.
4. Customers see **on-time delivery of increments** and gain **feedback** on how the product works.
5. **Trust between customers and developers** is established, and a positive culture is created in which everyone expects the project to succeed.

MEMORY TRICK

Benefits = **C-U-V-F-T**: **C**hunks, **U**nstable requirements no problem, **V**isibility, **F**eedback and on-time delivery, **T**rust ("Clever Users Value Fast Teams").

Distributed Scrum ★★ IMPORTANT

Scrum was designed for **co-located** teams; today teams are often spread worldwide (cheaper staff, specialist skills, 24-hour working). Distributed Scrum needs:

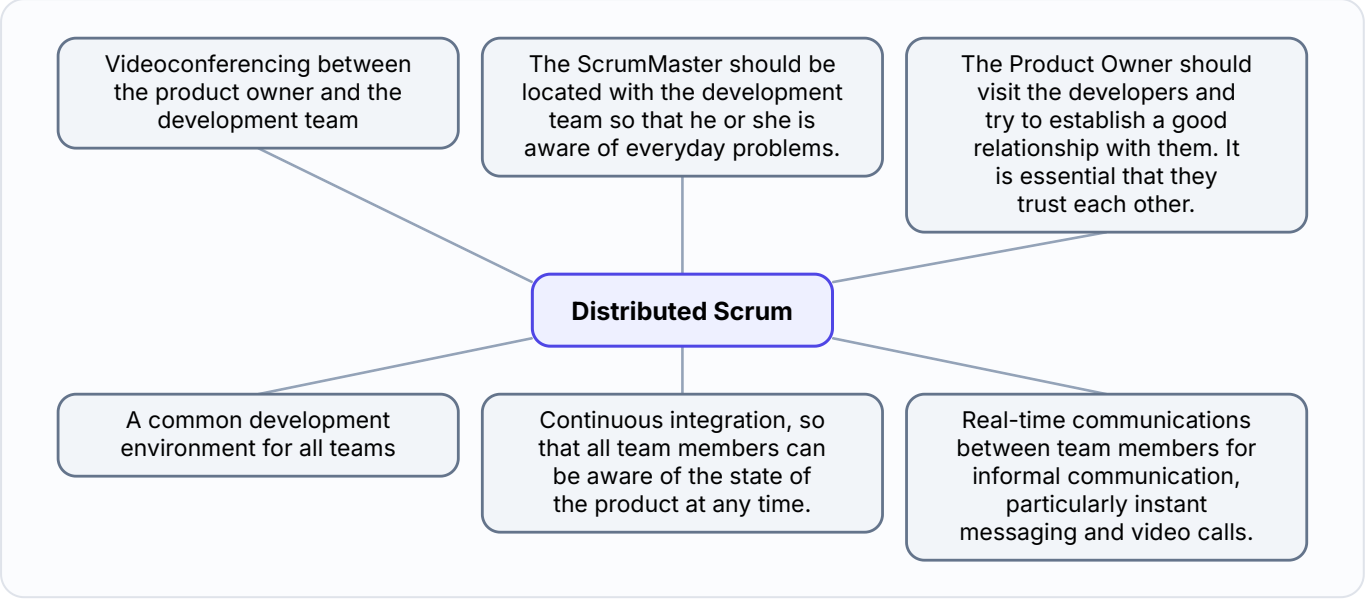


Fig 3.11 Distributed Scrum. Sommerville Fig 3.10 · Your slides, slide 43

3.9 Scaling agile & agile vs plan-driven in practice ★ READ ONCE

This section covers slides 44–64, which are in your slides but not named in the syllabus list. Read it once; learn only "Agile vs plan-driven: which factors decide" (★★).

Scaling agile methods ★ READ ONCE

Agile methods succeed for **small and medium-sized projects** built by a **small co-located team** (thanks, it is argued, to better communication). Scaling means coping with **larger, longer projects with multiple teams**, perhaps in different places. **Scaling up**: agile for **large systems** too big for a small team. **Scaling out**: agile

across a large organization with long development experience. Keep the **fundamentals**: flexible planning, frequent releases, continuous integration, test-driven development, good communications.

Practical problems with agile methods ★ READ ONCE

Problem	Detail
Contracts	Agile informality is **incompatible with the legal approach to contracts**. Specification-based contracts preclude interleaving, so contracts must **pay for developer time, not functionality**, which legal departments see as high risk.
Maintenance	Agile suits new development, yet most money goes on **maintenance**. Issues: are agile systems **maintainable** with minimal documentation, and can agile **evolve a system** for change requests? Key problems: **lack of product documentation; keeping customers involved; development team continuity**. Agile relies on the team knowing what has to be done, but in long-lifetime systems the original developers will not always be there.
Distributed teams	Agile methods are designed for **small co-located teams**, yet much development now uses worldwide distributed teams.

Agile principles and organizational practice ★ READ ONCE

Principle	Why it is hard in practice
Customer involvement	Needs a customer willing and able to spend time with the team and represent all stakeholders; representatives are busy, and external stakeholders (regulators) are hard to represent.
Embrace change	Prioritizing changes is extremely hard when many stakeholders differ.
Incremental delivery	Short-term planning doesn't fit longer business and marketing cycles; marketing may need features months ahead.
Maintain simplicity	Under delivery pressure there may be no time for desirable simplifications.
People, not process	Some people's personalities don't suit the intense involvement of agile methods.

Agile vs plan-driven: which factors decide ★★ IMPORTANT

Most projects include **elements of both** plan-driven and agile processes. The right balance depends on three groups of factors:

Issue	Questions to ask (plan-driven is favoured when…)
System issues	**How large?** Too large for a small co-located team that communicates informally. **What type?** Needs a lot of analysis and a detailed design before implementation. **Lifetime?** Long-lifetime, so documentation must tell the support team the developers' intentions. **External regulation?** Regulated, so detailed documentation is needed for the safety case.
People and teams	**How skilled are the designers and programmers?** Agile may need higher skill than translating a detailed design into code. **How is the team organized?** Distributed teams may need design documents. **What support technology?** Without design documents, IDE visualization and program-analysis tools are essential.
Organizational issues	Traditional engineering organizations have a **plan-based culture**. **Is a detailed specification standard practice? Will customer representatives be available to give feedback on increments** (is incremental delivery realistic)? **Can informal agile fit a culture of detailed documentation?**

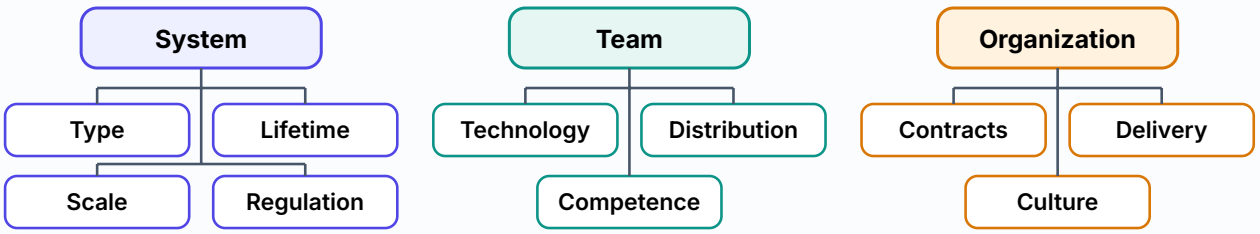


Fig 3.12 Factors influencing the choice of plan-based or agile development. Sommerville Fig 3.12 · Your slides, slide 54

Agile methods for large systems ★ READ ONCE

Six factors make large systems hard for agile (slide 60 draws them around "Large software system"):

- **System of systems**: separate communicating systems, each built by its own team, often in different places and time zones.
- **Brownfield development**: interaction with existing systems resists incremental development.
- **System configuration**: much of the work is configuration, not original code.
- **Regulatory constraints**: external rules limit how they are developed.
- **Prolonged procurement**: long timescales make coherent teams hard to keep.
- **Diverse stakeholders**: impossible to involve them all.

IBM's agility at scale model (slide 61; three nested circles). **Core agile development**: value-driven life-cycle, self-organizing teams, focus on construction. **Disciplined agile delivery**: risk+value driven life-cycle, self-organizing with appropriate governance framework, full delivery life-cycle. **Agility at scale**: disciplined agile delivery where scaling factors apply: large team size, geographic distribution, regulatory compliance, domain complexity, organization distribution, technical complexity, organizational complexity, enterprise discipline.

Scaling up to large systems

1. A **completely incremental approach to requirements engineering is impossible**.
2. There **cannot be a single product owner** or customer representative.
3. It is **not possible to focus only on the code** of the system.
4. **Cross-team communication mechanisms** have to be designed and used.
5. **Continuous integration is practically impossible**; keep frequent builds and regular releases.

Multi-team Scrum

1. **Role replication**: each team has a Product Owner for its work component, and a ScrumMaster.
2. **Product architects**: one per team; together they design and evolve the overall architecture.
3. **Release alignment**: release dates aligned so a demonstrable, complete system is produced.
4. **Scrum of Scrums**: a daily meeting of team representatives to discuss progress and plan work.

Agile across organizations is hard because of (1) **project managers** reluctant to risk a new approach; (2) bureaucratic **quality procedures and standards**; (3) a wide **range of skill levels** (agile works best with high skills); (4) **cultural resistance** after long use of conventional processes.

Chapter 3 in one page

Core definitions

- **Agile methods**: incremental; rapid development, frequent releases, minimal documentation, high-quality code, customer involved.
- **Plan-driven**: stages, outputs planned in advance, iteration *within* activities. **Agile**: interleaved, outputs negotiated, iteration *across* activities.
- **User story**: a scenario of use on a story card, broken into tasks. **Refactoring**: constant code improvement. **Test-first**: automated tests written before the code.

Agile Manifesto (left over right)

- Individuals and interactions / processes and tools · Working software / comprehensive documentation · Customer collaboration / contract negotiation · Responding to change / following a plan

5 principles ("Can I Pass Every Module?")

- Customer involvement · Incremental delivery · People, not process · Embrace change · Maintain simplicity

10 XP practices

- Incremental planning · Small releases · Simple design · Test-first · Refactoring · Pair programming · Collective ownership · Continuous integration · Sustainable pace · On-site customer

XP release cycle

- Select stories → Break into tasks → Plan release → Develop/integrate/test → Release → Evaluate → repeat

Problems to remember

- Stories: completeness. Test-first: short-cut tests, hard UI tests, unknown coverage. Pairing: mixed productivity evidence.

Scrum

- Product Owner · ScrumMaster · Development team (≤ 7) · product backlog · sprint (2–4 weeks) · velocity · daily Scrum · potentially shippable increment.
- Cycle: backlog → select items → plan sprint → sprint → review sprint → repeat. Benefits: C-U-V-F-T.

Key points

- Many real methods mix plan-based and agile. Scaling agile to large systems is hard: they need up-front design and some documentation.

Diagrams to practise

- Fig 3.1 · Fig 3.3 · Fig 3.5 · Fig 3.9 · Fig 3.11

Likely exam questions

2M **Define agile methods. What is their aim?**

How to answer → Definition (3.1) + aim: reduce overheads, respond quickly to change without excessive rework (3.2).

4M **State the Agile Manifesto.**

How to answer → Opening sentence, four "X over Y" lines verbatim, closing sentence.

6M **Explain the principles of agile methods.**

How to answer → Five-row table in 3.2.

6M **Differentiate plan-driven and agile development with a neat diagram.**

How to answer → Fig 3.1 → comparison table (iteration within vs across activities, documents, delivery, change, best use).

10M **Explain Extreme Programming: its release cycle and practices.**

How to answer → Definition → Fig 3.3, one line per box → all 10 practices.

5M **How does XP reflect the principles of agile methods?**

How to answer → Five-row table "XP and agile principles" (3.3).

8M **What are user stories? Explain with the "prescribing medication" example.**

How to answer → Definition → 4 points → story (Fig 3.4) → task cards (Fig 3.5) → completeness problem.

5M **What is refactoring? Why is it needed? Give examples.**

How to answer → Definition; design for change vs refactoring; benefits and architecture caveat; 3 examples (Fig 3.6).

8M **Explain test-first development in XP and its problems.**

How to answer → 4 testing features → TDD points (JUnit) → Fig 3.8 → customer role, automation → 3 problems.

5M **What is pair programming? State its advantages.**

How to answer → Definition → 4 advantages → productivity evidence.

2M **Define: sprint, velocity, product backlog, ScrumMaster, Product Owner.**

How to answer → Scrum terminology table (3.8).

10M **Explain the Scrum sprint cycle with a neat diagram.**

How to answer → Fig 3.9 → 5 steps → never extended → ScrumMaster's role → three phases of Scrum.

5M **Explain teamwork in Scrum and the benefits of Scrum.**

How to answer → ScrumMaster duties + daily Scrum → 5 benefits.

4M **What are the requirements of Distributed Scrum?**

How to answer → Fig 3.11, six points.

6M **Which factors decide between agile and plan-driven development?**

How to answer → Fig 3.12 → system, team and organizational questions (3.9).

CIE-1 · UNIT 2

Unit 2: Requirements Engineering & Design Concepts

2

How to find out, write down, check and manage what a system must do, and the core ideas every good software design is built on.

SYLLABUS, WORD FOR WORD

Requirements engineering: Functional and Non-functional requirements, Requirements engineering processes, Requirements elicitation, Requirements specification, Requirements validation, Requirements change.

Design Concepts: Design Within the Context of Software Engineering, Design Concepts, The Design Model.

CHAPTERS IN THIS UNIT

CHAPTER 4 — **Requirements Engineering**
Functional & non-functional requirements · RE processes · Elicitation · Specification · Validation · Change
page 65

CHAPTER 5 — **Design Concepts**
Design in software engineering · The design process · Design concepts · The design model
page 101

COURSE OUTCOME

CO2: Analyze the functional and non-functional requirements and design concepts for the given problem.

LAB COMPONENT

Analysis of the real-time case study application.

CIE-1 SCOPE

Whole unit is in CIE-1.

UNIT 2 · CHAPTER 4

Requirements Engineering

Functional & non-functional requirements · RE processes · Elicitation · Specification · Validation · Change

STUDY TIME
About 5 hours (two sittings)

SOURCES
Your Ch 4 slides (88 slides) + Sommerville Ch 4

EXAM WEIGHT
Very high: FR vs NFR, NFR types, elicitation, SRS structure and validation are asked almost every time

AFTER THIS CHAPTER YOU CAN

- Define RE, user requirements and system requirements
- Differentiate functional, non-functional and domain requirements
- Draw the NFR classification tree with Mentcare examples
- Turn a vague goal into a verifiable NFR using metrics
- Draw the spiral RE process and the elicitation cycle
- Compare interviews, ethnography, stories and scenarios
- Write requirements as sentences, forms, tables and use cases
- Give the structure of a software requirements document
- List validation checks, techniques and review checks
- Explain requirements management and change management

4.1 Requirements, readers and stakeholders ★★★ MUST KNOW

IN SIMPLE WORDS

Before an architect draws a house, the owner says what is needed: "three bedrooms, a kitchen facing east, a total cost under 40 lakh rupees, and it must follow the city building code". The first two are *services* the house must provide; the last two are *constraints*. Software is the same. The list of services and constraints is the **requirements**; the work of finding, writing and checking that list is **requirements engineering**.

The **requirements** for a system are the descriptions of the **services** that a system should provide and the **constraints** on its operation. They reflect what customers need from a system that serves a purpose such as controlling a device, placing an order or finding information.

DEFINITION — LEARN THIS LINE

Requirements engineering (RE) — the process of establishing the services that a customer requires from a system and the constraints under which it operates and is developed.

System requirements — the descriptions of the system services and constraints that are generated during the requirements engineering process.

Sommerville's wording is also worth quoting: the process of finding out, analysing, documenting and checking these services and constraints is called requirements engineering.

What is a requirement? ★★ IMPORTANT

The term is not used consistently in industry. A requirement **may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification**. This is inevitable because requirements may serve a **dual function**:

- They may be the **basis for a bid for a contract** — therefore they must be *open to interpretation*, so that different companies can propose different solutions.
- They may be the **basis for the contract itself** — therefore they must be *defined in detail*.
- Both these statements may be called "requirements".

★ REQUIREMENTS ABSTRACTION (DAVIS)

"If a company wishes to let a contract for a large software development project, it must define its needs in a sufficiently abstract way that a solution is not pre-defined. The requirements must be written so that several contractors can bid for the contract, offering, perhaps, different ways of meeting the client organization's needs. Once a contract has been awarded, the contractor must write a system definition for the client in more detail so that the client understands and can validate what the software will do. Both of these documents may be called the requirements document for the system."

In short: an **abstract** version is written so that contractors can bid; a **detailed** version is written after the contract is awarded so that the client can check (validate) what will be built. Many problems in RE come from not separating these two levels clearly — which is why we name them separately.

User requirements and system requirements ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

User requirements — statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.

System requirements — a structured document setting out detailed descriptions of the system's functions, services and operational constraints. It defines what should be implemented, so it may be part of a contract between client and contractor.

Basis	User requirements	System requirements
Level of detail	High-level, abstract; may range from broad feature statements to fairly precise descriptions	Detailed, precise descriptions of functions, services and operational constraints
Notation	Natural language plus diagrams	Structured document (natural language, forms, tables, models)
Written for	Customers: client managers, end-users	Engineers: system architects, software developers
Purpose	Agree what services are needed; basis for a bid	Define exactly what is to be implemented; may be part of the contract

Basis	User requirements	System requirements
Also called	User requirements definition	System requirements specification / functional specification

Fig 4.1 shows the difference using the **Mentcare** system (a patient information system for mental health care, one of the textbook case studies). One general user requirement is expanded into five specific system requirements.

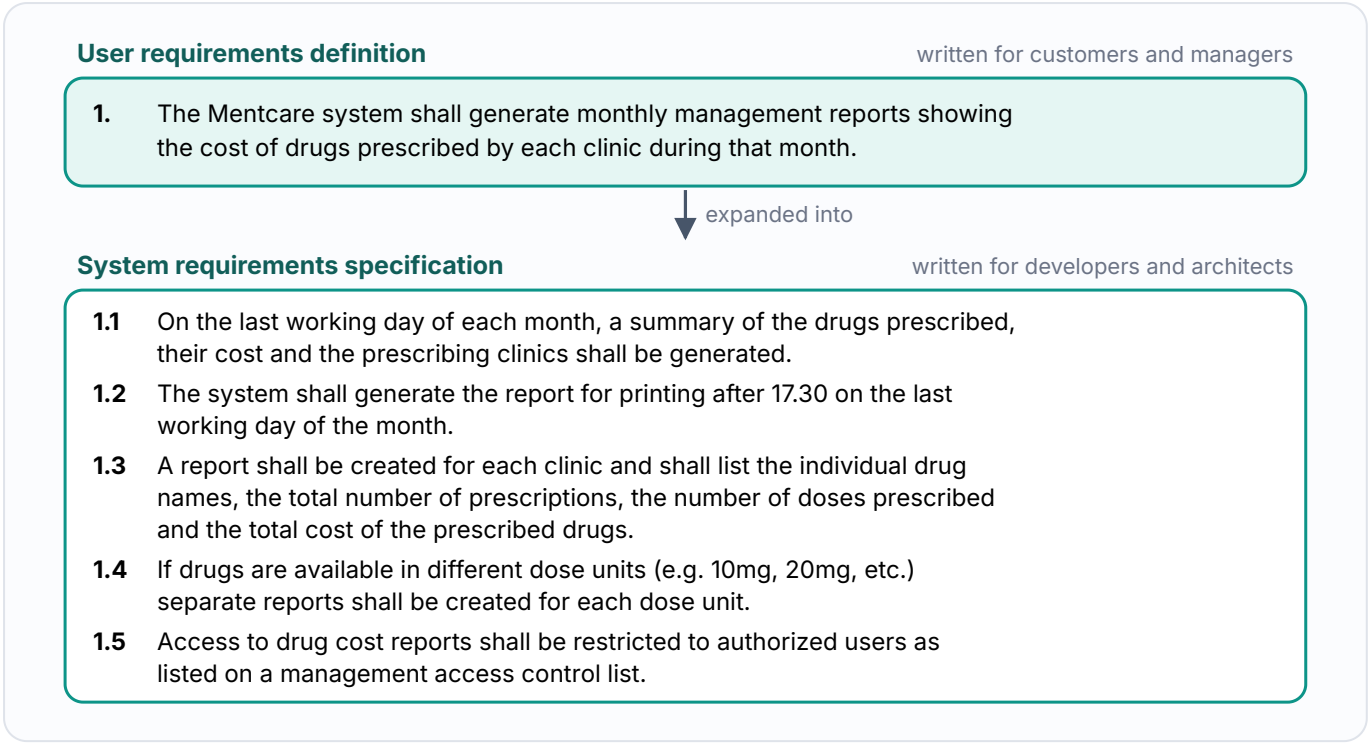


Fig 4.1 User and system requirements: one user requirement expanded into five system requirements. Sommerville Fig 4.1 · Your slides, slide 7

Requirements are written at different levels of detail because **different readers use them in different ways** (Fig 4.2). Readers of user requirements are not usually concerned with how the system will be implemented; readers of system requirements need to know more precisely what the system will do, because they must support business processes with it or are involved in implementing it.

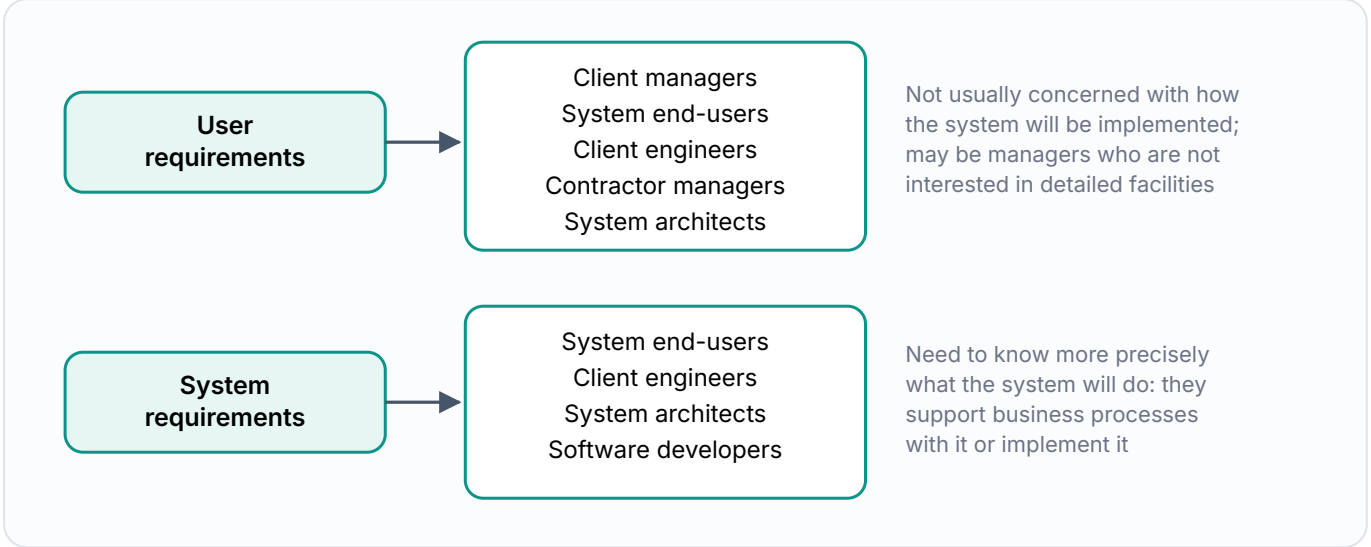


Fig 4.2 Readers of different types of requirements specification. Sommerville Fig 4.2 · Your slides, slide 8

✎ EXAM TIP

For "Differentiate user and system requirements" (4–5 marks): write both definitions, a 4-row table, the Mentcare drug-cost example (one user requirement → system requirements 1.1–1.5) and draw Fig 4.2. The figure alone earns easy marks.

System stakeholders ★★★ MUST KNOW

The readers in Fig 4.2 are examples of **stakeholders**. Besides users, many other people have an interest in a system.

DEFINITION — LEARN THIS LINE

System stakeholder — any person or organization who is affected by the system in some way and so who has a legitimate interest in it.

Stakeholder types (range from end-users through managers to external stakeholders):

- **End users** — the people who use the system in their daily work.
- **System managers** — the people who manage and run the system.
- **System owners** — the organisation that pays for and owns the system.
- **External stakeholders** — e.g. regulators, who certify the acceptability of the system.

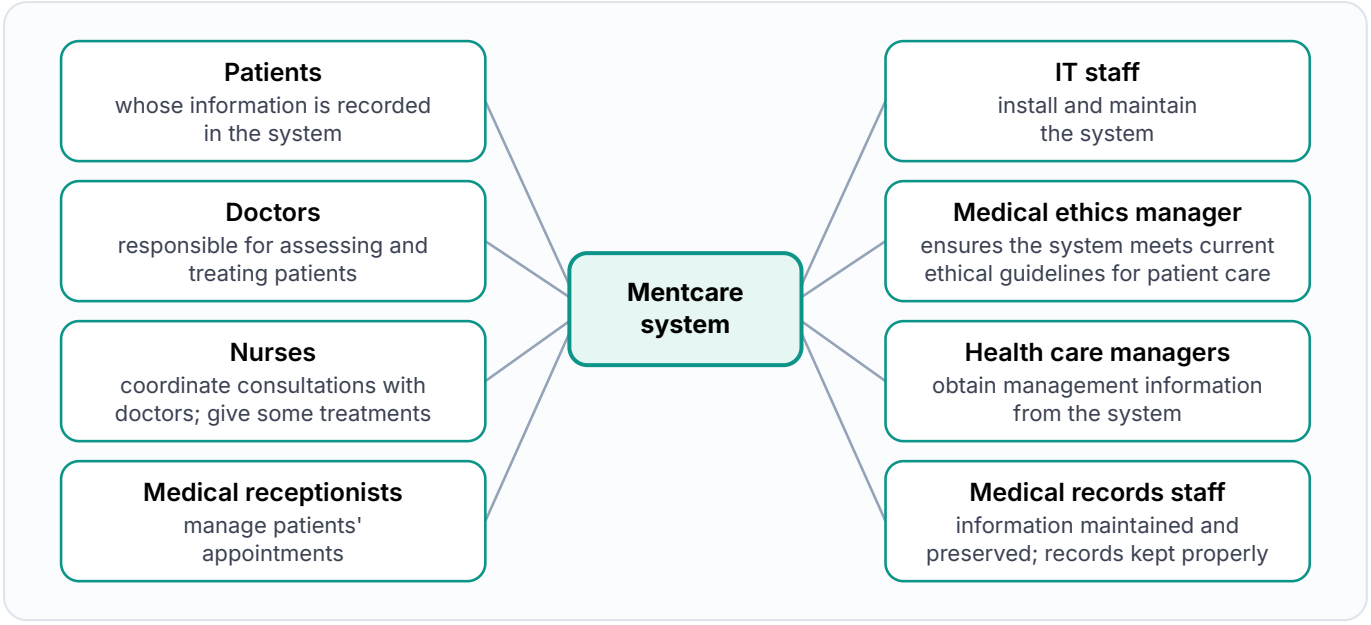


Fig 4.3 The eight stakeholder groups of the Mentcare system (the textbook also includes patients' relatives). Drawn for this guide from your slides, slides 10–11

Before RE starts: the feasibility study ★ READ ONCE

Some understanding of the requirements is needed before deciding to build or buy a system at all. This early RE gives a high-level view of what the system might do and its benefits, which is assessed in a **feasibility study** — a short, focused study early in the RE process that answers three questions: (1) Does the system contribute to the overall objectives of the organisation? (2) Can it be implemented within schedule and budget using current technology? (3) Can it be integrated with other systems that are used? If any answer is "no", you should probably not go ahead.

Agile methods and requirements ★★ IMPORTANT

- Many agile methods argue that producing detailed system requirements is a **waste of time**, as requirements change so quickly.
- The requirements document is therefore **always out of date**.
- Agile methods usually use **incremental requirements engineering** and may express requirements as **user stories** (Chapter 3), written on cards or whiteboards; the user prioritises the stories for the next increment.
- This is practical for **business systems** but problematic for systems that require **pre-delivery analysis** (e.g. critical systems) or systems developed by **several teams**.

Sommerville's middle path: even with agile development, write a **short supporting document** that defines the business and dependability requirements, because requirements that apply to the whole system are easily forgotten when you focus on the features of the next release.

4.2 Functional and non-functional requirements ★★★ MUST KNOW

IN SIMPLE WORDS

Functional = what the system *does* (search, generate a report, identify a user). **Non-functional** = how well it must do it and under which rules (how fast, how reliable, how secure, which laws and standards apply). For a car, "the car shall have brakes" is functional; "it shall stop from 100 km/h within 40 metres" and "it shall meet emission laws" are non-functional.

DEFINITION — LEARN THIS LINE

Functional requirements — statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations. They may also state what the system should *not* do.

Non-functional requirements — constraints on the services or functions offered by the system, such as timing constraints, constraints on the development process, standards, etc. They often apply to the system as a whole rather than to individual features or services.

Domain requirements — constraints on the system from the domain of operation (the application area, e.g. medicine, banking, railways).

The line between them is not sharp. A user requirement about security, such as limiting access to authorised users, looks non-functional. When developed in more detail it generates requirements that are clearly functional, such as including user authentication facilities. Requirements are not independent: one requirement often **generates or constrains** other requirements (Fig 4.4).

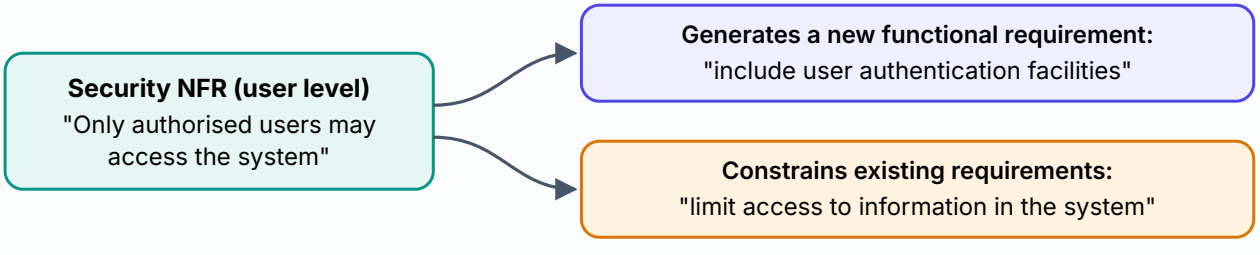


Fig 4.4 One non-functional requirement can generate new functional requirements and restrict existing ones. Drawn for this guide from Sommerville 4.1

Functional requirements ★★★ MUST KNOW

- Describe **functionality or system services** — what the system should do. They are statements of the services the system must provide, or descriptions of **how some computations must be carried out**.
- Depend on the **type of software**, the **expected users** and the **type of system** where the software is used.
- **Functional user requirements** may be high-level statements of what the system should do. They are written in natural language so that users and managers can understand them.
- **Functional system requirements** should describe the system services in detail — the functions, their inputs and outputs, and exceptions. They are written for developers.

> **EXAMPLE — MENTCARE FUNCTIONAL REQUIREMENTS**
>
> 1. A user shall be able to search the appointments lists for all clinics.
> 2. The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
> 3. Each staff member using the system shall be uniquely identified by his or her 8-digit employee number.
>
> Functional requirements can be written at different levels of detail: requirement 1 is general, requirement 3 is very specific.

Information requirements: if an off-the-shelf product will meet the need, a detailed functional specification is pointless. Focus instead on **information requirements** — what information people need to do their work and how it is delivered and organised (e.g. what goes into the daily list of expected patients).

Domain requirements ★★ IMPORTANT

Domain requirements are derived from the **application domain** of the system rather than from the specific needs of system users. They may:

- be **new functional requirements** in their own right,
- **constrain existing** functional requirements, or
- set out **how particular computations must be carried out**.

The problem with domain requirements: software engineers may not understand the characteristics of the domain, so they may not know whether a domain requirement has been **missed out** or **conflicts** with other requirements. Domain requirements are found during requirements discovery, from stakeholders and from documentation (Section 4.4). For instance, the insulin pump's rule for computing a dose from the rate of change of blood sugar (Section 4.5) comes from medical knowledge, not from a user's personal wish — it sets out how a computation must be carried out.

Requirements imprecision ★★★ MUST KNOW

- Problems arise when functional requirements are **not precisely stated**.
- **Ambiguous** requirements may be interpreted in different ways by developers and users.
- A developer naturally chooses the interpretation that is **simplest to implement**. If that is not what the customer wanted, new requirements and changes are needed — causing disputes, **delayed delivery and higher cost**.

Consider the term **"search"** in Mentcare requirement 1:

User intention

Search for a patient name across **all appointments in all clinics**. Rationale: patients with mental health problems are sometimes confused and may go to a different clinic from the one booked.

Developer interpretation

Search for a patient name in **an individual clinic**: the user chooses a clinic, then searches. Easier to build, but needs more user input and takes longer.

Requirements completeness and consistency ★★★ MUST KNOW

In principle, requirements should be both complete and consistent:

Complete They should include descriptions of all facilities (services and information) required.

Consistent There should be no conflicts or contradictions in the descriptions of the system facilities.

In practice, because of system and environmental complexity, it is **impossible to produce a complete and consistent requirements document** (except for very small systems). Two reasons: (1) it is easy to make **mistakes and omissions** when writing specifications for large, complex systems; (2) large systems have **many stakeholders** with different backgrounds and often inconsistent needs, and the inconsistencies may only appear after deeper analysis or during development.

Non-functional requirements ★★★ MUST KNOW

- NFRs define **system properties and constraints**, e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations (the data representations used in interfaces), etc.
- **Process requirements** may also be specified, mandating a particular IDE, programming language or development method.
- NFRs often relate to **emergent properties** — properties of the system as a whole (such as reliability, response time, memory use) rather than of one component.
- NFRs may be more critical than functional requirements. If these are not met, the system may be useless.

> **EXAMPLE**
>
> Users can usually find a way round a function that does not quite meet their needs, but not round a failed NFR. If an **aircraft** system does not meet its reliability requirements, it will not be certified as safe for operation; if an **embedded control system** fails to meet its performance requirements, the control functions will not operate correctly.

Non-functional requirements implementation

You can usually point to the component that implements a functional requirement (e.g. a formatting component for reports). NFRs are harder to locate — their implementation may be spread throughout the system, for two reasons:

1. NFRs may affect the **overall architecture** of a system rather than the individual components. For example, to ensure that performance requirements are met, you may have to organise the system to minimise communications between components.
2. A single NFR, such as a **security requirement**, may generate a number of related **functional requirements** that define system services that are required. It may also generate requirements that **restrict existing requirements** (Fig 4.4).

Types of non-functional requirement ★★★ MUST KNOW

NFRs arise from user needs because of budget constraints, organisational policies, the need for interoperability with other software or hardware systems, or external factors such as safety regulations or privacy legislation. Sommerville classifies them by **where they come from**: the product, the organisation, or outside (Fig 4.5).

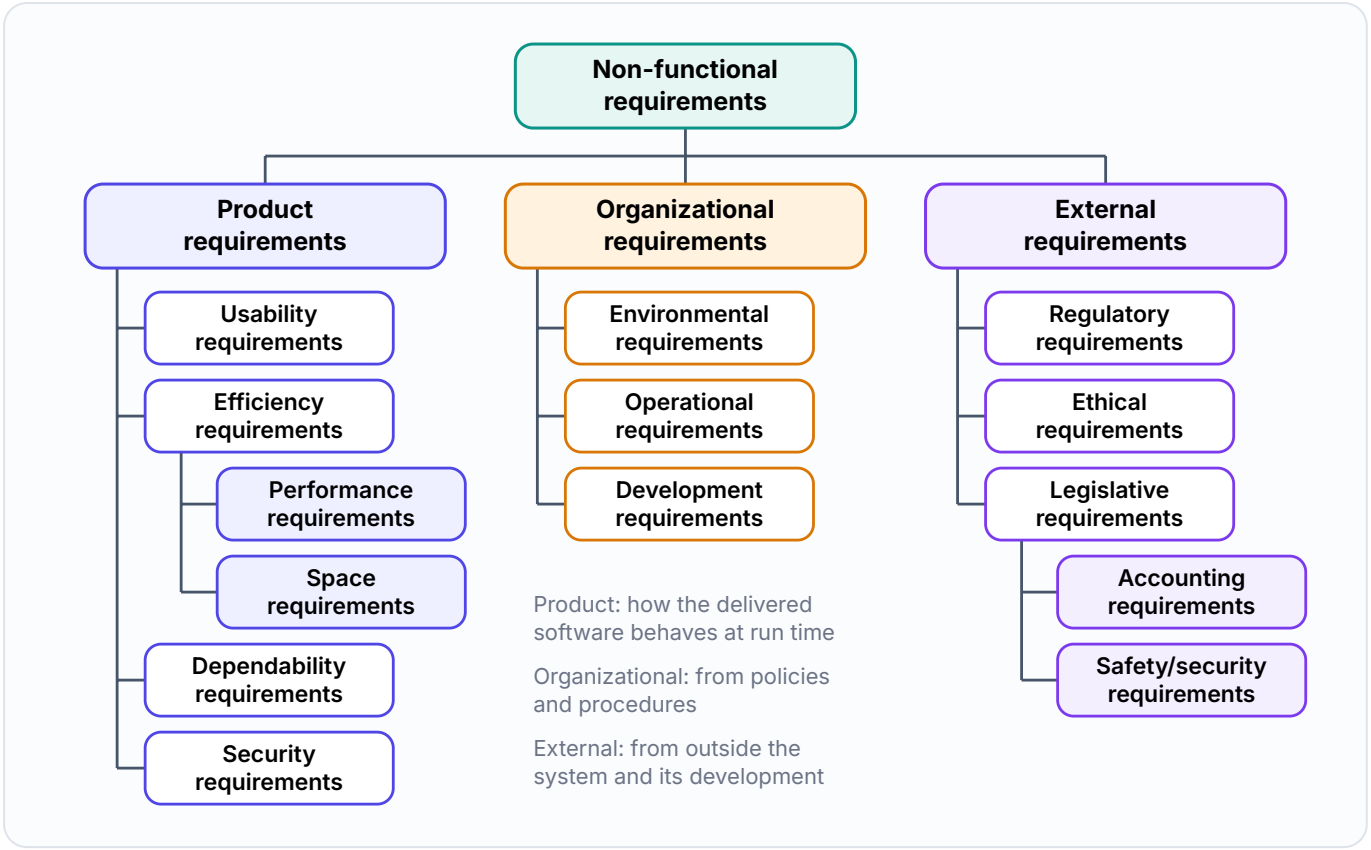


Fig 4.5 Types of non-functional requirement. Shaded boxes are sub-types (Efficiency → Performance, Space; Legislative → Accounting, Safety/security). Sommerville Fig 4.3 · Your slides, slide 20

Class	Meaning (exam wording)	Sub-types and examples
Product requirements	Requirements which specify that the delivered product must behave in a particular way — they specify or constrain the run-time behaviour of the software.	**Usability**; **efficiency** (**performance** — how fast it executes; **space** — how much memory it needs); **dependability** (e.g. reliability — acceptable failure rate); **security**. E.g. execution speed, reliability.
Organisational requirements	Requirements which are a consequence of organisational policies and procedures (in the customer's and developer's organisations).	**Operational** — how the system will be used; **development** — programming language, development environment, process standards; **environmental** — the operating environment of the system. E.g. process standards used, implementation requirements.
External requirements	Requirements which arise from factors which are external to the system and its development process.	**Regulatory** — what must be done for a regulator (e.g. a nuclear safety authority) to approve it; **legislative** — operate within the law (accounting, safety/security); **ethical** — acceptable to users and the general public. E.g. interoperability requirements, legislative requirements.

MEMORY TRICK

Three roots — **P-O-E**: what the **P**roduct does at run time, what the **O**rganisation's rules say, what the world **E**xternal to the project demands.
Leaves: Product → **U-E-D-S** "*U Eat Dosa Slowly*" (Usability, Efficiency [Performance, Space], Dependability, Security) · Organisational → **E-O-D** "*Every Office Decides*" (Environmental, Operational, Development) · External → **R-E-L** "the *REaL* world" (Regulatory, Ethical, Legislative [Accounting, Safety/security]).

Examples of non-functional requirements in the Mentcare system

Type	Requirement	Why it is this type
Product requirement	The Mentcare system shall be available to all clinics during normal working hours (Mon–Fri, 08:30–17:30). Downtime within normal working hours shall not exceed five seconds in any one day.	An **availability** requirement: when the system must be available and the allowed downtime. It says nothing about functionality — it is a constraint for the designers.
Organisational requirement	Users of the Mentcare system shall authenticate themselves using their health authority identity card.	Comes from **policy**: the health authority is moving to a standard procedure where users swipe an identity card instead of using a login name.
External requirement	The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.	Comes from **privacy legislation**: the system must follow a national privacy standard.

Goals and verifiable requirements ★★★ MUST KNOW

Non-functional requirements may be very difficult to state precisely, and imprecise requirements may be difficult to verify. Stakeholders often propose NFRs as general **goals** — ease of use, the ability to recover from failure, rapid user response. Goals set out good intentions but leave room for interpretation and for dispute once the system is delivered.

DEFINITION — LEARN THIS LINE

Goal — a general intention of the user, such as ease of use.

Verifiable non-functional requirement — a statement using some measure that can be objectively tested.

Goals are still helpful to developers, as they convey the intentions of the system users. Whenever possible, though, write NFRs **quantitatively** so they can be objectively tested.

Usability requirement	Statement
Goal (not testable)	"The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized."
Testable NFR	"Medical staff shall be able to use all the system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use."

⚠ WATCH OUT

Your slides say **four** hours of training; the 10th-edition textbook text says **two** hours. Either is accepted — what matters is that the rewritten requirement has **measurable numbers** (training time, errors per hour). These can be checked by software instrumentation that counts user errors during testing.

Why quantifying NFRs is hard: customers find it difficult to translate goals into measurable requirements; for some goals, such as **maintainability**, there are no simple metrics; customers may not relate a number (e.g. for reliability) to their everyday experience; and objectively verifying measurable NFRs can be very expensive.

Metrics for specifying non-functional requirements ★★★ MUST KNOW

These metrics can be measured while the system is being tested, to check whether it meets its NFRs.

Property	Measure	What it tells you
Speed	Processed transactions/second · User/event response time · Screen refresh time	How quickly the system works
Size	Mbytes · Number of ROM chips	How much memory or storage it needs
Ease of use	Training time · Number of help frames	How fast people learn it; how much help they need
Reliability	Mean time to failure · Probability of unavailability · Rate of failure occurrence · Availability	How rarely it fails; how often it is available for use
Robustness	Time to restart after failure · Percentage of events causing failure · Probability of data corruption on failure	How well it copes with and recovers from failure
Portability	Percentage of target dependent statements · Number of target systems	How easily it moves to other platforms

MEMORY TRICK

"**S**ix **S**mart **E**ngineers **R**arely **R**un **P**oorly" → Speed, Size, Ease of use, Reliability, Robustness, Portability.

NFR conflicts and where to put NFRs

- NFRs often **conflict and interact** with other requirements. The Mentcare ID-card requirement needs a card reader on each computer, but another requirement may ask for mobile access from doctors' and nurses' tablets or smartphones, which have no card readers — so an alternative identification method may be needed.
- It is difficult to separate FRs and NFRs in the requirements document. If NFRs are stated separately, relationships are hard to see; ideally, **highlight** NFRs related to emergent properties (performance, reliability) in a separate section or by marking them.
- NFRs such as reliability, safety and confidentiality are particularly important for **critical systems**.

✎ EXAM TIP

A verifiable NFR = **property + measure + number + unit + condition**. For example: "The system shall display the results of a patient-name search within 2 seconds for 95% of searches when 50 users are logged in." Examiners reward the number.

Functional vs non-functional requirements — the comparison ★★★ MUST KNOW

Basis	Functional requirements	Non-functional requirements
Meaning	Services the system should provide; how it reacts to inputs and behaves in particular situations	Constraints on the services or functions; properties of the system
Question answered	What should the system do?	How well, and under what constraints, must it do it?
Scope	Usually one feature or service	Often the system as a whole (emergent properties)
Classification	Functional user requirements and functional system requirements	Product, organisational, external
Mentcare example	Search appointment lists for all clinics; daily list of expected patients; staff identified by 8-digit number	Available 08:30–17:30 with ≤ 5 s downtime per day; ID-card authentication; privacy standard HStan-03-2006-priv
Where implemented	Can usually be traced to particular components	Spread through the system; may shape the whole architecture
If not met	Users can often work around the gap	System may be useless (e.g. not certified as safe)
How verified	Test that the function gives the right output for given inputs	Measure with metrics: response time, MTTF, training time, etc.
Ease of stating	Easier, but must avoid ambiguity ("search")	Harder; often stated as vague goals that must be made measurable

FR vs NFR for an app you use every day ★★ IMPORTANT

Practise on a **food-delivery app**. These requirements are illustrations written for this guide, not textbook text.

#	Requirement	Kind	NFR type (Fig 4.5)
1	The app shall let a customer search restaurants by restaurant name and by dish.	FR	—
2	The app shall show the live location of the delivery partner after the order is picked up.	FR	—
3	The app shall let a customer cancel an order before the restaurant accepts it.	FR	—
4	Search results shall appear within 2 seconds for 95% of searches.	NFR	Product → efficiency (performance)
5	The ordering service shall be available 99.5% of the time in each month.	NFR	Product → dependability

#	Requirement	Kind	NFR type (Fig 4.5)
6	A first-time user shall be able to place an order within 3 minutes without help.	NFR	Product → usability
7	The app shall be developed in Kotlin following the company coding standard.	NFR	Organisational → development
8	Card payments shall comply with the national payment-data regulations.	NFR	External → regulatory / legislative

? QUICK CHECK

"The system shall print a monthly drug-cost report" — FR or NFR? And "Each report shall be generated within 10 seconds"?

Answer: The first is functional (a service). The second is a non-functional product requirement (efficiency → performance); it is verifiable because it contains a number.

4.3 Requirements engineering processes ★★★ MUST KNOW

The processes used for RE **vary widely** depending on the application domain, the people involved and the organisation developing the requirements. However, there are a number of **generic activities common to all processes**:

1. **Requirements elicitation** — discovering requirements by working with stakeholders.
2. **Requirements analysis** — classifying, organising, prioritising and negotiating them (done together with elicitation).
3. **Requirements validation** — checking that the requirements define the system the customer really wants.
4. **Requirements management** — managing changes to the requirements.

The textbook describes the same work as **three key activities**: discovering requirements by interacting with stakeholders (**elicitation and analysis**); converting them into a standard form (**specification**); and checking that they define the system the customer wants (**validation**) — with **requirements management** handling change. In practice, RE is an iterative activity in which these processes are interleaved.

⚠ WATCH OUT

Slides: 4 generic activities (elicitation, analysis, validation, management). Textbook: 3 key activities (elicitation & analysis, specification, validation). In the exam write **elicitation & analysis, specification, validation and management** — that covers both versions.

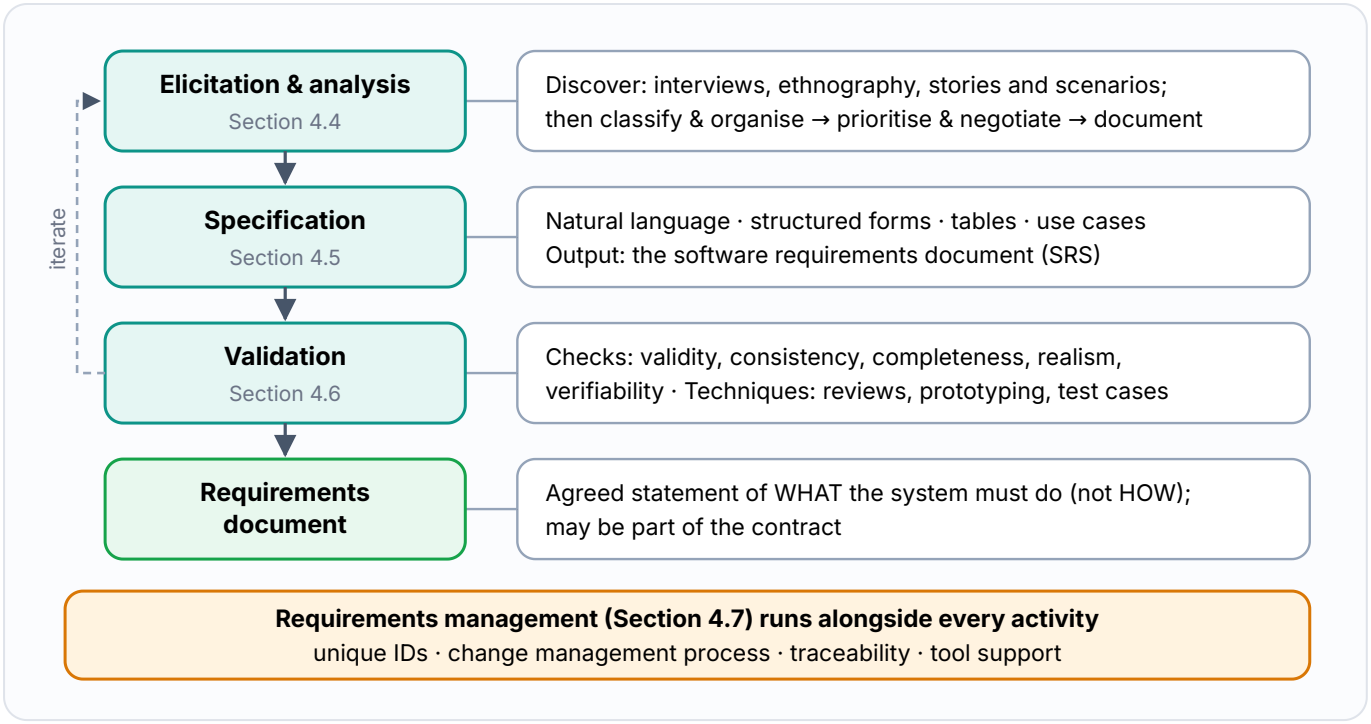


Fig 4.6 Requirements engineering at a glance — a map of this chapter. Drawn for this guide

A spiral view of the requirements engineering process ★★★ MUST KNOW

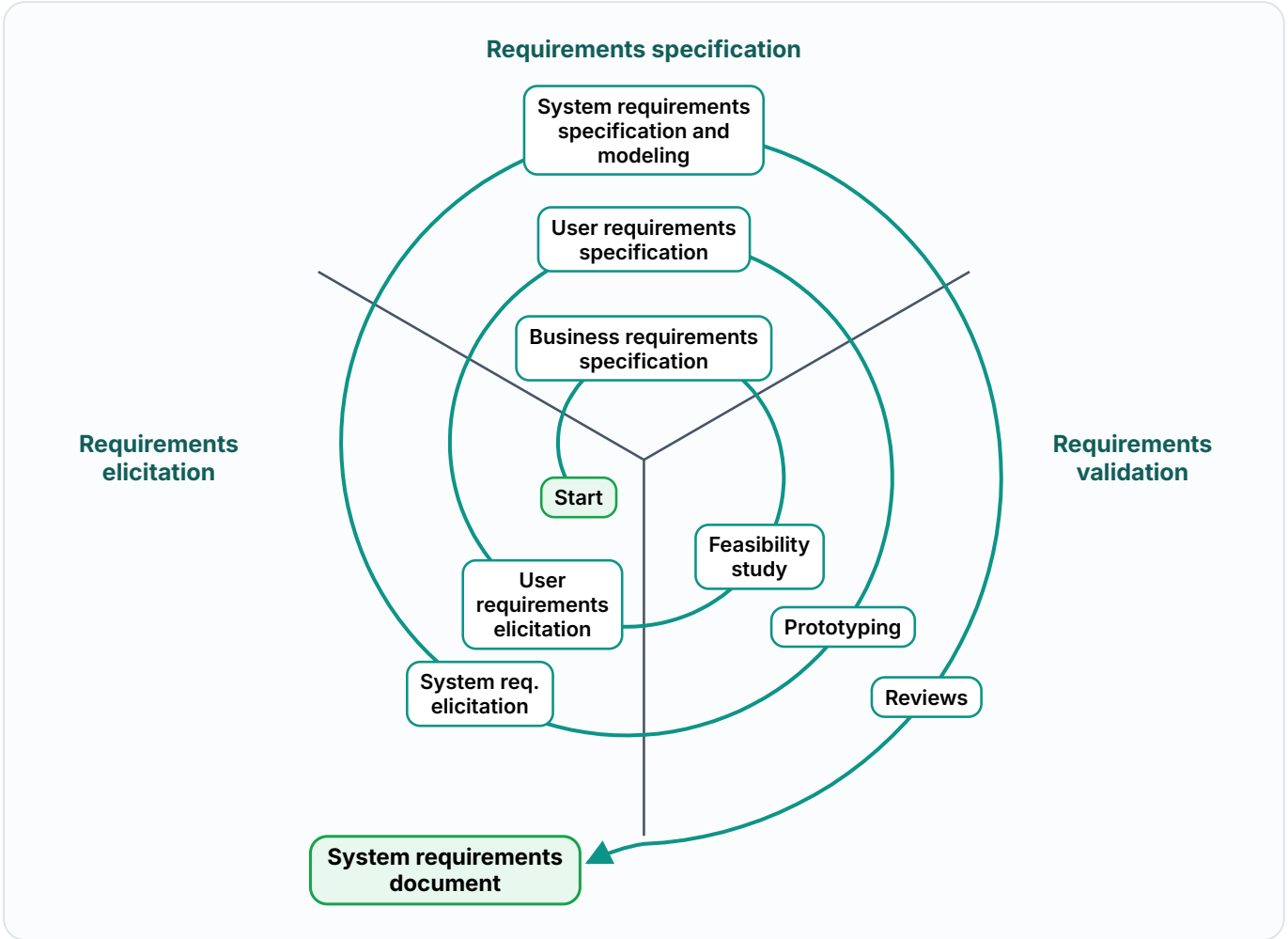


Fig 4.7 A spiral view of the requirements engineering process: each loop runs elicitation → specification → validation at a greater level of detail. Sommerville Fig 4.6 · Your slides, slide 29

- The three activities — **elicitation, specification and validation** — are organised as an iterative process around a **spiral**. Each loop adds detail: **business → user → system** requirements.
- The **output** of the RE process is a **system requirements document**.
- The time and effort spent on each activity in an iteration depends on the **stage** of the overall process, the **type of system** being developed and the **budget** available.
- **Early** in the process, most effort goes on understanding high-level **business and non-functional requirements** and the **user requirements**. **Later**, in the outer rings, more effort goes on eliciting and understanding the NFRs and more **detailed system requirements**.
- The number of iterations can vary: the spiral can be **exited** after some or all of the user requirements have been elicited. **Agile development** can be used instead of prototyping, so that the requirements and the implementation are developed together.
- In virtually all systems, **requirements change** (people understand their needs better, the buying organisation changes, the hardware, software and organisational environment changes). Changes must be managed — see Section 4.7.

> ✎ **EXAM TIP**
> To draw Fig 4.7 quickly: draw a "Y" (three lines from a centre); label the sectors **specification** (top), **elicitation** (left), **validation** (right); draw a spiral outwards from the centre; write the nine activities along it in order — *Start → business requirements spec → feasibility study → user requirements elicitation → user requirements spec → prototyping → system requirements elicitation → system requirements spec & modelling → reviews* — and end with an arrow to "System requirements document".

4.4 Requirements elicitation ★★★ MUST KNOW

> **IN SIMPLE WORDS**
> "Elicit" means to draw something out. Requirements rarely arrive written down — you draw them out of people by talking to them, watching them work and discussing realistic examples with them.

Requirements elicitation and analysis (sometimes called requirements elicitation or requirements discovery):

- Involves technical staff working with customers to find out about the **application domain**, the **services** that the system should provide and the system's **operational constraints**.
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called **stakeholders**.

The aim is to understand the **work that stakeholders do** and how they might use a new system to support it. Software engineers work with a range of system stakeholders to find out about the application domain, work activities, the services and features wanted, the **required system performance**, **hardware constraints**, **other systems**, etc. The stages are: requirements **discovery**; requirements **classification and organisation**; requirements **prioritisation and negotiation**; and requirements **specification**.

Problems of requirements elicitation ★★★ MUST KNOW

Problem (slide wording)	What it means (textbook detail)
1. Stakeholders don't know what they really want	They know only in the most general terms, find it hard to say what the system should do, and may make unrealistic demands because they don't know what is feasible.
2. Stakeholders express requirements in their own terms	They use their own language and implicit knowledge of their work; requirements engineers without domain experience may not understand them.
3. Different stakeholders may have conflicting requirements	Diverse stakeholders express requirements in different ways; engineers must find all sources and discover commonalities and conflicts.
4. Organisational and political factors may influence the requirements	E.g. managers may demand specific requirements because these will increase their influence in the organisation.
5. Requirements change during the analysis process	The economic and business environment is dynamic; the importance of requirements changes; new stakeholders may emerge who were not originally consulted.

The requirements elicitation and analysis process ★★★ MUST KNOW

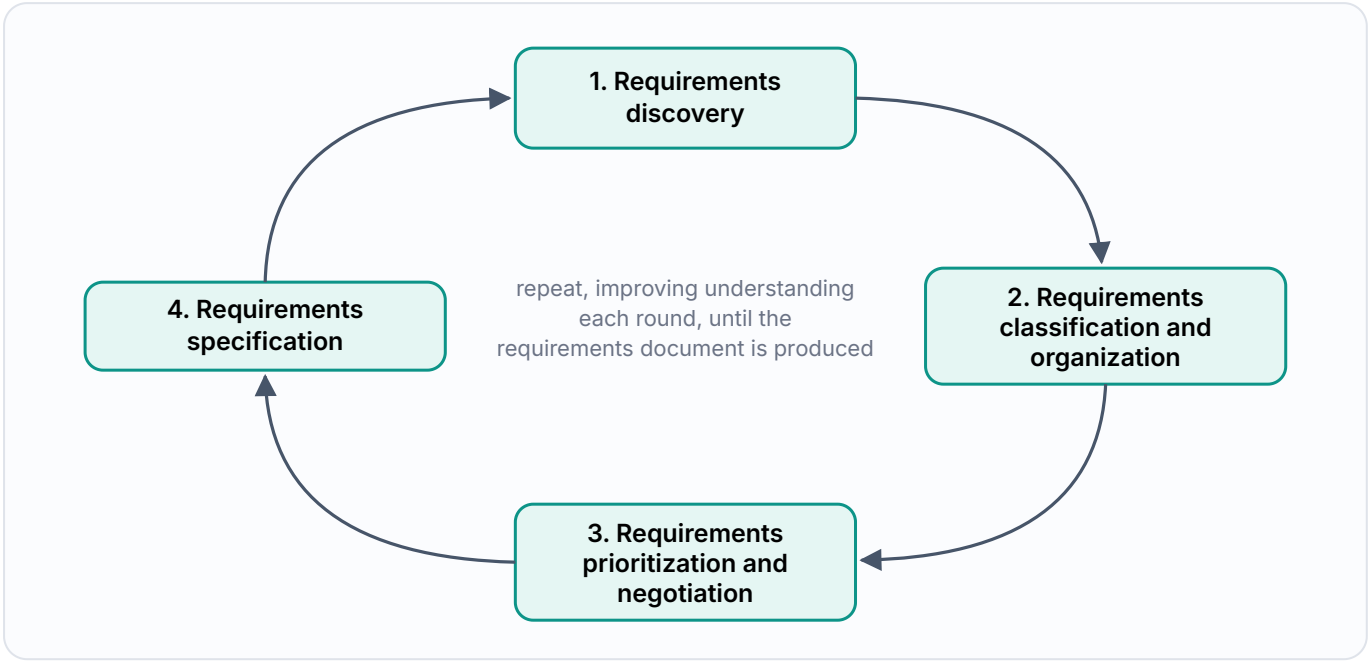


Fig 4.8 The requirements elicitation and analysis process (step 4 is called "requirements documentation" in the 10th-edition textbook). Sommerville Fig 4.7 · Your slides, slide 35

1. **Requirements discovery** — interacting with stakeholders to discover their requirements. Domain requirements (from stakeholders and documentation) are also discovered at this stage.
2. **Requirements classification and organisation** — takes the unstructured collection of requirements, groups related requirements and organises them into coherent clusters.
3. **Prioritisation and negotiation** — prioritising requirements and resolving requirements conflicts. With many stakeholders, requirements will conflict; stakeholders usually meet to agree compromise requirements.

4. **Requirements specification** — requirements are documented and input into the next round of the spiral. An early draft of the requirements document may be produced, or requirements kept informally on whiteboards, wikis or other shared spaces.

- The process is **iterative**, with continual feedback between activities. It starts with discovery and ends with documentation; the analyst's understanding improves each round; it ends when the requirements document is produced.
- Each organisation has its own version of this general model, depending on local factors: staff expertise, the type of system and the standards used.
- **Viewpoints** help organise stakeholder information: a viewpoint collects the requirements of one group of stakeholders who have something in common (e.g. end-users, managers). Viewpoints can also represent domain requirements and constraints from other systems. Alternatively, a model of the system architecture can be used to link requirements to subsystems.
- Hold **regular stakeholder meetings**: stakeholders who feel their views were ignored may deliberately try to undermine the RE process.
- When documenting, use **simple language and diagrams**, and a shared document or wiki that all stakeholders can access.

Requirements discovery ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Requirements discovery** — the process of gathering information about the required and existing systems and distilling the user and system requirements from this information.

- Interaction is with system stakeholders, from managers to external regulators.
- Systems normally have a range of stakeholders.
- Stakeholder information is supplemented by knowledge of existing systems and their usage, and by documents of various kinds.

There are two fundamental approaches: **interviewing**, where you talk to people about what they do, and **observation or ethnography**, where you watch people doing their job to see what artifacts they use and how they use them. Use a **mix** of both; stories and scenarios then help to discuss the requirements you derive.

Interviewing ★★★ MUST KNOW

Formal or informal interviews with stakeholders are part of most RE processes. The RE team asks stakeholders about the system they use now and the system to be developed, and derives requirements from the answers.

Closed interviews

Based on a **pre-determined list of questions**: the stakeholder answers a predefined set of questions.

Open interviews

No predefined agenda: various issues are explored with stakeholders to develop a better understanding of their needs.

Effective interviewing

- Be **open-minded**, avoid pre-conceived ideas about the requirements, and be willing to listen to stakeholders — if a stakeholder suggests something surprising, be willing to change your mind about the system.

- **Prompt** the interviewee to get discussions going using a **springboard question**, a **requirements proposal**, or by working together on a **prototype** system. Saying "tell me what you want" rarely gives useful information; people talk more easily in a defined context.

Interviews in practice

- Normally a **mix of closed and open-ended** interviewing: some questions must be answered, and these lead to less structured discussion. Completely open-ended discussions rarely work well.
- Interviews are good for getting an **overall understanding** of what stakeholders do, how they might interact with the system, and the difficulties they face with current systems.
- Interviewers need to be open-minded, without pre-conceived ideas of what the system should do.
- You need to prompt the user to talk about the system by **suggesting requirements** rather than simply asking them what they want. Unless you have a prototype to show, don't expect specific, detailed requirements — you analyse the information and generate the requirements yourself.

Problems with interviews

- Application specialists may use **language (jargon)** to describe their work that isn't easy for the requirements engineer to understand.
- Interviews are **not good for understanding domain requirements**:
  - requirements engineers cannot understand specific domain terminology;
  - some domain knowledge is so familiar that people find it hard to articulate, or think it isn't worth articulating. (For a librarian, it goes without saying that all acquisitions are catalogued before being added to the library — so the interviewer never hears it.)
- Interviews are also poor for **organisational requirements and constraints**: there are subtle power relationships, and people are reluctant to reveal the real (not the published) structure of decision-making to a stranger.
- On its own, interviewing is liable to miss essential information, so use it together with other techniques.

Ethnography ★★★ MUST KNOW

Software systems are used in a social and organisational environment. One reason why many systems are delivered but never used is that their requirements do not take proper account of how social and organisational factors affect the practical operation of the system.

> **DEFINITION — LEARN THIS LINE**
> **Ethnography** — an observational technique in which a social scientist spends a considerable time observing and analysing how people actually work, immersed in the working environment, to derive requirements for software that supports that work.

- People **do not have to explain or articulate** their work — which is useful, because their work is often "second nature" to them.
- **Social and organisational factors** of importance may be observed. (E.g. a workgroup may self-organise so that members can cover for each other — something never mentioned in an interview.)
- Ethnographic studies have shown that work is usually **richer and more complex than suggested by simple system models**. Suchman's pioneering study of office work found real practice far richer than office-automation models assumed — the main reason those systems had no significant effect on productivity.
- Its value is that it discovers **implicit** requirements that reflect how people actually work, not the formal processes defined by the organisation.

Scope of ethnography

Ethnography is particularly effective for discovering two types of requirements:

1. Requirements derived from the way that people **actually work**, rather than the way in which process definitions suggest that they ought to work. *Example:* air traffic controllers may switch off a conflict-alert system that sounds warnings even when planes are far apart, and use other strategies to keep planes on safe paths.
2. Requirements derived from **cooperation and awareness of other people's activities** — awareness of what other people are doing leads to changes in the ways in which we do things. *Example:* controllers watch adjacent sectors to predict how many aircraft will enter their own sector, so an automated ATC system should show some of the work in adjacent sectors.

Limitation: ethnography is effective for understanding **existing processes** but **cannot identify new features** that should be added to a system. Commentators suggest Nokia used ethnography to study how people used their phones, while Apple ignored current use and revolutionised the industry with the iPhone. Because it focuses on the end-user, ethnography is also weak for broader organisational or domain requirements — so it is one technique among several.

Focused ethnography

- Developed in a project studying the **air traffic control** process.
- **Combines ethnography with prototyping**.
- Prototype development results in **unanswered questions**, which focus the ethnographic analysis.
- The problem with ethnography is that it studies **existing practices**, which may have some historical basis that is no longer relevant.

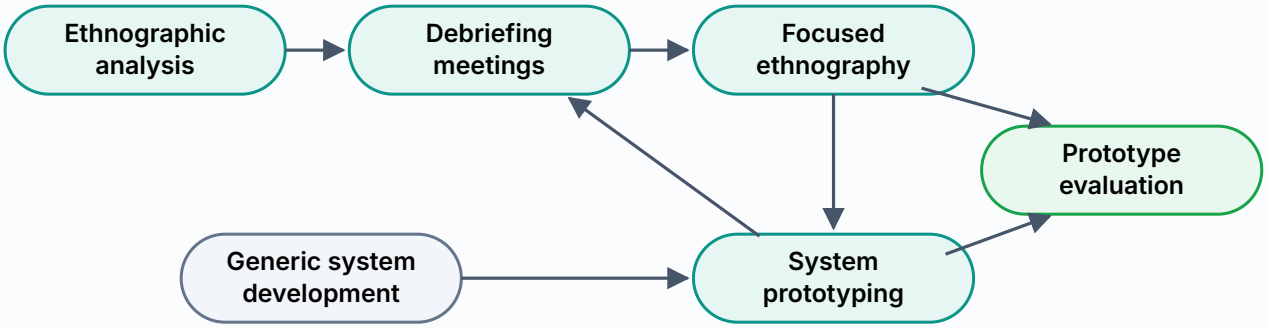


Fig 4.9 Ethnography and prototyping for requirements analysis. Sommerville Fig 4.8 · Your slides, slide 44

The ethnography **informs** the development of the prototype, so fewer prototype refinement cycles are needed. The prototyping **focuses** the ethnography by identifying problems and questions, which are discussed with the ethnographer in debriefing meetings; the ethnographer then looks for answers in the next phase of the study.

Basis	Interviewing	Ethnography
Method	Talk to people about what they do	Watch people doing their job
Who explains the work?	The stakeholder must put it into words	Nobody needs to; the observer records it

Basis	Interviewing	Ethnography
Good for	Overall understanding of tasks, how people would use the system, problems with current systems	Real working practice; cooperation and awareness; social and organisational factors
Weak for	Domain requirements (jargon, tacit knowledge); organisational and political issues	New features (innovation); broader organisational and domain requirements
Time needed	Relatively short sessions	Considerable time in the workplace
Typical risk	Interviewer's pre-conceived ideas; missing "obvious" knowledge	Copying existing practices whose historical basis is no longer relevant

Stories and scenarios ★★★ MUST KNOW

- Scenarios and user stories are **real-life examples** of how a system can be used.
- Stories and scenarios are a description of how a system may be used for a **particular task** — what people do, what information they use and produce, and what systems they may use.
- Because they are based on a practical situation, stakeholders can **relate to them** and can comment on their situation with respect to the story.

Story vs scenario: both describe system use for a task; they differ in structure and level of detail. A **story** is narrative text giving a high-level, "big picture" description. A **scenario** is a structured form of user story with specific information such as inputs and outputs. Parts of a story are developed into scenarios.

> **STORY — PHOTO SHARING IN THE CLASSROOM (ILEARN)**
>
> Jack is a primary school teacher in Ullapool (a village in northern Scotland). He has decided that a class project should be focused around the fishing industry in the area, looking at the history, development and economic impact of fishing. As part of this, pupils are asked to gather and share reminiscences from relatives, use newspaper archives and collect old photographs related to fishing and fishing communities in the area. Pupils use an iLearn wiki to gather together fishing stories and SCRAN (a history resources site) to access newspaper archives and photographs. However, Jack also needs a photo sharing site as he wants pupils to take and comment on each other's photos and to upload scans of old photographs that they may have in their families.
>
> Jack sends an email to a primary school teachers group, which he is a member of, to see if anyone can recommend an appropriate system. Two teachers reply and both suggest that he uses KidsTakePics, a photo sharing site that allows teachers to check and moderate content. As KidsTakePics is not integrated with the iLearn authentication service, he sets up a teacher and a class account. He uses the iLearn setup service to add KidsTakePics to the services seen by the pupils in his class so that when they log in, they can immediately use the system to upload photos from their mobile devices and class computers.

The advantage of stories is that **everyone can relate to them**. The iLearn team put such stories on a wiki and invited teachers and students across the country to comment — reaching a wider community than they could ever interview. User stories in agile methods such as Extreme Programming are really narrative scenarios.

What a scenario should include

A **scenario** is a structured form of user story. It starts with an outline of the interaction, and details are added during elicitation. Scenarios should include:

1. A description of the **starting situation** (what the system and users expect when the scenario starts);
2. a description of the **normal flow of events**;
3. a description of **what can go wrong** (and how the resulting problems are handled);
4. information about **other concurrent activities**;
5. a description of the **state when the scenario finishes**.

> **MEMORY TRICK**
> **S-N-W-O-E** — "*Start, Normal, Wrong, Others, End*". Think of a short film: opening scene, normal plot, what goes wrong, side plots, final scene.

Scenario part	Uploading photos to KidsTakePics (iLearn)
Initial assumption	A user or a group of users have one or more digital photographs to be uploaded to the picture sharing site. These are saved on either a tablet or laptop computer. They have successfully logged on to KidsTakePics.
Normal	The user chooses upload photos and is prompted to select the photos to be uploaded on their computer and to select the project name under which the photos will be stored. They should also be given the option of inputting keywords that should be associated with each uploaded photo. Uploaded photos are named by creating a conjunction of the user name with the filename of the photo on the local computer. On completion of the upload, the system automatically sends an email to the project moderator asking them to check new content, and generates an on-screen message to the user that this has been done.
What can go wrong	No moderator is associated with the selected project. An email is automatically generated to the school administrator asking them to nominate a project moderator. Users should be informed that there could be a delay in making their photos visible. Photos with the same name have already been uploaded by the same user. The user should be asked if they wish to re-upload the photos with the same name, rename the photos or cancel the upload. If they choose to re-upload the photos, the originals are overwritten. If they choose to rename the photos, a new name is automatically generated by adding a number to the existing file name.
Other activities	The moderator may be logged on to the system and may approve photos as they are uploaded.
System state on completion	User is logged on. The selected photos have been uploaded and assigned a status "awaiting moderation". Photos are visible to the moderator and to the user who uploaded them.

The key feature that makes KidsTakePics different from other photo sites is that a **teacher moderates** uploaded photos before they are shared. The scenario is much more detailed than the story, so it can be used to propose specific requirements.

Basis	Story	Scenario	Use case (Section 4.5)
Form	Narrative text	Structured text under fixed headings	UML diagram + textual description
Detail	High level, "big picture"	Detailed: inputs, outputs, exceptions	Names actors and each type of interaction
Best use	Start discussion with many stakeholders	Propose specific requirements	Describe all interactions; also used in design

Basis	Story	Scenario	Use case (Section 4.5)
Example	Jack's photo-sharing story	Uploading photos to KidsTakePics	Mentcare use cases (Fig 4.10)

4.5 Requirements specification ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Requirements specification — the process of writing down the user and system requirements in a requirements document.

- **User requirements** have to be understandable by end-users and customers who do not have a technical background. Write them in natural language with simple tables, forms and intuitive diagrams; avoid software jargon and structured or formal notations; describe only external behaviour, not architecture or design.
- **System requirements** are more detailed requirements and may include more technical information. They are the starting point for system design.
- The requirements may be part of a **contract** for the system development, so it is important that they are as **complete as possible**.
- Ideally requirements are clear, unambiguous, easy to understand, complete and consistent. In practice this is almost impossible, because stakeholders interpret requirements differently and there are inherent conflicts.

Ways of writing a system requirements specification ★★★ MUST KNOW

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features, to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract.

MEMORY TRICK

N-S-D-G-M — "*Nice Students Draw Good Maps*": Natural language, Structured natural language, Design description languages, Graphical notations, Mathematical specifications.

Requirements and design ★★ IMPORTANT

In principle, requirements should state **what** the system should do, and the design should describe **how** it does this. In practice, requirements and design are **inseparable**:

- A **system architecture may be designed to structure the requirements** — the requirements are organised by subsystem (as was done for iLearn).
- The system may **inter-operate with other systems** that generate design requirements (existing systems constrain the design).
- The use of a **specific architecture to satisfy non-functional requirements** may be a domain requirement (e.g. N-version programming to achieve reliability).
- This may be the consequence of a **regulatory requirement** — a regulator who must certify that the system is safe may insist on an architecture that has already been certified.

Natural language specification ★★★ MUST KNOW

- Requirements are written as natural language sentences supplemented by diagrams and tables.
- Natural language is used because it is **expressive, intuitive and universal**, so the requirements can be understood by users and customers. It has been used since the 1950s and is still the most widely used way of specifying requirements.
- But it is also potentially **vague and ambiguous**, and its meaning depends on the background of the reader.

Guidelines for writing requirements

1. **Invent a standard format** and use it for all requirements. Standardising makes omissions less likely and requirements easier to check (ideally one or two sentences per requirement).
2. **Use language in a consistent way.** Use **shall** for mandatory requirements and **should** for desirable requirements.
3. **Use text highlighting** (bold, italic or colour) to identify key parts of the requirement.
4. **Avoid the use of computer jargon**, abbreviations and acronyms — words like "architecture" and "module" are easily misunderstood.
5. **Include an explanation (rationale)** of why a requirement is necessary, and who proposed it (its source), so you know whom to consult if it has to change.

Problems with natural language

Problem	Meaning
Lack of clarity	Precision is difficult without making the document difficult to read.
Requirements confusion	Functional and non-functional requirements tend to be mixed up.
Requirements amalgamation	Several different requirements may be expressed together (in one sentence).

EXAMPLE — INSULIN PUMP REQUIREMENTS IN NATURAL LANGUAGE

3.2 The system **shall** measure the blood sugar and deliver insulin, if required, every 10 minutes. *(Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)*

3.6 The system **shall** run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. *(A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)*

Guidelines in action: a numbered standard format, "shall" for mandatory, the key word highlighted, and the rationale in brackets.

Structured specifications ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Structured specification — an approach to writing requirements where the freedom of the requirements writer is limited and requirements are written in a standard way (using templates/forms).

- Keeps most of the expressiveness and understandability of natural language, but imposes uniformity.
- This works well for some types of requirements, e.g. requirements for an **embedded control system**, but is sometimes **too rigid** for writing **business system** requirements.
- Templates may use programming-language constructs to show alternatives and iteration, and highlight key elements with shading or different fonts. The specification can be organised around the objects, functions or events of the system.
- (In the VOLERE method, each user requirement is first written on a card with fields such as rationale, dependencies on other requirements, source and supporting materials.)

Form-based specifications — the information on the form

1. Definition of the **function or entity**.
2. Description of **inputs** and where they come from.
3. Description of **outputs** and where they go to.
4. Information about the information needed for the computation and other entities used (the **"requires"** part).
5. Description of the **action** to be taken.
6. **Pre and post conditions** (if appropriate) — what must be true before the function is called, and what is true after it.
7. The **side effects** (if any) of the function.

Insulin Pump / Control Software / SRS / 3.3.2	
Function	Compute insulin dose: safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2); the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose — the dose in insulin to be delivered.

Insulin Pump / Control Software / SRS / 3.3.2	
Destination	Main control loop.
Action	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result is rounded to zero then CompDose is set to the minimum dose that can be delivered.
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Pre-condition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Post-condition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

A structured specification of a requirement for an insulin pump (Sommerville Fig 4.13; your slides, slides 60–61).

Structured specifications **reduce variability** and organise requirements more effectively. But it is still hard to write complex computations (such as the insulin dose) clearly — so add **tables** or graphical models.

Tabular specification ★★★ MUST KNOW

- Used to **supplement natural language**.
- Particularly useful when you have to define a number of possible **alternative courses of action**.
- For example, the insulin pump system bases its computations on the **rate of change of blood sugar level**, and the tabular specification explains how to calculate the insulin requirement for different scenarios. Here r0, r1 and r2 are the last three sugar readings, r2 being the current one.

Condition	Action
Sugar level falling (r2 < r1)	CompDose = 0
Sugar level stable (r2 = r1)	CompDose = 0
Sugar level increasing and rate of increase decreasing ((r2 − r1) < (r1 − r0))	CompDose = 0
Sugar level increasing and rate of increase stable or increasing ((r2 − r1) ≥ (r1 − r0))	CompDose = round ((r2 − r1)/4) If rounded result = 0 then CompDose = MinimumDose

? QUICK CHECK

Readings r0 = 4, r1 = 5, r2 = 6. What is CompDose?

Answer: The level is rising and the rate of increase is stable ((6 − 5) = 1 ≥ (5 − 4) = 1), so CompDose = round(1/4) = round(0.25) = 0; because the rounded result is 0, CompDose = MinimumDose.

Use cases ★★★ MUST KNOW

- Use cases are a kind of **scenario** that are included in the **UML** (they were first introduced in the Objectory method).
- Use cases identify the **actors** in an interaction and describe the **interaction** itself.
- A set of use cases should describe **all possible interactions** with the system.
- A **high-level graphical model** is supplemented by a more detailed **tabular description** (Chapter 5).

- UML **sequence diagrams** may be used to add detail to use cases by showing the sequence of event processing in the system.

Notation: actors — people or other systems — are drawn as **stick figures**; each class of interaction is a **named ellipse**; **lines** link actors with the interactions they take part in. Arrowheads may be added to show how an interaction is initiated.

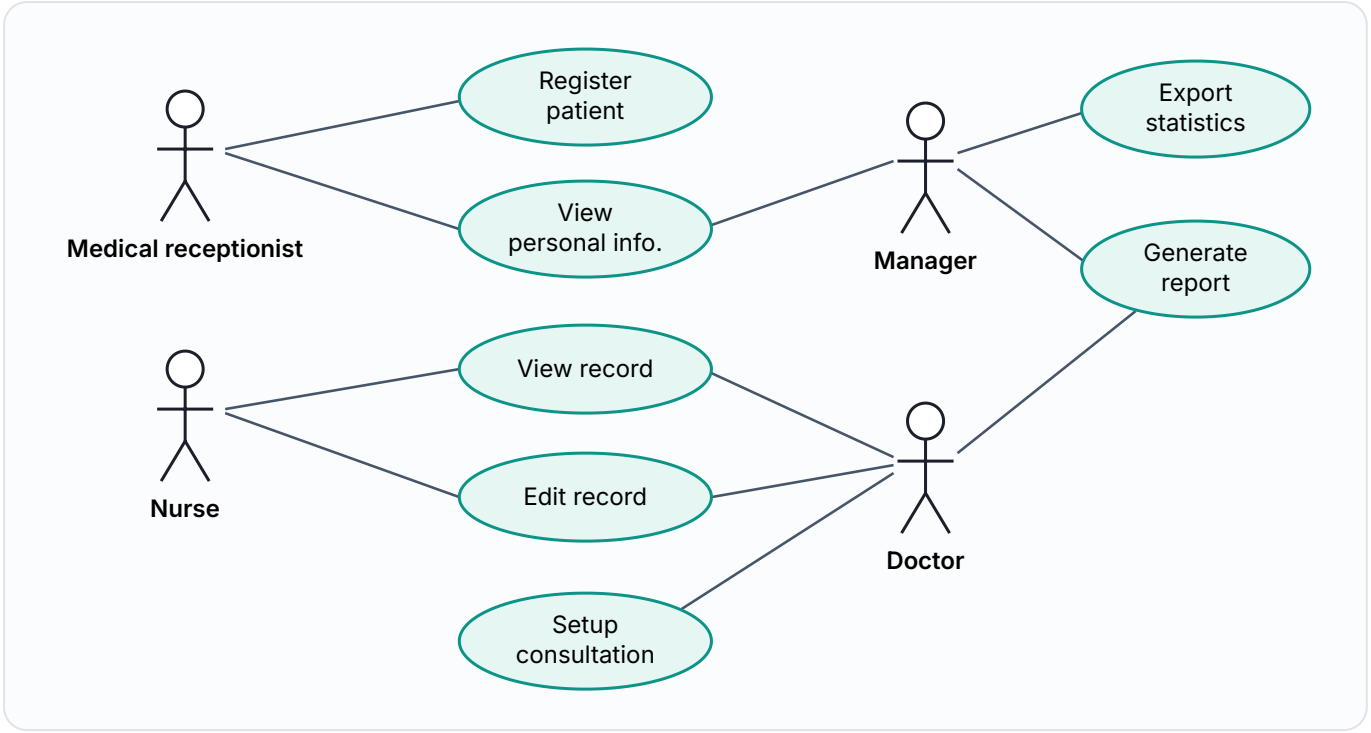


Fig 4.10 Use cases for the Mentcare system: four actors and seven use cases. Sommerville Fig 4.15 · Your slides, slide 65

EXAMPLE — TEXTUAL DESCRIPTION OF THE SETUP CONSULTATION USE CASE

Setup consultation allows two or more doctors, working in different offices, to view the same patient record at the same time. One doctor initiates the consultation by choosing the people involved from a dropdown menu of doctors who are online. The patient record is then displayed on their screens, but only the initiating doctor can edit the record. In addition, a text chat window is created to help coordinate actions. It is assumed that a phone call for voice communication can be separately arranged.

Sommerville's view: stakeholders often don't understand the term "use case", don't find the diagram useful and are not interested in every detailed interaction — so use cases are often **more helpful in system design** than in RE. Some people treat each use case as a single low-level scenario; others (Stevens and Pooley) treat a use case as a set of related scenarios — one for the normal interaction plus one for each possible exception.

The software requirements document ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software requirements document (also called the software requirements specification, **SRS**) — the official statement of what is required of the system developers. It should include both a definition of user requirements and a specification of the system requirements.

- It is an **agreed statement** of the system requirements, organised so that both system customers and software developers can use it.
- It is **NOT a design document**. As far as possible, it should set out **WHAT** the system should do rather than **HOW** it should do it.
- It is essential when systems are **outsourced**, when **different teams** develop different parts of the system, and when a **detailed analysis** of the requirements is mandatory. It may not be needed for software products or business systems.
- Agile methods argue that it is out of date as soon as it is written; they collect user requirements incrementally as short user stories.

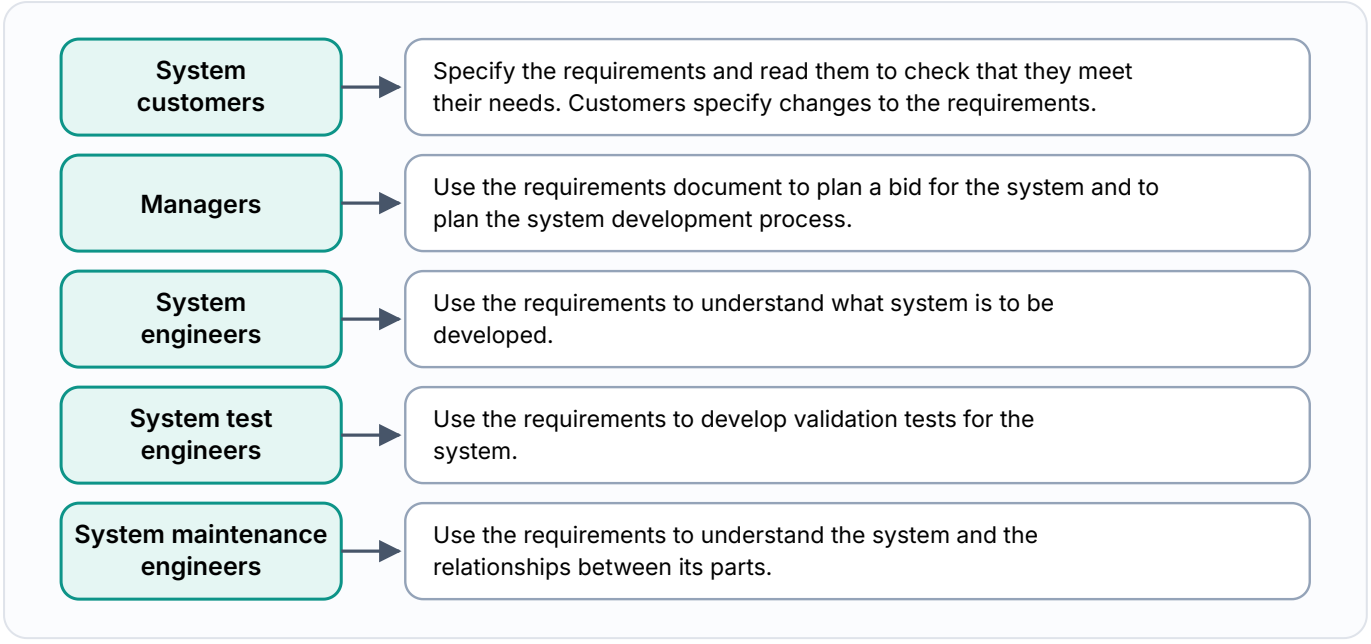


Fig 4.11 Users of a requirements document and how they use it. Sommerville Fig 4.16 · Your slides, slide 67

Because its users are so diverse, the document is a **compromise**: it must describe requirements for customers, define them in precise detail for developers and testers, and include information about likely future evolution (which helps designers avoid restrictive decisions and helps maintenance engineers adapt the system).

Requirements document variability

- Information in the requirements document depends on the **type of system** and the **approach to development** used.
- Systems developed **incrementally** will, typically, have **less detail** in the requirements document (with in-house iterative development, details are added during development).
- **Critical systems** need detailed requirements, because safety and security must be analysed; **outsourced** systems need detailed and precise specifications.
- Requirements document **standards** have been designed, e.g. the **IEEE standard**. These are mostly applicable to the requirements for large systems engineering projects — such as military command-and-control systems with a long lifetime, developed by a group of organisations.

The structure of a requirements document

One possible organisation, based on the IEEE standard and extended with information about predicted system evolution:

Chapter	Description
Preface	Defines the expected readership of the document and describes its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	Describes the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organisation commissioning the software.
Glossary	Defines the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Describes the services provided for the user. The non-functional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	Presents a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.
System requirements specification	Describes the functional and non-functional requirements in more detail. If necessary, further detail may also be added to the non-functional requirements. Interfaces to other systems may be defined.
System models	Graphical system models showing the relationships between the system components and between the system and its environment. Examples: object models, data-flow models, semantic data models.
System evolution	Describes the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. Useful for system designers, as it may help them avoid design decisions that would constrain likely future changes.
Appendices	Detailed, specific information related to the application being developed — for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organisation of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

MEMORY TRICK

"PIG-U, four S's, A-I": **P**reface, **I**ntroduction, **G**lossary, **U**ser requirements definition → **S**ystem architecture, **S**ystem requirements specification, **S**ystem models, **S**ystem evolution → **A**ppendices, **I**ndex.

A long document for a complex system built by several companies needs a full table of contents and index. A document for an in-house software product leaves out many of these chapters and focuses on the user requirements and high-level non-functional requirements.

4.6 Requirements validation ★★★ MUST KNOW

IN SIMPLE WORDS

Validation asks: "Have we written down the *right* system?" — before anyone starts building it. Catching a wrong requirement on paper is cheap; catching it after delivery is very expensive.

DEFINITION — LEARN THIS LINE

Requirements validation — the process of demonstrating that the requirements define the system that the customer really wants.

- It overlaps with elicitation and analysis, as it is concerned with finding problems with the requirements.
- Requirements error costs are high, so validation is very important: fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.
- Why so expensive? A requirements change usually means the system design and implementation must also change, and the system must then be retested.

Requirements checking ★★★ MUST KNOW

Check	Question to ask	Textbook detail
Validity	Does the system provide the functions which best support the customer's needs?	Requirements reflect the real needs of users — which may have changed since they were elicited.
Consistency	Are there any requirements conflicts?	No contradictory constraints and no different descriptions of the same function.
Completeness	Are all functions required by the customer included?	The document defines all functions and constraints intended by the system user.
Realism	Can the requirements be implemented given available budget and technology?	Use knowledge of existing technology; also consider the budget and schedule.
Verifiability	Can the requirements be checked?	Requirements are written so that tests can show the delivered system meets each one — this reduces disputes between customer and contractor.

MEMORY TRICK

V-C-C-R-V — "*Very Careful Clients Request Verification*": Validity, Consistency, Completeness, Realism, Verifiability.

Requirements validation techniques ★★★ MUST KNOW

1. **Requirements reviews** — systematic manual analysis of the requirements by a team of reviewers, who check for errors and inconsistencies.
2. **Prototyping** — using an executable model of the system to check requirements (Chapter 2). End-users and customers experiment with it and feed back requirements changes.
3. **Test-case generation** — developing tests for requirements to check testability. If a test is difficult or impossible to design, the requirement will usually be hard to implement and should be reconsidered. Writing tests from the user requirements before any code is an integral part of test-driven

development.

Requirements reviews ★★★ MUST KNOW

- Regular reviews should be held **while the requirements definition is being formulated**.
- **Both client and contractor staff** should be involved in reviews.
- Reviews may be **formal** (with completed documents) or **informal**. Good communication between developers, customers and users can resolve problems at an early stage.
- In a review, people from the customer and the developer read the document in detail and check for errors, anomalies and inconsistencies; once these are recorded, customer and developer negotiate how to solve them.

Review checks

Review check	Question
Verifiability	Is the requirement realistically testable?
Comprehensibility	Is the requirement properly understood?
Traceability	Is the origin of the requirement clearly stated?
Adaptability	Can the requirement be changed without a large impact on other requirements?

⚠ WATCH OUT

Don't mix the two lists. **Requirements checks** (5): validity, consistency, completeness, realism, verifiability. **Review checks** (4): verifiability, comprehensibility, traceability, adaptability. Only **verifiability** appears in both.

Be realistic about validation. It is hard to show that requirements meet a user's needs: users must picture the system in operation and imagine how it fits their work — hard even for skilled professionals. You rarely find all problems during validation, so further changes to correct omissions and misunderstandings are inevitable after the document has been agreed.

4.7 Requirements change ★★★ MUST KNOW

The requirements for large software systems are **always changing**. One reason is that these systems often address **"wicked" problems** — problems that cannot be completely defined — so the requirements are bound to be incomplete. During development, the stakeholders' understanding of the problem keeps changing, and the requirements must evolve to reflect it (Fig 4.12).

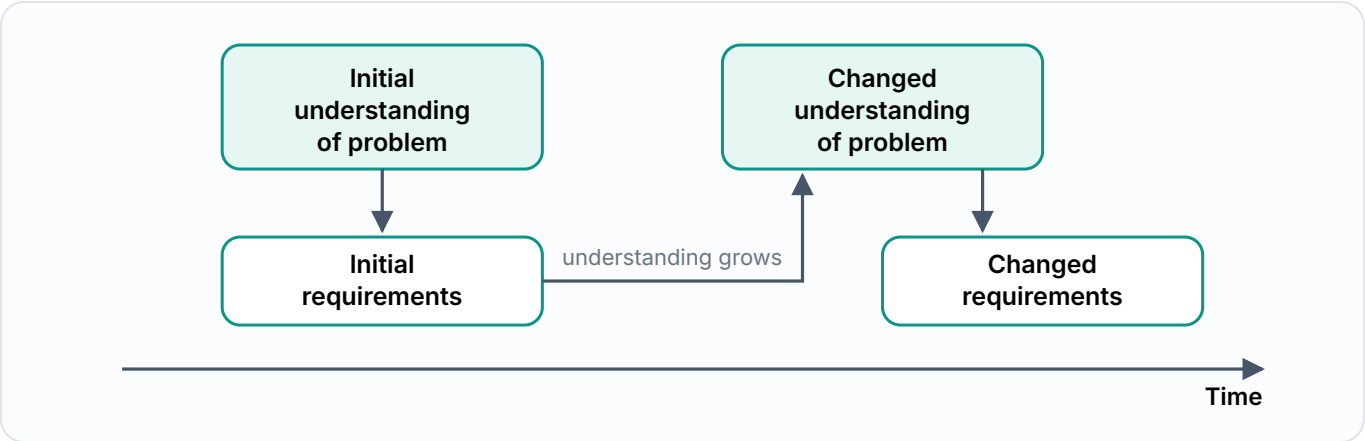


Fig 4.12 Requirements evolution: as understanding of the problem changes, the requirements change. Sommerville Fig 4.18 · Your slides, slide 80

Why requirements change ★★★ MUST KNOW

Once a system is installed and regularly used, new requirements inevitably emerge. Some come from errors and omissions in the original requirements, but most come from changes to the system's environment:

1. **The business and technical environment of the system always changes after installation.** New hardware may be introduced, it may be necessary to interface the system with other systems, business priorities may change (with consequent changes in the system support required), and new legislation and regulations may be introduced that the system must necessarily abide by.
2. **The people who pay for a system and the users of that system are rarely the same people.** System customers impose requirements because of organisational and budgetary constraints. These may conflict with end-user requirements and, after delivery, new features may have to be added for user support if the system is to meet its goals.
3. **Large systems usually have a diverse user community**, with many users having different requirements and priorities that may be conflicting or contradictory. The final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.

Enduring and volatile requirements ★★ IMPORTANT

Basis	Enduring requirements	Volatile requirements
Meaning	Associated with the core, slow-to-change activities of an organisation — its fundamental work activities	More likely to change; associated with supporting activities that reflect *how* the organisation does its work rather than the work itself
Illustration (Mentcare)	Recording patients and the treatments they receive — a clinic always needs this	The layout of a management report, or the way staff log in (e.g. moving to ID cards)

Requirements management ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
>
> **Requirements management** — the process of managing changing requirements during the requirements engineering process and system development.

- New requirements emerge as a system is being developed and after it has gone into use.

- You need to **keep track of individual requirements** and maintain **links between dependent requirements**, so that you can assess the impact of requirements changes.
- You need to establish a **formal process for making change proposals** and linking these to system requirements. Requirements management should start as soon as a draft of the requirements document is available.
- **Agile** processes handle change without a formal process: the user prioritises the change and, if it is high priority, decides which planned features to drop. The problem: users are not necessarily the best judges of whether a change is cost-effective. With many stakeholders, an **independent authority** that balances everyone's needs is often better.

Requirements management planning ★★★ MUST KNOW

Planning establishes the **level of requirements management detail** that is required. The requirements management decisions are:

Decision	What it means
Requirements identification	Each requirement must be uniquely identified so that it can be cross-referenced with other requirements (and used in traceability assessments).
A change management process	The set of activities that assess the impact and cost of changes (Fig 4.13).
Traceability policies	Define the relationships between each requirement, and between the requirements and the system design, that should be recorded — and how these records are maintained.
Tool support	Tools may range from specialist requirements management systems to spreadsheets and simple database systems.

Tool support is needed for **requirements storage** (a secure, managed data store accessible to everyone involved), **change management** (tracking suggested changes and the responses to them) and **traceability management** (discovering related requirements). Small systems can manage with shared web documents, spreadsheets and databases; large systems use specialised tools such as **DOORS**.

> **TRACEABILITY IN SIMPLE WORDS**
>
> Traceability is the "family tree" of each requirement: where it came from (its source), which other requirements depend on it, and which parts of the design implement it. When one requirement changes, you can follow the links to see how the change ripples through the system.

Requirements change management ★★★ MUST KNOW

Change management is applied to all proposed changes after the requirements document has been approved. It decides **whether a requirements change should be accepted** — whether the benefits of the change justify its cost. A formal process means all change proposals are treated consistently and the document is changed in a controlled way.

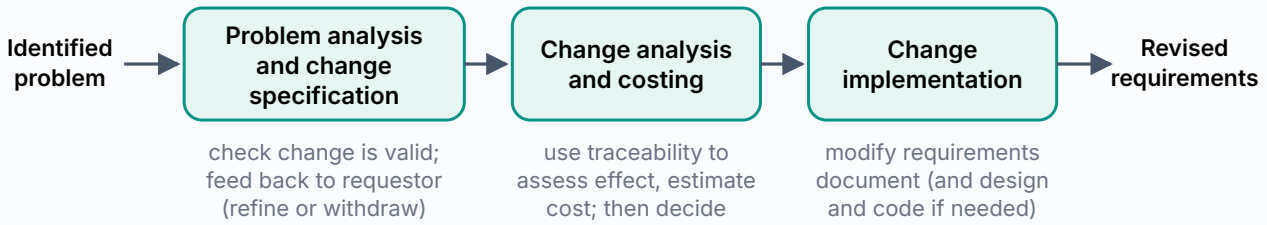


Fig 4.13 Requirements change management. Sommerville Fig 4.19 · Your slides, slides 83–84

1. **Problem analysis and change specification** — the process starts with an identified requirements problem or a specific change proposal. During this stage, the problem or the change proposal is analysed to check that it is valid. This analysis is fed back to the change requestor, who may respond with a more specific requirements change proposal, or decide to withdraw the request.
2. **Change analysis and costing** — the effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. The cost is estimated in terms of changes to the requirements document and, if appropriate, to the design and implementation. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.
3. **Change implementation** — the requirements document and, where necessary, the system design and implementation are modified. Ideally, the document should be organised so that changes can be easily implemented (modular sections, few external references).

> ⚠ **WATCH OUT**
>
> Emergency changes: there is always a temptation to change the system first and update the requirements document later. The document and the implementation then get out of step, because changes are easily forgotten. If an emergency change is unavoidable, update the requirements document as soon as possible.

4.8 Lab: analysing a real-time case study ★★ IMPORTANT

The lab asks you to analyse the requirements of a **real-time** application — software that must react to its inputs within fixed time limits. The textbook's **insulin pump** is a good model: an embedded system that collects information from a sensor and controls a pump that delivers a controlled dose of insulin. It is **safety-critical**: if it fails, the user's health may be damaged. Use the same steps for any case study (weather station, traffic signal, ATM, ...).

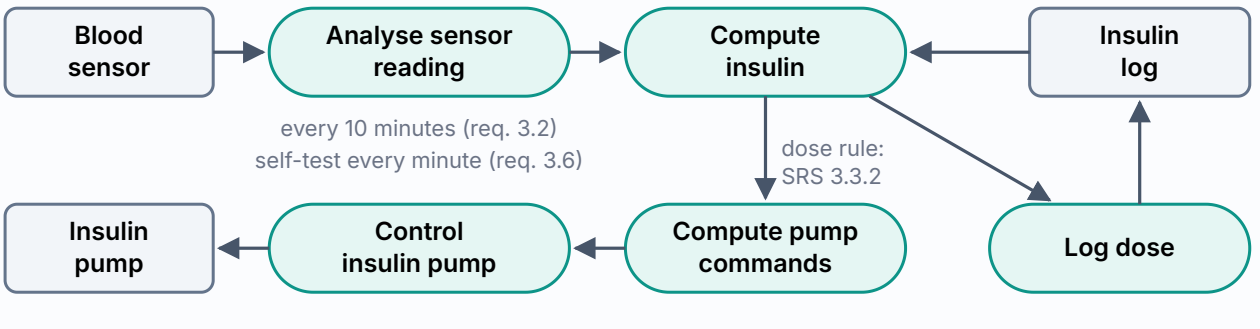


Fig 4.14 What the insulin pump software does, with the textbook requirements that govern each step. Simplified from Sommerville Fig 1.5 (activity model of the insulin pump)

1. **Understand the system and its environment** — read the case; sketch the hardware and the flow of data (Fig 4.14). Note the timing constraints.
2. **Identify the stakeholders** and the type of each (end users, managers, owners, external).
3. **Choose elicitation techniques** — interviews, observation, stories/scenarios — and say why.
4. **Write user requirements** — high-level, in natural language.
5. **Write functional system requirements** — numbered, "shall", with a rationale.
6. **Write non-functional requirements** — classify each (product / organisational / external) and make it measurable (metrics table).
7. **Specify one critical function** as a structured form or table; add a scenario or use-case diagram.
8. **Validate and plan for change** — apply the five checks; give every requirement an ID; mark enduring vs volatile ones.

Step	Insulin pump — model answer
System	Embedded, real-time, safety-critical control system: reads a blood-sugar sensor, computes the insulin needed and sends signals to a miniaturised pump (one pulse = one unit of insulin). Hardware: sensor, controller, pump, insulin reservoir, needle assembly, clock, alarm, two displays, power supply.
Stakeholders ***(illustrative)***	Patients with diabetes (end users); doctors who set up treatment; the manufacturer's engineers who maintain the device; medical-device regulators (external).
Elicitation ***(illustrative)***	Interviews with doctors and patients; observing how patients currently measure blood sugar and inject; scenarios such as "sugar level rising quickly after a meal".
User requirements (textbook)	(1) The system shall be available to deliver insulin when required. (2) The system shall perform reliably and deliver the correct amount of insulin to counteract the current level of blood sugar.
Functional requirements (textbook)	3.2 measure blood sugar and deliver insulin, if required, every 10 minutes · 3.6 run a self-test routine every minute · 3.3.2 compute insulin dose (structured form + table in Section 4.5).
Non-functional ***(illustrative wording)***	**Product (dependability):** user requirement (1) above is an availability requirement · **Product (performance):** each dose computation shall complete within the 10-minute measurement cycle · **Organisational (development):** the software shall be developed to the manufacturer's safety-critical process standard · **External (regulatory):** the pump shall be approved by the national medical-device regulator before use.
Validation	**Verifiability:** "every 10 minutes", "every minute" and the dose table are testable; "deliver the correct amount" becomes testable only through the table. **Realism:** can the controller hardware meet the timing? **Consistency:** the pre-condition (reservoir holds at least one maximum dose) must agree with the self-test and alarm requirements.
Change ***(illustrative)***	Enduring: the dose computation and safety requirements. Volatile: display layout, logging details. Give IDs (3.2, 3.3.2, 3.6…) so changes can be traced.

✎ EXAM TIP

Lab record layout: 1 Problem statement · 2 Stakeholders · 3 Elicitation method · 4 User requirements · 5 Functional requirements (numbered) · 6 NFRs (classified, with metrics) · 7 One structured or tabular specification · 8 Use-case diagram or scenario · 9 Validation checklist (V-C-C-R-V).

Chapter 4 in one page

Key definitions

- **RE**: establishing the services a customer requires and the constraints under which the system operates and is developed.
- **User reqs**: natural language + diagrams, for customers. **System reqs**: detailed, structured; may be part of the contract.
- **Stakeholder**: anyone affected by the system with a legitimate interest — end users, system managers, system owners, external.
- **FR**: services, reactions to inputs, behaviour in situations (and what not to do), or how computations are carried out. **NFR**: constraints on services — timing, process, standards; whole system. **Domain req**: constraint from the domain.
- **Goal** vs **verifiable NFR** (has a measure that can be objectively tested).
- **Validation**: show the requirements define the system the customer really wants.
- **Requirements management**: managing changing requirements during RE and development.

Lists to write in full

- **NFR tree**: Product (Usability, Efficiency [Performance, Space], Dependability, Security) · Organisational (Environmental, Operational, Development) · External (Regulatory, Ethical, Legislative [Accounting, Safety/security]).
- **Metrics**: speed, size, ease of use, reliability, robustness, portability.
- **Elicitation problems**: don't know what they want · own terms · conflicts · politics · change.
- **Scenario parts**: start, normal flow, what goes wrong, other activities, end state.
- **NL guidelines**: standard format · shall/should · highlighting · no jargon · rationale. **NL problems**: lack of clarity, confusion, amalgamation.

Processes

- **RE activities**: elicitation & analysis → specification → validation, plus management; iterative (spiral: business → user → system requirements).
- **Elicitation cycle**: discovery → classification & organisation → prioritisation & negotiation → specification.
- **Techniques**: interviews (closed/open) · ethnography (focused = + prototyping) · stories · scenarios · use cases.
- **Notations**: natural language, structured NL, design description languages, graphical, mathematical.
- **Form fields**: function, inputs+source, outputs+destination, requires, action, pre/post-conditions, side effects.
- **SRS chapters** (PIG-U-SSSS-AI): preface, introduction, glossary, user reqs, system architecture, system reqs spec, system models, system evolution, appendices, index.
- **Validation checks**: validity, consistency, completeness, realism, verifiability. **Techniques**: reviews, prototyping, test-case generation. **Review checks**: verifiability, comprehensibility, traceability, adaptability.
- **Why change**: environment changes · payers ≠ users · diverse users. **RM planning**: identification, change process, traceability policies, tool support. **Change mgmt**: problem analysis & change spec → change analysis & costing → change implementation.

Diagrams to practise

- Readers of requirements (4.2) · NFR tree (4.5) · spiral (4.7) · elicitation cycle (4.8) · ethnography + prototyping (4.9) · Mentcare use cases (4.10) · users of SRS (4.11) · evolution (4.12) · change management (4.13).

Likely exam questions

2M **Define requirements engineering. What is a stakeholder? Give two Mentcare stakeholders.**

How to answer → Slide definition of RE (services + constraints); stakeholder definition; e.g. doctors, medical receptionists (Section 4.1).

2M **Differentiate a goal and a verifiable non-functional requirement with an example.**

How to answer → Two definitions + the usability goal and its testable version (training hours, errors per hour).

4M **Distinguish between user requirements and system requirements. Who reads each?**

How to answer → Definitions → table → Mentcare drug-cost example (Fig 4.1) → readers diagram (Fig 4.2).

10M **Differentiate functional and non-functional requirements with suitable examples. Why can NFRs be more critical than FRs?**

How to answer → Definitions (also domain requirements) → comparison table (6+ rows) → Mentcare FR and NFR examples → aircraft/embedded examples of criticality → NFR implementation (architecture; generates FRs).

10M **With a neat diagram, explain the classification of non-functional requirements. Give an example of each main type.**

How to answer → Draw Fig 4.5 → define product, organisational, external with sub-types → Mentcare examples (availability, ID card, privacy standard).

5M **Explain the metrics used to specify non-functional requirements.**

How to answer → Six-row table: speed, size, ease of use, reliability, robustness, portability with their measures; why quantify (goals are not testable).

8M **Explain the spiral view of the requirements engineering process with a diagram.**

How to answer → List the RE activities → draw Fig 4.7 → early vs later effort, exit after user requirements, output = system requirements document.

8M **Explain the requirements elicitation and analysis process with a diagram. What problems arise during elicitation?**

How to answer → Fig 4.8 → four activities one line each → iterative → five problems (table in Section 4.4).

10M **Explain interviewing and ethnography as elicitation techniques. Compare them.**

How to answer → Closed/open interviews, effective interviewing, problems → ethnography definition, two types of requirements it finds, focused ethnography + Fig 4.9 → comparison table.

6M **What are stories and scenarios? What should a scenario include? Illustrate with the iLearn photo-upload scenario.**

How to answer → Story vs scenario → five parts (S-N-W-O-E) → fill them in for KidsTakePics.

6M **Explain the different notations for writing a system requirements specification.**

How to answer → Five-row table: natural language, structured NL, design description languages, graphical, mathematical — with one weakness each.

6M **Give the guidelines for writing natural language requirements. What problems does natural language cause?**

How to answer → Five guidelines (format, shall/should, highlight, no jargon, rationale) → three problems → insulin pump 3.2 example.

8M **Explain structured and tabular specification using the insulin pump example.**

How to answer → Definition, when suitable → seven form fields → the SRS 3.3.2 form → the four-row condition/action table.

6M **What are use cases? Draw the use case diagram for the Mentcare system (or for an ATM).**

How to answer → Five use-case points → notation (stick figures, ellipses, lines) → Fig 4.10 → one textual description (Setup consultation).

10M **Explain the structure of a software requirements document. Who uses it and how?**

How to answer → Definition (WHAT not HOW) → Fig 4.11 users → ten chapters with one line each → variability (IEEE standard, incremental = less detail).

8M **What is requirements validation? Explain the checks and techniques used, and the review checks.**

How to answer → Definition + 100× cost → five checks → three techniques → reviews → four review checks.

10M **Why do requirements change? Explain requirements management planning and the change management process with a diagram.**

How to answer → Wicked problems + Fig 4.12 → three reasons → four planning decisions + tool support → Fig 4.13 with three stages.

10M **For a real-time system (insulin pump or wilderness weather station), identify stakeholders and write functional and non-functional requirements.**

How to answer → Follow the eight lab steps in Section 4.8: stakeholders → user reqs → numbered "shall" FRs → classified, measurable NFRs → one structured spec → validation checks.

UNIT 2 · CHAPTER 5

Design Concepts

Design in software engineering · The design process · Design concepts · The design model

STUDY TIME
About 3.5 hours

SOURCES
Pressman 7th ed Ch 8 (no slides for this chapter) + notes from the 8th/9th ed

EXAM WEIGHT
High: design concepts, cohesion vs coupling and the design model are classic 8–10 mark questions

AFTER THIS CHAPTER YOU CAN

- Say where design sits and why it decides quality
- Draw how the requirements model becomes the design model
- List the quality guidelines and FURPS attributes
- Explain every design concept with an example
- Compare abstraction/refinement and cohesion/coupling
- Draw the modularity–cost graph
- Name the 5 design-class types and 4 well-formed traits
- Draw the design-model dimensions and explain each element

5.1 Design within the context of software engineering ★★★ MUST KNOW

What is software design? ★★★ MUST KNOW

IN SIMPLE WORDS

Requirements say **what** the system must do. Design decides **how** it will be built — its parts, how they connect, what data they keep, how users and other systems talk to it. It is the architect's *blueprint* that programmers follow.

Software design covers the set of **principles**, **concepts** and **practices** that lead to a high-quality system or product. Principles give the overall philosophy; concepts must be understood before design practice is applied; practice creates representations of the software that guide construction (coding and testing).

DEFINITION — LEARN THIS LINE

Design model — a representation of the software that gives detail about the **software architecture, data structures, interfaces and components** needed to implement the system. (The requirements model describes the required **data, function and behaviour**.)

Who / why? Software engineers do it. The design can be assessed for quality and improved *before* code is written and tested — design is where software quality is established.

Steps	Represent the **architecture** → model the **interfaces** (to users, other systems/devices, and between its own components) → design the **components**. All follow the basic design concepts.
Work product	A **design model** with architectural, interface, component-level and deployment representations.
Check	The team reviews it for errors, inconsistencies and omissions, better alternatives, and whether it can be built within constraints, schedule and cost.

Firmness, commodity and delight. Vitruvius said good buildings show these three; so should good software — **firmness**: no bugs that inhibit its function; **commodity**: suitable for the purpose intended; **delight**: pleasurable to use. Designers reach such a model by **diversification** (collecting a repertoire of alternatives — components, solutions, knowledge) and then **convergence** (choosing the elements that meet the requirements and rejecting the rest until one configuration remains).

Where design sits ★★★ MUST KNOW

Design is the **technical kernel** of software engineering and is applied **whatever process model** is used. It begins once requirements have been analysed and modelled; it is the last software engineering action within the modeling activity and sets the stage for **construction** (code generation and testing).

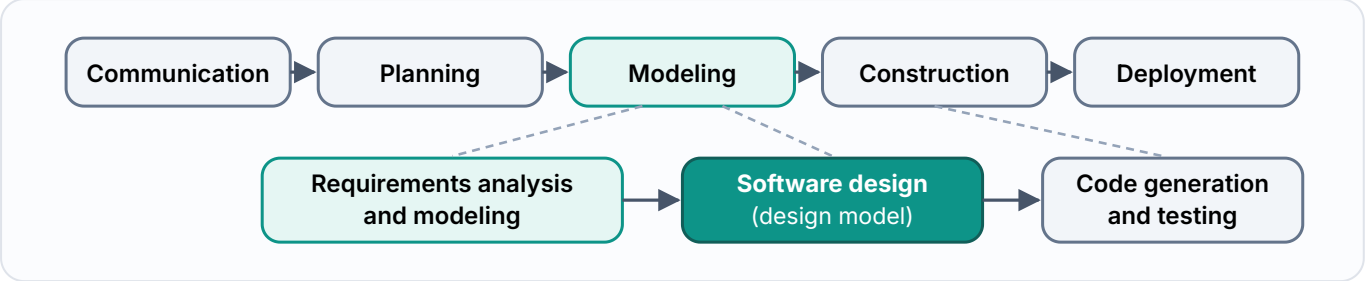


Fig 5.1 Design is the last action of modeling; its output, the design model, is the blueprint for construction (shown on Pressman's five framework activities). Drawn for this guide from Pressman 8.1

Translating the requirements model into the design model ★★★ MUST KNOW

The requirements model — its **scenario-based, class-based, flow-oriented and behavioural elements** — feeds the design task. Using design notation and methods, design produces the **four design models**: a **data/class design, an architectural design, an interface design and a component-level design**.

Design model	What it defines	Main input
Data/class design	Transforms class models into **design class realizations** and the **data structures** needed for implementation (partly with the architecture; more detail as each component is designed).	CRC objects and relationships; class attributes
Architectural design	The **relationships between major structural elements**, the **architectural styles and design patterns** that meet the requirements, and the **constraints** on implementation — the "framework" of the system.	The requirements model
Interface design	How the software **communicates** with interoperating systems and with the **humans** who use it. An interface implies a flow of information (data and/or control) and a specific behaviour.	Usage scenarios, behavioural models

Design model	What it defines	Main input
Component-level design	Transforms structural elements of the architecture into a **procedural description of software components**.	Class-based, flow and behavioural models

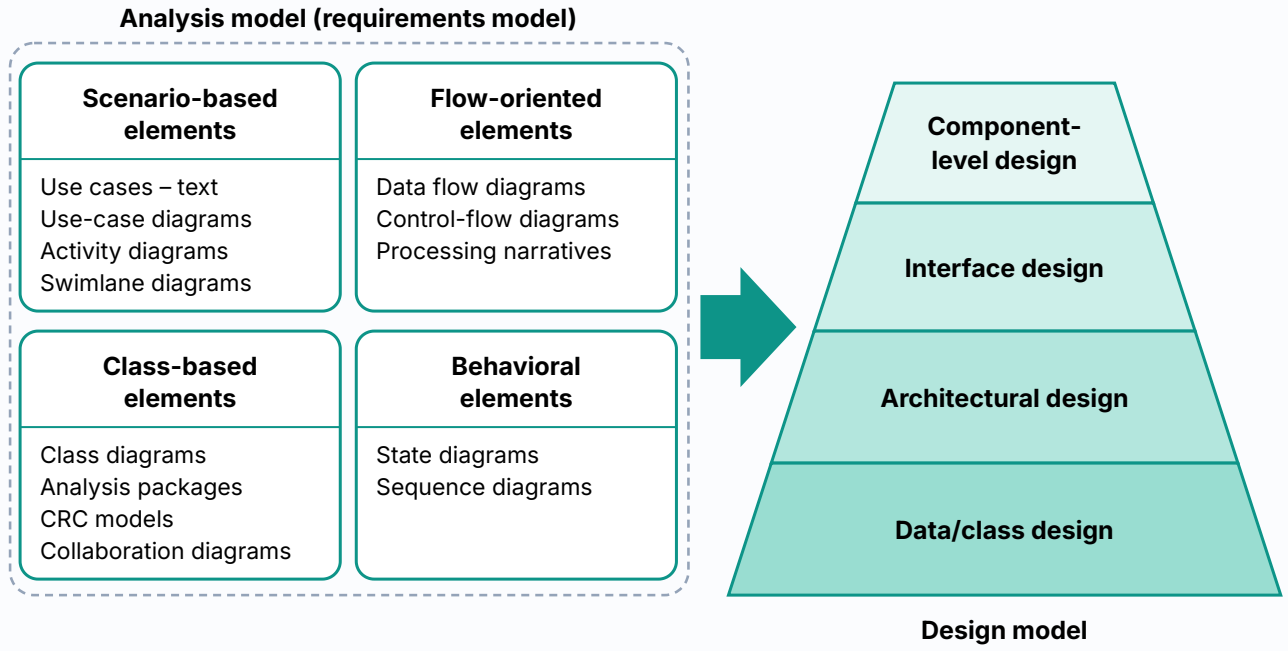


Fig 5.2 Translating the requirements model into the design model: the four kinds of requirements element feed the design task, which produces the four layers of the design model. Pressman Fig 8.1

★ KEY POINT

Design should **always begin with data** — the foundation for all other elements (the base of the pyramid). Then the **architecture** is derived; only then are the other design tasks done.

Why is design important? Quality ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Importance of design — in one word, **quality**: design is the place where **quality is fostered** in software engineering.

1. Design gives **representations of software that can be assessed for quality**.
2. It is the **only way to accurately translate stakeholders' requirements** into a finished product.
3. It is the **foundation for all the software engineering and support activities** that follow.
4. **Without design** the system may be unstable — it **fails when small changes are made**, is **difficult to test**, and its **quality cannot be assessed until late**, when time is short and money is spent.

Design vs coding (SafeHome): code is just the language used to *represent* a design. A programming language shows details (data structures, algorithms) well but not **architecture or component collaboration** — "a screwed-up architecture can ruin even the best code". The difference is the **level of abstraction**.

5.2 The design process ★★ IMPORTANT

Design is an **iterative process** that translates requirements into a **"blueprint"** for constructing the software. The first blueprint is *holistic* — at a **high level of abstraction**, traceable directly to the system objective and to the data, functional and behavioural requirements. Each iteration **refines** it to **lower levels of abstraction**, still traceable to requirements but more subtly.

Three characteristics of a good design ★★★ MUST KNOW

McGlaughlin's three characteristics guide the evaluation of a design (each is also a *goal* of the design process):

1. It must **implement all the explicit requirements** of the requirements model and **accommodate all the implicit requirements** desired by stakeholders.
2. It must be a **readable, understandable guide** for those who generate code and those who test and support the software.
3. It should give a **complete picture of the software**, addressing the data, functional and behavioural domains from an implementation perspective.

Quality guidelines ★★ IMPORTANT

1. A design should exhibit an **architecture** that (a) uses recognizable **architectural styles or patterns**, (b) is made of **components with good design characteristics**, and (c) can be implemented in an **evolutionary fashion** (small systems may be designed linearly).
2. A design should be **modular** — logically partitioned into elements or subsystems.
3. It should contain **distinct representations of data, architecture, interfaces and components**.
4. It should lead to **data structures** appropriate for the classes, drawn from recognizable data patterns.
5. It should lead to **components with independent functional characteristics**.
6. It should lead to **interfaces that reduce the complexity of connections** between components and with the environment.
7. It should be derived using a **repeatable method** driven by information from requirements analysis.
8. It should use a **notation that effectively communicates its meaning**.

These come from fundamental design principles, systematic methodology and thorough review. During design there is no code to test, so quality is checked by **technical reviews**: a meeting of two to four people — a **review leader** (plans and runs it), a **recorder** (takes notes) and the **producer** (whose work is reviewed) — who read the design beforehand and list errors, omissions and ambiguities (typically 90 minutes to 2 hours).

FURPS quality attributes ★★★ MUST KNOW

Hewlett-Packard's **FURPS** attributes are a **target for all software design**:

Attribute	Assessed / measured by
Functionality	The **feature set and capabilities** of the program, the **generality** of the functions delivered, and the **security** of the overall system.
Usability	**Human factors**, overall **aesthetics**, **consistency** and **documentation**.
Reliability	**Frequency and severity of failure**, **accuracy** of output, **mean-time-to-failure (MTTF)**, ability to **recover** from failure, and **predictability**.

Attribute	Assessed / measured by
Performance	**Processing speed, response time, resource consumption, throughput** and **efficiency**.
Supportability	**Extensibility, adaptability, serviceability** (together = *maintainability*) plus **testability, compatibility, configurability**, ease of **installation** and ease of **localizing problems**.

The attributes are **not weighted equally** (one system stresses functionality/security, another performance, another reliability), but all must be considered as design begins, not after construction starts.

> **MEMORY TRICK**
> **FURPS** — "**F**ast **U**sers **R**arely **P**ick **S**low apps". Supportability hides maintainability (extensibility + adaptability + serviceability).

Evolution of design and common features of design methods ★★ IMPORTANT

Evolution (about six decades): modular programs and top-down refinement → structured programming → translating data flow or data structure into a design → object-oriented design → software architecture and design patterns → aspect-oriented methods, model-driven and test-driven development.

All design methods share four characteristics: (1) a **mechanism for translating** the requirements model into a design representation; (2) a **notation** for functional components and their interfaces; (3) **heuristics for refinement and partitioning**; (4) **guidelines for quality assessment**.

Generic task set for design ★★ IMPORTANT

1. **Design data structures** for the data objects and attributes in the information domain model.
2. **Select an architectural style** appropriate for the software.
3. **Partition the analysis model into design subsystems** and allocate them in the architecture (each subsystem functionally cohesive; design subsystem interfaces; allocate analysis classes or functions).
4. **Create design classes or components**: translate analysis classes into design classes; check against design criteria and inheritance; define methods and messages; select design patterns; review and revise.
5. **Design interfaces** to external systems or devices.
6. **Design the user interface**: review task analysis; specify action sequences from scenarios; build a behavioural model; define interface objects and control mechanisms; review.
7. **Conduct component-level design**: specify algorithms at a low level of abstraction; refine each interface; define component data structures; review and correct errors.
8. **Develop a deployment model**.

5.3 Design concepts ★★★ MUST KNOW

A set of fundamental design concepts has **stood the test of time**; each gives the designer a foundation for more sophisticated methods, in both traditional and OO development. They help answer: **What criteria partition software into components? How is function or data-structure detail separated from a conceptual representation? What uniform criteria define the technical quality of a design?** As M. A. Jackson put it, wisdom begins with knowing the difference between **getting a program to work** and **getting it right**.

THE CONCEPTS IN TEXTBOOK ORDER + MEMORY TRICK

1 Abstraction · **2** Architecture · **3** Patterns · **4** Separation of concerns · **5** Modularity · **6** Information hiding · **7** Functional independence · **8** Refinement · **9** Aspects · **10** Refactoring · **11** OO design concepts · **12** Design classes · (newer editions: **13** Dependency inversion · **14** Design for test).

"**A**n **A**rchitect **P**lans **S**eparate **M**odules, **H**ides **I**nfo, keeps them **F**unctionally **I**ndependent, **R**efines, adds **A**spects, **R**efactors, thinks in **O**bjects and **D**esign classes."

1 · Abstraction ★★★ MUST KNOW

You say "book a train ticket" without listing the forty clicks behind it. A modular solution can be described at many **levels of abstraction**: at the **highest** level, in broad terms using the **language of the problem environment**; at **lower** levels, in more detail, mixing problem-oriented and **implementation-oriented terms**; at the **lowest** level, in a form that can be **directly implemented**.

DEFINITION — LEARN THIS LINE

Procedural abstraction — a sequence of instructions that have a **specific and limited function**. The **name** implies these functions, but specific **details are suppressed**.

Data abstraction — a **named collection of data that describes a data object**.

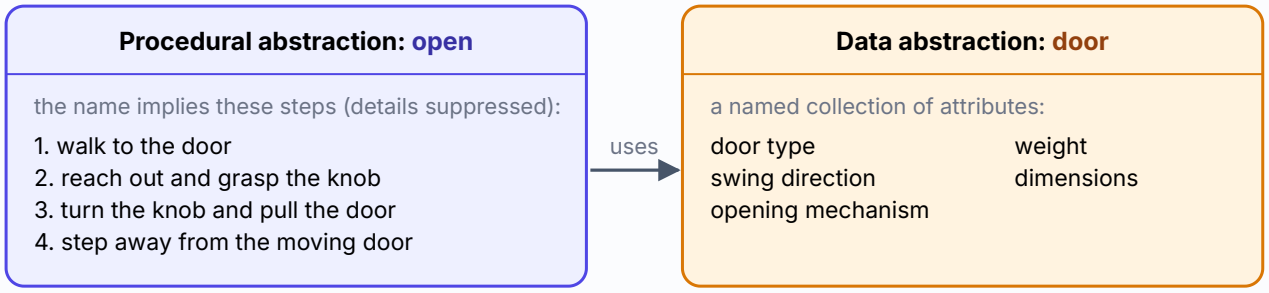


Fig 5.3 The textbook's door example: the procedural abstraction *open* uses the attributes of the data abstraction *door*. Drawn for this guide from Pressman 8.3.1

The steps behind *open* can be **replaced** (e.g., an automatic door with a sensor) as long as the function implied by the name stays the same — users of the abstraction are unaffected. SafeHome-style example: data abstraction **Sensor** (type, id, location, threshold) with procedural abstraction **armSystem()**.

✎ EXAM TIP

Open with Grady Booch: "Abstraction is one of the fundamental ways that we as humans cope with complexity." Then the levels, both types and the door example — a complete 5-mark answer.

2 · Architecture ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software architecture — "the overall structure of the software and the ways in which that structure provides **conceptual integrity** for a system" (Shaw and Garlan).

In simplest form it is the **structure or organization of program components (modules)**, the **manner in which they interact**, and the **structure of data** they use (more broadly, major system elements and their interactions). Think of it as the software's **floor plan**. The architectural rendering is the **framework** for

detailed design; **architectural patterns** solve common design problems.

Property to specify (Shaw and Garlan)	Meaning
Structural properties	The **components** (modules, objects, filters) and how they are **packaged and interact** — e.g., objects encapsulate data and processing and interact by invoking methods.
Extra-functional properties	How the architecture achieves **performance, capacity, reliability, security, adaptability** and other characteristics.
Families of related systems	Draw on **repeatable patterns** found in families of similar systems — the ability to **reuse architectural building blocks**.

Five kinds of architectural model: **structural** (an organized collection of program components); **framework** (repeatable design frameworks of similar applications — higher abstraction); **dynamic** (behaviour — how structure or configuration changes with external events); **process** (the business or technical process the system must accommodate); **functional** (the functional hierarchy). **Architectural description languages (ADLs)** describe components and how they connect. There is debate whether architecture is derived *separately* (between requirements engineering and design) or is an *integral part* of design. Advice: don't let architecture "just happen" — design it explicitly.

> **MEMORY TRICK**
> Five models — "**S**ome **F**amous **D**esigners **P**lan **F**irst": Structural, Framework, Dynamic, Process, Functional.

3 · Patterns ★★ IMPORTANT

> **DEFINITION — LEARN THIS LINE**
> **Design pattern** — "a named nugget of insight which conveys the essence of a **proven solution** to a **recurring problem** within a certain **context** amidst competing concerns" (Brad Appleton).

A pattern describes a design structure that solves a particular problem in a specific context and amid **"forces"** that affect how it is applied. Its description lets the designer decide (1) whether it is **applicable** to the current work, (2) whether it can be **reused** (saving design time), and (3) whether it can **guide the development of a similar** but functionally or structurally different pattern. (Christopher Alexander: a pattern can be used "a million times over, without ever doing it the same way twice".) *Everyday example:* a traffic roundabout — one named, proven idea for crossing traffic, built differently each time. *Software:* the Observer pattern — when a SafeHome sensor trips, the display, alarm and dialler that depend on it are all notified.

4 · Separation of concerns ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Separation of concerns** — any complex problem can be more easily handled if it is **subdivided into pieces that can each be solved and/or optimized independently** (Dijkstra). A **concern** is a feature or behaviour specified in the requirements model.

If problem p1 is perceived as more complex than p2, it needs more effort. And the perceived complexity of two problems **combined** is often **greater than the sum** of each taken separately:

$$C(p1) > C(p2) \Rightarrow E(p1) > E(p2) \quad \text{and} \quad C(p1 + p2) > C(p1) + C(p2) \Rightarrow E(p1 + p2) > E(p1) + E(p2)$$

Hence **divide and conquer**. Separation of concerns appears in **modularity**, **aspects**, **functional independence** and **refinement**.

> ⚠ **WATCH OUT**
> It can be taken too far: split a problem into very many tiny pieces and each is easy, but **integration may be very difficult** — see the modularity graph (Fig 5.4).

5 · Modularity ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Modularity** — software is divided into **separately named and addressable components**, sometimes called **modules**, that are integrated to satisfy problem requirements. It is the most common manifestation of separation of concerns. "Modularity is the single attribute of software that allows a program to be **intellectually manageable**" (Myers).

Monolithic software (one large module) cannot be easily grasped — its number of **control paths**, **span of reference**, **number of variables** and overall **complexity** make understanding close to impossible. So the design is broken into modules — but not indefinitely:

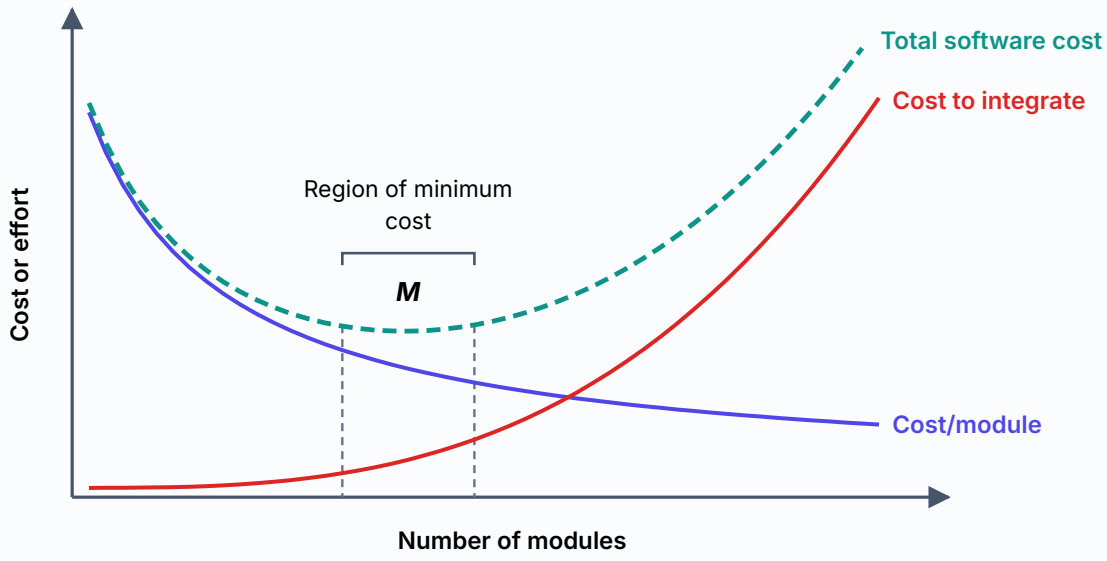


Fig 5.4 Modularity and software cost: cost per module falls as modules are added, the cost to integrate rises, and total cost is lowest near *M* modules. Pressman Fig 8.2

- **Cost/module** falls as the number of modules rises (same requirements, more modules ⇒ smaller modules); the **cost to integrate** rises.
- The **total cost** is U-shaped: some number ***M*** of modules gives minimum development cost, but we cannot predict *M* with assurance.
- So modularize, but stay in the **vicinity of *M*** — avoid undermodularity and overmodularity.

Why modularize? Development can be more easily **planned**; **increments** can be defined and delivered; **changes** are easier to accommodate; **testing and debugging** are more efficient; long-term **maintenance** has fewer serious side effects.

6 · Information hiding ★★★ MUST KNOW

Modularity raises the question "How do I decompose a solution to obtain the *best* set of modules?" Parnas answers it:

> **DEFINITION — LEARN THIS LINE**
>
> **Information hiding** — modules should be "characterized by design decisions that (each) hides from all others": the **information (algorithms and data) inside a module is inaccessible to other modules that have no need for it**.

Effective modularity comes from **independent modules that communicate only the information necessary** to achieve software function. **Abstraction** defines the procedural/informational entities; **hiding** defines and **enforces access constraints** to procedural detail and **local data structures**, which sit behind a **module interface**.

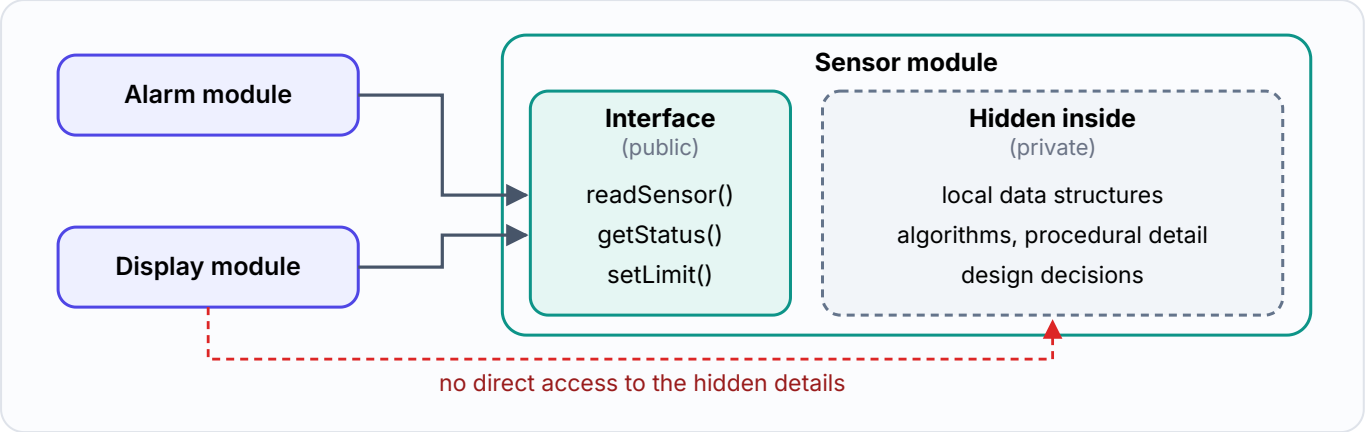


Fig 5.5 Information hiding: other modules use only the interface; data structures and algorithms stay hidden, so changing them cannot break the callers. Drawn for this guide

Benefit: greatest when **modifications are needed during testing and maintenance** — because most detail is hidden, errors introduced during modification are less likely to propagate to other parts of the software. (Like driving a car through its steering, pedals and gears: the engine can change from petrol to electric and you still drive the same way.)

7 · Functional independence: cohesion and coupling ★★★ MUST KNOW

Functional independence grows directly out of **separation of concerns, modularity, abstraction and information hiding** (Wirth, Parnas; solidified by Stevens, Myers and Constantine).

> **DEFINITION — LEARN THIS LINE**
>
> **Functional independence** — achieved by developing modules with **"single-minded" function** and an **"aversion" to excessive interaction** with other modules: each module addresses a **specific subset of requirements** and has a **simple interface**.
>
> **Cohesion** — an indication of the **relative functional strength** of a module (the degree to which it focuses on **just one thing**).
>
> **Coupling** — an indication of the **relative interdependence among modules** (the degree to which a module is connected to other modules and the outside world).

Why independence? Independent modules are **easier to develop** (function is compartmentalized, interfaces simplified — vital for teams) and **easier to maintain and test** (secondary effects of changes are limited, **error propagation is reduced**, and **reuse** becomes possible). "Functional independence is a key to good design, and design is the key to software quality."

- **Cohesion** extends information hiding: a cohesive module performs a **single task** needing little interaction with others — ideally it does **just one thing**. Multiple functions are sometimes necessary, but **"schizophrenic" modules** (many unrelated functions) must be avoided. Aim: high cohesion.
- **Coupling** depends on **interface complexity**, the **point at which entry or reference** is made to a module, and **what data pass across the interface**. Aim: lowest possible coupling — simple connectivity is easier to understand and less prone to the **"ripple effect"** (an error in one place propagating through the system).

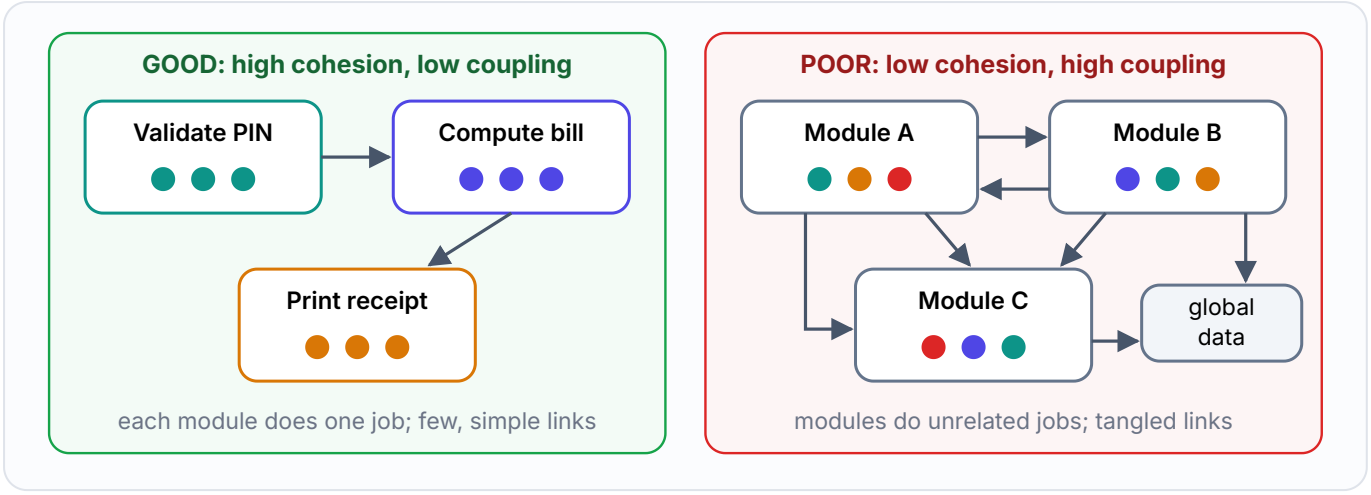


Fig 5.6 Functional independence. Dots of one colour = related functions. Left: single-minded modules with one simple link each. Right: mixed functions, many links and shared global data — a change ripples everywhere. Drawn for this guide

Point	Cohesion	Coupling
Measures	Functional strength **inside** one module	Interdependence **between** modules
Goal	**High** — the module does one thing	**Low** — few, simple connections
Depends on	How closely its functions and data are related	Interface complexity, point of entry/reference, data passed
Bad case	"Schizophrenic" module with unrelated jobs	"Ripple effect" of errors through the system

MEMORY TRICK

Co-HE-sion = inside one module; COUPLE-ing = between two. Good design = **high cohesion, low coupling** — "strong teams, few meetings".

Types of cohesion and coupling (Pressman Ch 10, component-level design)

Chapter 8 only defines the two ideas; their types are listed later in the book. Learn them if your paper asks for "types".

Cohesion type	Meaning (highest level first)
Functional	Shown mainly by operations: the component performs a targeted computation and returns a result.

Cohesion type	Meaning (highest level first)
Layer	Packages/components/classes: a higher layer uses services of a lower layer, but lower layers never access higher ones.
Communicational	All operations that access the same data are defined within one class.

Coupling categories: **content** (one component secretly modifies another's internal data — violates information hiding); **common** (components share a global variable); **control** (A() passes a control flag that directs B's logic); **stamp** (ClassB is the type of an argument of a ClassA operation); **data** (long strings of data arguments are passed); **routine call** (one operation invokes another); **type use** (A uses a data type defined in B); **inclusion/import** (A includes or imports B's package or content); **external** (communication with infrastructure — OS, database, telecom).

The classic scales (structured design — also widely examined)

Cohesion, worst → best: coincidental (unrelated parts) → logical (similar tasks, one chosen by a flag) → temporal (tasks done at the same time, e.g., start-up) → procedural (steps in a fixed order) → communicational (parts share the same data) → sequential (output of one part feeds the next) → functional (all parts serve one well-defined task).
Coupling, worst → best: content (uses/changes another's internals) → common (shared global data) → external (shared external format, protocol or device) → control (passes a control flag) → stamp (passes a whole data structure when only part is needed) → data (passes only the elementary data needed).

8 · Refinement (stepwise refinement) ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Stepwise refinement — a **top-down design strategy** proposed by **Niklaus Wirth**: a program is developed by **successively refining levels of procedural detail**; a macroscopic statement of function (a procedural abstraction) is decomposed step by step **until programming-language statements are reached**.

Refinement is a **process of elaboration**: start with a statement of function (or information) at a high level of abstraction that says nothing about internal workings, then add **more detail at each successive refinement**.

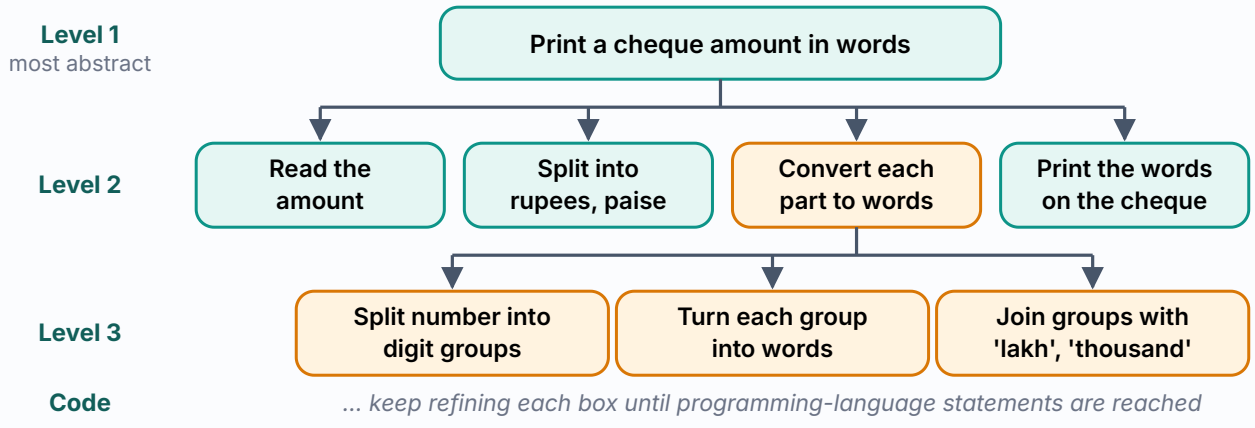


Fig 5.7 Stepwise refinement of a cheque writer (the textbook's exercise 8.12a): each level elaborates one box of the level above. Drawn for this guide

Abstraction and refinement are complementary. Abstraction lets you specify procedure and data internally but **suppress the need for "outsiders" to know low-level details**; refinement **reveals low-level details** as design progresses. Together they build the complete design model.

Point	Abstraction	Refinement
Direction	Hides detail — up to the big picture	Adds detail — down towards code
Purpose	Outsiders need not know low-level details	Reveals low-level details as design progresses
Result	Named procedural and data abstractions	A hierarchy of ever more detailed steps

⚠ **WATCH OUT**

Don't jump straight to full detail and skip refinement steps — it causes **errors and omissions** and makes the design much harder to review.

9 · Aspects ★★ IMPORTANT

Requirements analysis uncovers **concerns** — "requirements, use cases, features, data structures, quality-of-service issues, variants, intellectual property boundaries, collaborations, patterns and contracts". Ideally each is isolated, but **some concerns span the entire system** and cannot easily be compartmentalized.

DEFINITION — LEARN THIS LINE

Crosscutting — requirement A crosscuts requirement B "if a software decomposition [refinement] has been chosen in which B cannot be satisfied without taking A into account."

Aspect — a **representation of a crosscutting concern**.

Example (SafeHomeAssured.com): requirement **A** (use case ACS-DCV) lets a registered user view video from cameras placed in a space — design representation A*. Requirement **B**: a registered user must be **validated** before using SafeHomeAssured.com — it applies to *every* function; design representation B*. **B* crosscuts A***, so B* is an **aspect** of the WebApp.

Aspects must be identified so that the design can accommodate them during refinement and modularization. Ideally an aspect is implemented as a **separate module (component)**, not as fragments **"scattered" or "tangled"** through many components; the architecture should support a mechanism for defining such a module. Typical crosscutting concerns: security, logging, error handling.

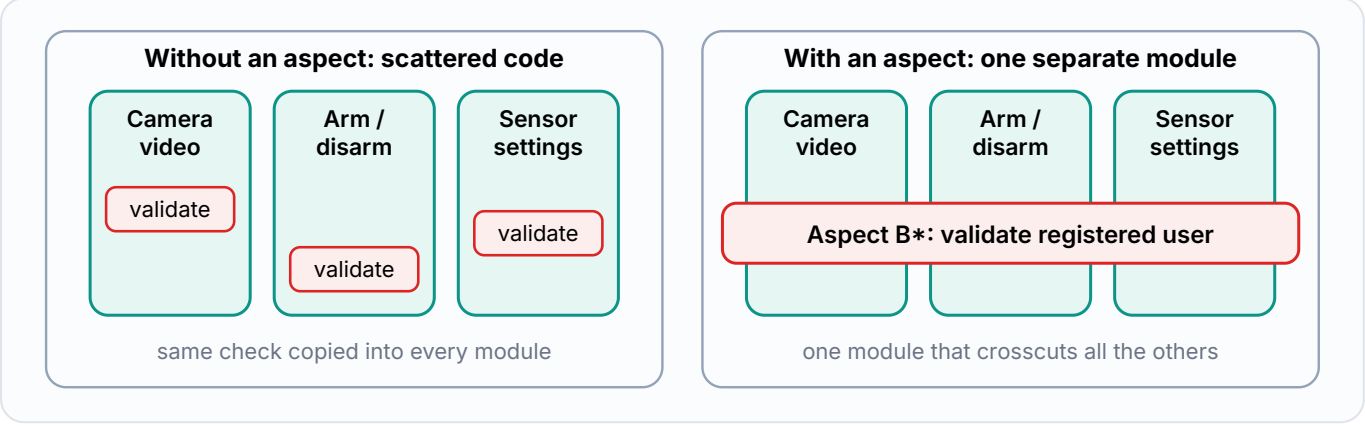


Fig 5.8 A crosscutting concern (user validation) as scattered/tangled fragments (left) versus as a separate aspect module (right). Drawn for this guide from Pressman 8.3.9

10 · Refactoring ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Refactoring — "the process of changing a software system in such a way that it **does not alter the external behavior** of the code [design] yet **improves its internal structure**" (Fowler).

An important design activity in many **agile methods** (e.g., Extreme Programming): a reorganization that **simplifies the design (or code) of a component without changing its function or behaviour**. The design is examined for **redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures**, or any other failure that can be corrected. Result: software that is **easier to integrate, test and maintain**.

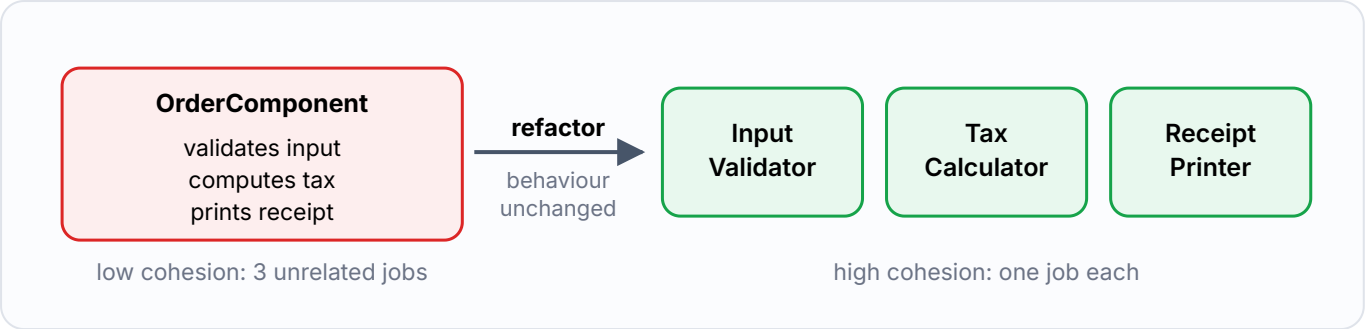


Fig 5.9 The textbook's refactoring case: a component performing three loosely related functions (low cohesion) is refactored into three highly cohesive components. Drawn for this guide from Pressman 8.3.10

Refinement vs refactoring: refinement *adds detail* top-down while the design is being created; refactoring is done *after* a design or code exists — "kind of an optimization pass through the software" — and changes only internal structure. Refactoring does **not** mean redesigning everything iteratively.

11 · Object-oriented design concepts ★★ IMPORTANT

The chapter names **classes and objects, inheritance, messages and polymorphism** and explains them in Appendix 2. Coad and Yourdon: **object oriented = objects + classification + inheritance + communication**.

Class	Encapsulates the **data and procedural abstractions** describing a real-world entity — a template/blueprint for similar objects.
Object	An **instance** of a class; it inherits the class's attributes and operations.
Attributes / operations	Attributes describe the class (values from a domain); operations (**methods, services**) are the algorithms that process the data.
Encapsulation	Data, operations and related information **packaged under one name** and reused as one component.
Inheritance	A **subclass** (specialization) inherits **all attributes and operations** of its **superclass** (generalization) — reuse is direct; superclass changes propagate down.
Messages	A message, *message(<parameters>)*, stimulates behaviour in the receiving object, which executes an operation.
Polymorphism	Lets **different operations have the same name**, reducing the effort to extend a design and decoupling objects.

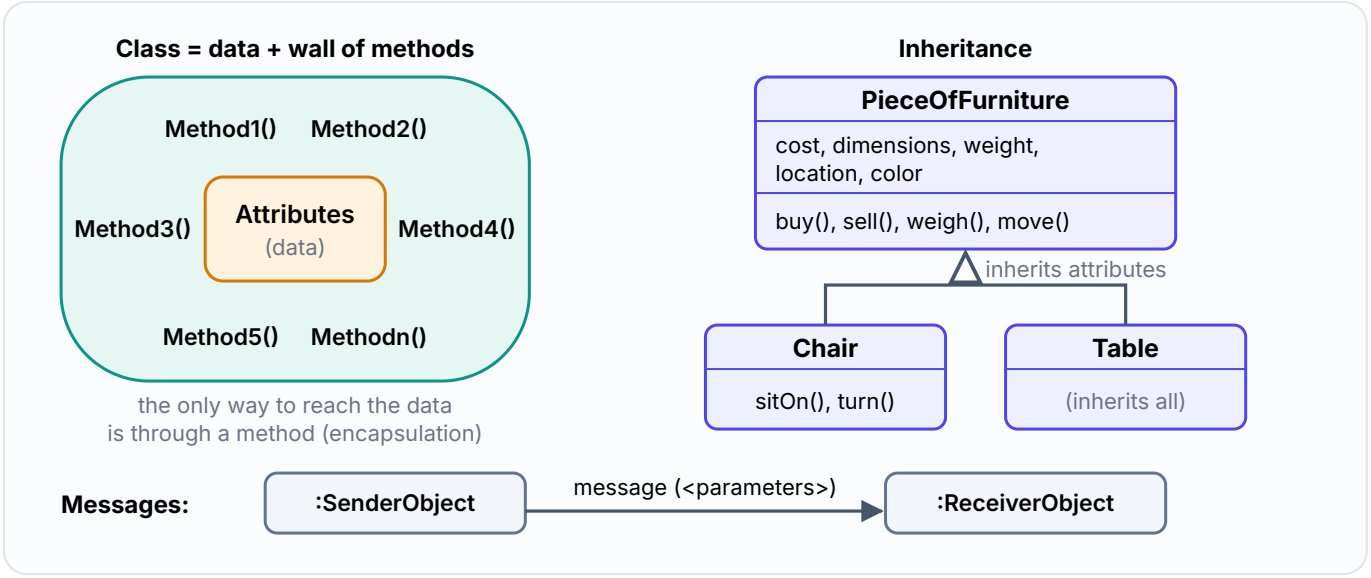


Fig 5.10 Core OO ideas: a class hides its data behind a "wall" of methods; subclasses inherit from a superclass; objects interact by messages. Redrawn from Pressman Appendix 2 (Figs A2.1, A2.2, furniture example)

Polymorphism example: a conventional program that draws line graphs, pie charts, histograms and Kiviat diagrams needs control logic (`if graphtype = linegraph then DrawLineGraph(data)` ...), and every new graph type means a new module *and* edited logic. In OO design each graph is a subclass of **Graph** with its own `draw` operation, so the design reduces to `draw <graphtype>` ; a new type is just a new subclass. A new class can be built from scratch, inherit from a class with most of what it needs, inherit after restructuring the hierarchy, or override existing characteristics; misused inheritance (e.g., multiple inheritance) makes designs error-prone.

> ★ KEY POINT
> The class "wall" **achieves information hiding**; methods working on few attributes **raise cohesion**; communicating only through methods **lowers coupling**. OO concepts reinforce concepts 6 and 7.

12 · Design classes ★★★ MUST KNOW

Analysis classes (from the requirements model) describe problem-domain elements that users see, at a high level of abstraction and in business jargon. **Design classes** (1) **refine the analysis classes** with the detail needed to implement them and (2) implement a **software infrastructure** supporting the business solution — with much more technical detail.

Design class type (Ambler)	Definition — each type is a layer of the design architecture	Example
User interface	All abstractions needed for **human–computer interaction (HCI)**, often visual elements of a metaphor (checkbook, order form, fax machine).	ControlPanelScreen
Business domain	Often **refinements of analysis classes**; the attributes and services (methods) to implement some business-domain element.	Sensor, Camera
Process	Implement **lower-level business abstractions** needed to fully manage the business domain classes.	SensorMonitor
Persistent	Represent **data stores** (e.g., a database) that **persist beyond the execution** of the software.	EventLogStore

Design class type (Ambler)	Definition — each type is a layer of the design architecture	Example
System	Implement **software management and control functions** so the system can operate and communicate within its environment and with the outside world.	NetworkConnection

Well-formed class (Arlow and Neustadt)	Meaning and textbook example
Complete and sufficient	**Complete:** contains all attributes and methods that can reasonably be expected from its name (class *Scene* in video-editing software has everything for creating a scene). **Sufficient:** only the methods needed for its intent — **no more and no less**.
Primitiveness	Each method does **one service**, and the class offers **no second way** to do the same thing (*VideoClip*: only *setStartPoint()* and *setEndPoint()* set start-point and end-point).
High cohesion	A **small, focused set of responsibilities**, single-mindedly implemented (all *VideoClip* editing methods work only on the clip's attributes).
Low coupling	Collaboration kept to an **acceptable minimum**; classes know little of other classes. **Law of Demeter**: a method should only send messages to methods in **neighbouring classes** ("only talk to your friends; don't talk to strangers").

MEMORY TRICK

Five types: "**U**nder **B**usy **P**rofessors, **P**upils **S**tudy" (User interface, Business domain, Process, Persistent, System). Well-formed: "**C**an **P**eople **H**ave **L**unch?" (Complete & sufficient, Primitiveness, High cohesion, Low coupling).

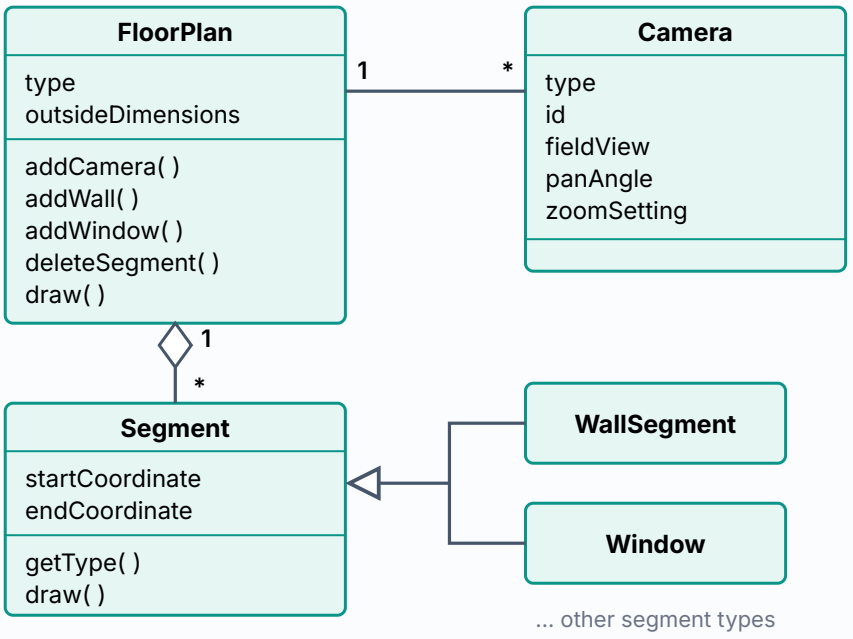


Fig 5.11 The analysis class FloorPlan refined into a design class: FloorPlan is an aggregation of Segments (lists of wall segments, windows, doors ...) and collaborates with many Cameras. New items can be added to or deleted from the aggregation easily. Pressman Fig 8.3

13 · Dependency inversion ★★ IMPORTANT (8th/9th edition)

In hierarchical architectures, high-level "control" components depend **directly** on low-level "worker" components, so changing a worker forces changes in the controller — the design is tightly **coupled**.

> **DEFINITION — LEARN THIS LINE**
>
> **Dependency inversion principle — high-level modules (classes) should not depend [directly] upon low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.**

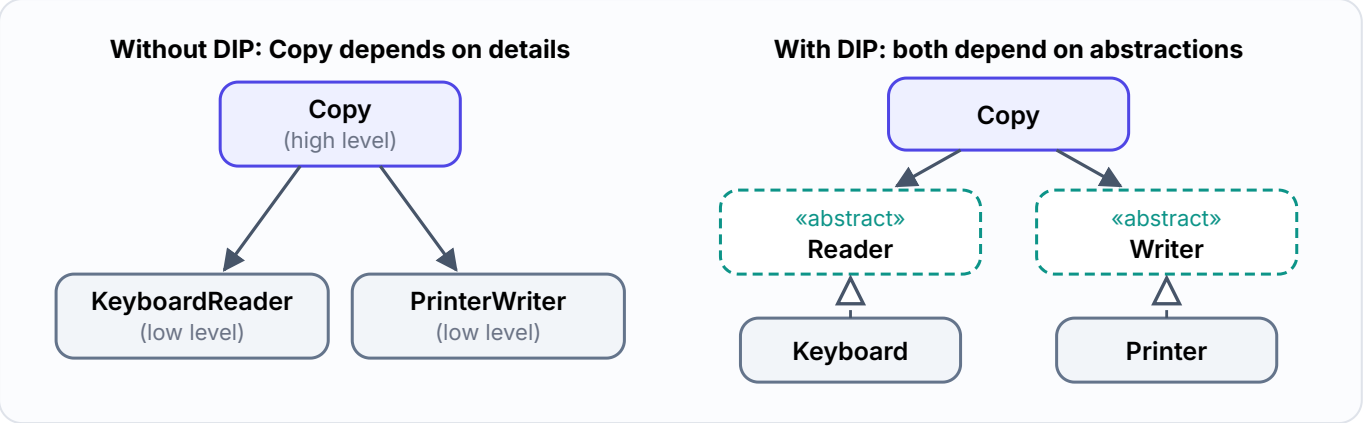


Fig 5.12 Dependency inversion: the control class Copy uses only abstract Reader/Writer classes; concrete Keyboard and Printer classes implement them (dashed arrow, hollow head). Drawn for this guide

The newer edition's illustration is a small program that reads keystrokes and prints them: the control module invokes the workers **through abstractions** (abstract classes). The abstract reader can point to a keyboard's *read()* in one context and a sensor's in another, so workers can be swapped without touching the controller. This **reduces coupling** and **improves testability**.

14 · Design for test ★★ IMPORTANT (8th/9th edition)

There is a chicken-and-egg debate about whether **software design or test-case design should come first**:

- In **test-driven development (TDD)** tests are written **before** other code and guide the design in short cycles: write a test → it fails → write just enough code → the test passes.
- If design comes first, it must include **seams** (also called **test hooks**) — places in the detailed design where test code can be inserted to **probe the state of the running software** and to **isolate the code under test** from its production environment. Seams must be **consciously designed at the component level**, thinking about the tests each component will need — like building a car with diagnostic ports already fitted.

5.4 Design concepts at a glance ★★★ MUST KNOW

Use this as the skeleton for "Explain the fundamental design concepts": one line each, then expand the ones you know best with a diagram.

Concept	One-line meaning	Why it matters
Abstraction	Name what something does, suppress how; procedural and data abstractions.	Think at one level at a time; reusable components.

Concept	One-line meaning	Why it matters
Architecture	Overall structure of the software giving conceptual integrity.	Framework for all detailed design.
Patterns	Named, proven solution to a recurring problem in a context.	Reuse of proven design knowledge; saves time.
Separation of concerns	Split a complex problem into independently solvable pieces.	Less total effort (divide and conquer).
Modularity	Separately named, addressable modules integrated to meet requirements.	Intellectually manageable; easier to plan, test, change.
Information hiding	Each module hides its design decisions (algorithms, data).	Errors made during change do not propagate.
Functional independence	Single-minded modules: high cohesion, low coupling.	Easier to develop, test, maintain, reuse; no ripple effect.
Refinement	Top-down, step-by-step elaboration down to code (Wirth).	Orderly detail; fewer errors and omissions.
Aspects	Representation of a crosscutting concern (e.g., security).	Crosscutting code kept in one module, not scattered.
Refactoring	Better internal structure, same external behaviour.	Simpler; easier to integrate, test, maintain.
OO design concepts	Classes, objects, inheritance, messages, polymorphism.	Encapsulation and reuse; supports hiding and low coupling.
Design classes	Analysis classes refined with technical detail; 5 types; well-formed.	Directly implementable classes and infrastructure.
Dependency inversion	High- and low-level modules both depend on abstractions.	Lower coupling; swappable parts; testability.
Design for test	Design seams/test hooks, or let tests drive design (TDD).	Components can be tested in isolation.

5.5 The design model ★★★ MUST KNOW

The two dimensions of the design model ★★★ MUST KNOW

The **process dimension** shows the **evolution of the design model** as design tasks are executed in the software process. The **abstraction dimension** shows the **level of detail** as each analysis-model element is transformed into a design equivalent and refined iteratively. The **dashed line** is the boundary between the analysis and design models — sometimes clear, sometimes the analysis model slowly blends into the design. Design uses many of the **same UML diagrams** as analysis, but **refined and elaborated** with more implementation-specific detail, emphasizing architectural structure and style, components, and interfaces.

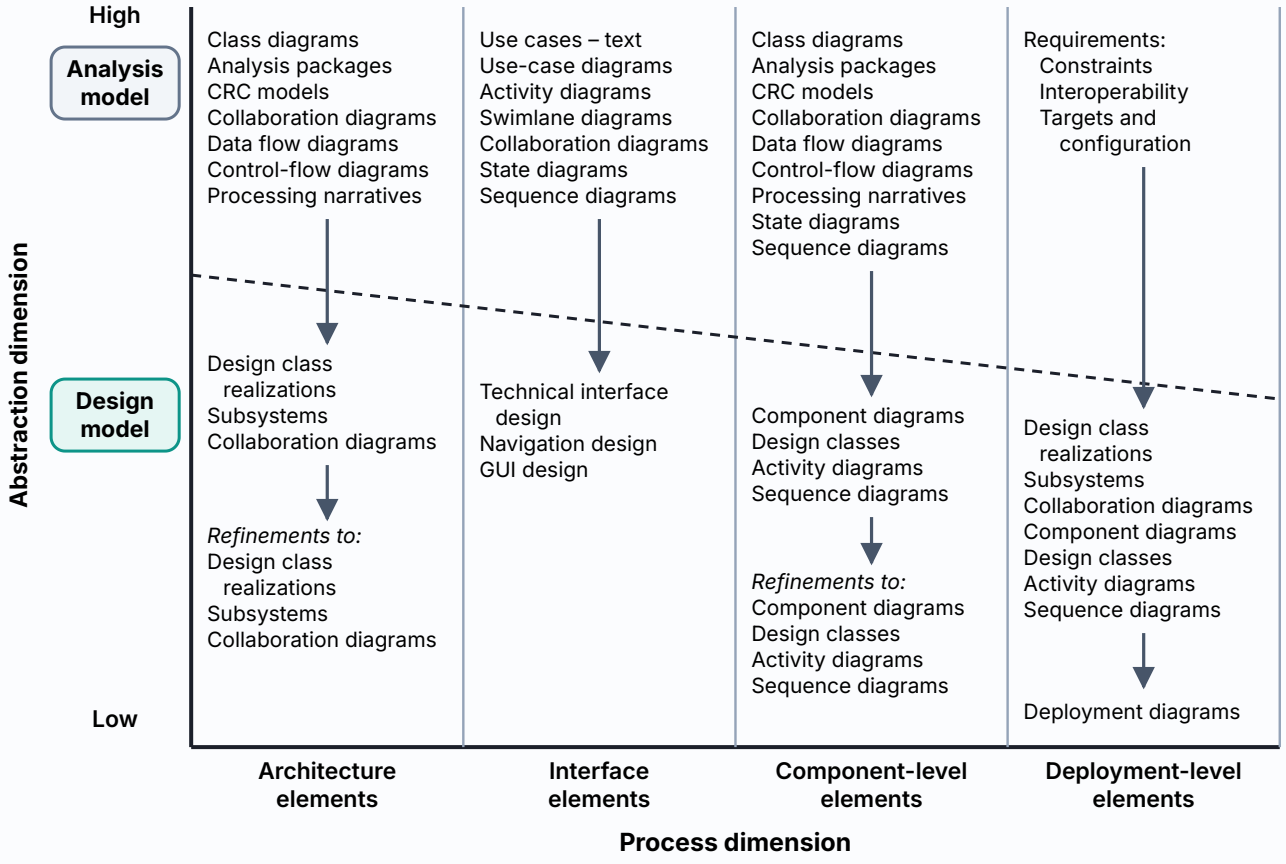


Fig 5.13 Dimensions of the design model. Right = the design evolves through the process; down = more detail. The dashed line separates the analysis model (above) from the design model (below). Pressman Fig 8.4

The elements on the horizontal axis are **not always developed in sequence**: preliminary **architectural design** usually comes first, then **interface** and **component-level design**, often **in parallel**; the **deployment model** is usually delayed until the design is fully developed. **Design patterns** can be applied at any point.

> ✎ **EXAM TIP**
>
> To draw it in the exam: two axes (abstraction high→low, process →), four columns, a sloping dashed line with "analysis model" above and "design model" below, and 2–3 diagram names per column with downward arrows.

Data design elements ★★ IMPORTANT

Data design (also called **data architecting**) builds a model of data/information at a **high level of abstraction** (the customer's/user's view), then refines it into progressively more **implementation-specific** representations. The **architecture of the data strongly influences the architecture of the software** that processes it. Data design matters at every level:

- **Program component level** — design of **data structures** and the **algorithms** that manipulate them.
- **Application level** — translation of the **data model** into a **database**, pivotal to the business objectives.
- **Business level** — data from disparate databases reorganized into a **data warehouse** for **data mining / knowledge discovery**.

Architectural design elements ★★★ MUST KNOW

Equivalent to the **floor plan of a house** — the layout of rooms, their size, shape and relationships, and the doors and windows that allow movement — architectural elements give an **overall view of the software**. The architectural model is derived from **three sources**:

1. information about the **application domain** of the software;
2. specific **requirements model elements** (e.g., data flow diagrams or analysis classes, their relationships and collaborations);
3. the availability of **architectural styles and patterns**.

It is usually drawn as a **set of interconnected subsystems**, often derived from analysis packages; each subsystem may have its own architecture (e.g., a GUI following a standard UI style).

Interface design elements ★★★ MUST KNOW

Like the **detailed drawings for the doors, windows and external utilities** of a house (how they operate, how water, power, gas and telephone come in and are distributed, where the doorbell and intercom are), interface elements show **how information flows into and out of the system** and **among the components** of the architecture. There are three:

Interface element	Key points
1. User interface	Now often called **usability design**. Includes **aesthetic** elements (layout, colour, graphics, interaction mechanisms), **ergonomic** elements (information layout and placement, metaphors, UI navigation) and **technical** elements (UI patterns, reusable components). The UI is usually a **unique subsystem** of the architecture.
2. External interfaces	To other **systems, devices, networks** or producers/consumers of information. Need definitive information about the other entity — gathered in requirements engineering and **verified** when interface design starts (it can change over time). Should include **error checking** and, if needed, **security features**.
3. Internal interfaces	Between **design components**; closely tied to component-level design. Design classes define the operations and **messaging schemes** for collaboration; each message carries the needed information for the requested operation. In input–process–output design, interfaces come from data flow and processing narratives.

DEFINITION — LEARN THIS LINE

Interface (UML) — "a specifier for the externally-visible [public] operations of a class, component, or other classifier (including subsystems) without specification of internal structure" — simply, a **set of operations** describing part of a class's behaviour and giving access to them.

In an advanced SafeHome, control-panel functions may be offered through a wireless PDA or mobile phone. **ControlPanel** implements *readKeyStroke()* and *decodeKey()*; to provide them to **WirelessPDA** and **MobilePhone**, an interface **KeyPad** is defined with **no attributes**, only the keypad operations. In UML this is a **realization**: part of ControlPanel's behaviour is implemented by realizing KeyPad's operations.

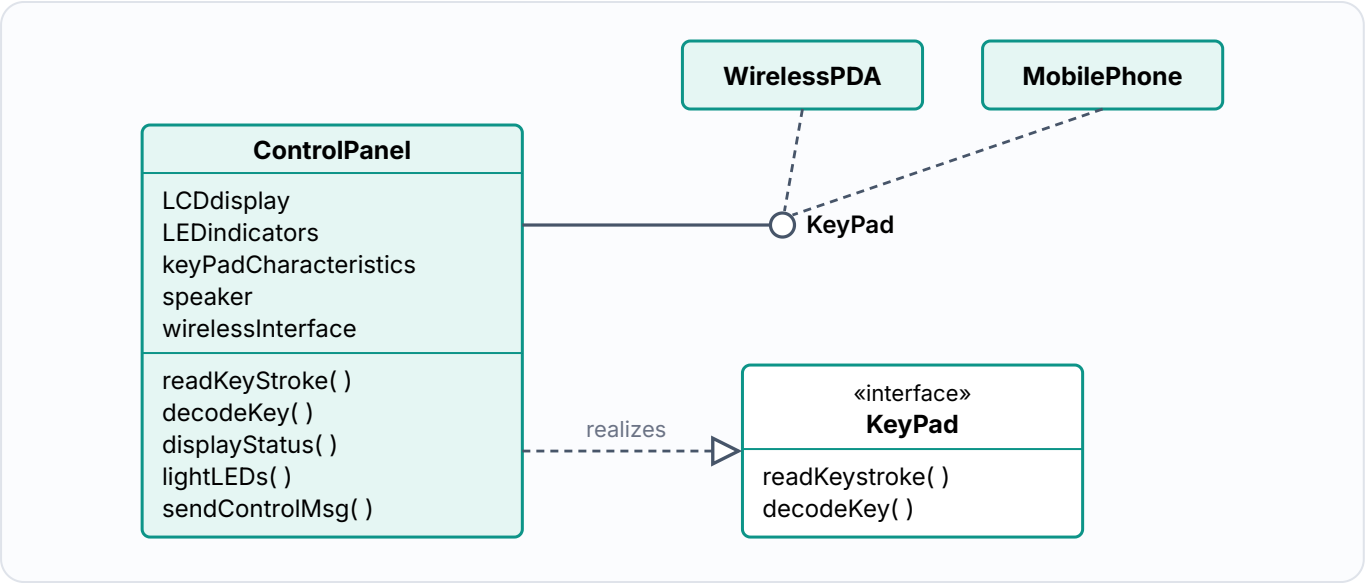


Fig 5.14 Interface representation for ControlPanel: KeyPad shown as a small labelled circle and as an «interface» box. The dashed line with an open triangle means ControlPanel *realizes* KeyPad, whose operations WirelessPDA and MobilePhone use. Pressman Fig 8.5

Component-level design elements ★★★ MUST KNOW

Like the **detailed drawings of each room** (wiring, plumbing, switches, sinks, cabinets, flooring), component-level design **fully describes the internal detail of each component**: **data structures** for all local data objects, **algorithmic detail** for all processing, and an **interface** giving access to all component operations. In OO work a component is drawn as a **UML component diagram** (Fig 5.15a): **SensorManagement** performs all SafeHome sensor functions (monitoring, configuring) and a dashed arrow links it to its assigned class **Sensor**. Details can be modelled with a **UML activity diagram** (processing logic), **pseudocode** or a **flowchart/box diagram** (procedural flow, following **structured programming** rules), and data structures in pseudocode or the implementation language.

Deployment-level design elements ★★ IMPORTANT

They show **how software functionality and subsystems are allocated within the physical computing environment**. SafeHome runs on a **home-based PC**, the **SafeHome control panel** and a **server at CPI Corp.** (Internet access). A **UML deployment diagram** (Fig 5.15b) shows the subsystems in each node — the PC houses security, surveillance, home management and communication, plus an **external access** subsystem for all outside access attempts. It begins in **descriptor form** (environment in general terms — "personal computer" could be a Mac, Windows PC, Sun workstation or Linux box) and is later redrawn in **instance form** (each specific, named hardware configuration identified) late in design or as construction begins.

All design model elements in one table ★★★ MUST KNOW

Element	House analogy	What it describes	Typical notation
Data	(the foundation)	Data model refined from the user's view into data structures, databases, data warehouses	Class diagrams, data structures
Architectural	Floor plan	Overall structure: interconnected subsystems, styles, patterns	Subsystems, design class realizations
Interface	Doors, windows, external utilities	Information flow in/out and between components: UI, external, internal	GUI/navigation design, «interface»

Element	House analogy	What it describes	Typical notation
Component-level	Detailed drawings of each room	Local data structures, algorithms, interface of each component	Component, activity diagrams, pseudocode
Deployment-level	(where each part is placed)	Allocation of subsystems to physical computing nodes	Deployment diagrams

⚠ **WATCH OUT**

The textbook counts the elements differently in different places: a margin note gives four (**data, architecture, components, interface**); the summary gives four (**architectural, interface, component-level, deployment-level**); the headings give **five**. If asked for "the elements of the design model", write all five.

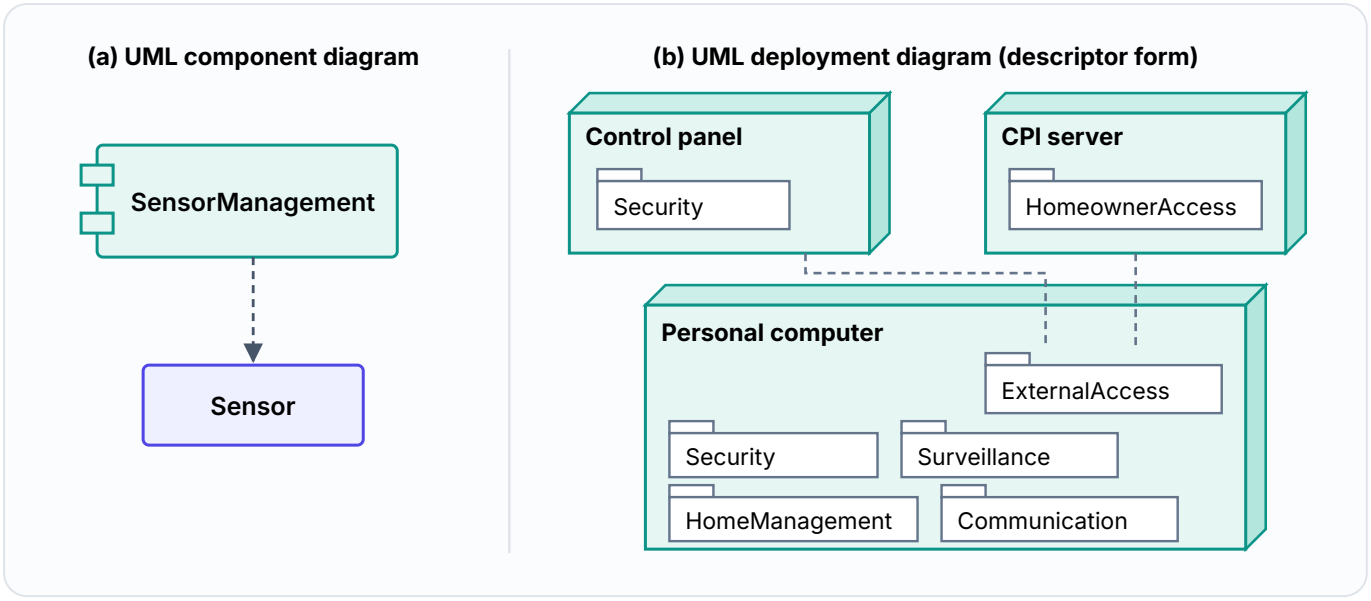


Fig 5.15 (a) UML component diagram: SensorManagement with its assigned class Sensor. (b) UML deployment diagram in descriptor form: three computing environments and the subsystems each houses. Pressman Figs 8.6 and 8.7

Chapter 5 in one page

Core ideas

- **Design model** = architecture, data structures, interfaces, components (requirements model = data, function, behaviour).
- **Why design?** Quality is fostered here: assessable, the only accurate translation of requirements, foundation for later work; without it the system is unstable.
- **Position**: last action of modeling, before construction, in any process model. Begin with data, then architecture.
- **Good design**: firmness, commodity, delight; diversification → convergence; iterative, high → low abstraction.

Lists to write word-perfect

- **3 characteristics**: explicit + implicit requirements; readable guide; complete picture (data, function, behaviour).
- **FURPS**: Functionality, Usability, Reliability, Performance, Supportability.
- **Methods share**: translation mechanism, notation, refinement/partitioning heuristics, quality guidelines.
- **Architecture**: structural, extra-functional, families of related systems; models: structural, framework, dynamic, process, functional.
- **Design classes**: user interface, business domain, process, persistent, system.
- **Well-formed**: complete & sufficient, primitiveness, high cohesion, low coupling (Law of Demeter).
- **Architecture sources**: domain, requirements elements, styles & patterns. **Interfaces**: UI, external, internal.

The concepts in one line each

- **Abstraction**: hide detail; procedural (*open*), data (*door*).
- **Architecture**: overall structure, conceptual integrity.
- **Patterns**: proven solution, recurring problem, context.
- **Separation of concerns**: divide and conquer.
- **Modularity**: named modules; stay near *M*.
- **Information hiding**: modules hide decisions (Parnas).
- **Functional independence**: high cohesion, low coupling.
- **Refinement**: top-down elaboration (Wirth).
- **Aspects**: crosscutting concern in its own module.
- **Refactoring**: better structure, same behaviour (Fowler).
- **OO**: class, object, inheritance, message, polymorphism.
- **Design classes**: 5 types, 4 well-formed traits.
- *Newer*: **dependency inversion**, **design for test**.

Diagrams to practise

- Fig 5.2 requirements → design pyramid
- Fig 5.4 modularity–cost graph
- Fig 5.6 cohesion / coupling
- Fig 5.13 design-model dimensions

Abstraction vs refinement

Abstraction hides low-level detail from outsiders; refinement reveals it step by step. Complementary.

Cohesion vs coupling

Cohesion = strength **inside** a module (high). Coupling = dependence **between** modules (low; no ripple effect).

Likely exam questions

2M **Define software design. Why is design called "the place where quality is fostered"?**

How to answer → Design model definition; representations can be assessed for quality before coding (5.1).

2M **What is procedural abstraction and data abstraction? Give an example of each.**

How to answer → Textbook definitions + *open* / *door* (Fig 5.3).

2M **Expand FURPS and state how each attribute is assessed.**

How to answer → FURPS table (5.2); supportability includes maintainability.

5M **Explain how the requirements model is translated into the design model, with a neat diagram.**

How to answer → Fig 5.2 → one line per design model and its input → "begin with data".

5M **What are the characteristics of a good design? List the design quality guidelines.**

How to answer → McGlaughlin's three characteristics + the eight guidelines (5.2).

10M **Explain the fundamental software design concepts.**

How to answer → Three questions they answer → 5.4 table as skeleton → expand abstraction, modularity (graph), information hiding, functional independence, refinement with small diagrams.

5M **Differentiate between abstraction and refinement.**

How to answer → Definitions (Booch / Wirth) → comparison table in concept 8 → "complementary".

6M **What is functional independence? Explain cohesion and coupling, and why high cohesion and low coupling are desired.**

How to answer → Definitions → Fig 5.6 → comparison table → benefits (develop, test, maintain, reuse, no ripple effect). If "types" are asked, add the coupling categories / cohesion scale.

6M **Explain modularity. Show the relation between number of modules and software cost with a graph.**

How to answer → Definition + Myers → Fig 5.4 (three curves, *M*) → avoid under-/over-modularity → five reasons.

4M **Explain information hiding and its benefits.**

How to answer → Parnas definition → Fig 5.5 → errors during modification don't propagate.

4M **What are aspects and crosscutting concerns? Explain with an example.**

How to answer → Concern → crosscut → aspect → SafeHomeAssured.com A*/B* → Fig 5.8.

4M **What is refactoring? How does it differ from refinement?**

How to answer → Fowler definition → what is examined → Fig 5.9 → refinement vs refactoring.

6M **Explain the five types of design classes and the characteristics of a well-formed design class.**

How to answer → Analysis vs design class → 5 types → 4 traits with Scene/VideoClip and the Law of Demeter.

10M **Explain the dimensions of the design model with a diagram, and describe each design model element.**

How to answer → Fig 5.13 → data, architectural (3 sources), interface (3 parts, Fig 5.14), component-level, deployment-level (Fig 5.15) with house analogies.

CIE-1 · UNIT 3

Unit 3: Architectural Design (CIE-1 part)

3

The big-picture structure of a system: the decisions architects make and the different views used to describe an architecture.

SYLLABUS, WORD FOR WORD

Architectural Design: Architectural design decisions, Architectural views, ~~Architectural patterns.~~

~~**Design and implementation:** Object-oriented design using the UML, Design patterns, Implementation issues.~~

CHAPTERS IN THIS UNIT

CHAPTER 6 — **Architectural Design: Decisions and Views**
Box-and-line models · Design decisions · Non-functional requirements · 4+1 views
page 125

COURSE OUTCOME

CO3: Develop architectural and detailed design using UML and design patterns.

LAB COMPONENT

High level and low level design of the analysed real-time case study application.

CIE-1 SCOPE

Only up to architectural views is in CIE-1. Struck-through topics come later.

UNIT 3 · CHAPTER 6

Architectural Design: Decisions and Views

Box-and-line models · Design decisions · Non-functional requirements · 4+1 views

STUDY TIME
About 2 hours

SOURCES
Your Ch 6 slides (1–20) + Sommerville Ch 6 (introduction, 6.1, 6.2)

EXAM WEIGHT
The Unit 3 part of CIE-1: short definitions plus one diagram answer (design decisions or 4+1 views)

AFTER THIS CHAPTER YOU CAN

- Define architectural design and its output
- Explain why architecture comes first, even in agile
- Draw and explain the packing robot architecture
- Contrast architecture in the small and large
- List three advantages of explicit architecture
- Discuss box-and-line diagrams and model uses
- Draw the architectural design decisions
- Match non-functional requirements to architecture
- Draw and explain the 4+1 view model
- Compare UML, ADLs and informal notations

6.1 What is architectural design? ★★★ MUST KNOW

Before builders lay a brick, an architect decides the overall layout of the building. Software needs the same big-picture plan. The **software architecture** of a system is its overall organization: the main parts (**components** or **sub-systems**) and the relationships between them. **Architectural design** produces this plan.

> **DEFINITION — LEARN THIS LINE**
>
> **Architectural design** — is concerned with understanding how a software system should be organized and designing the overall structure of that system.

Three points to write in any answer (slide 3):

1. **What it does** — works out how the system should be organized and designs its overall structure.
2. **Where it sits** — it is the first stage of the software design process and the critical link between design and requirements engineering, because it identifies the main structural components in a system and the relationships between them.
3. **What it produces** — an **architectural model** that describes how the system is organized as a set of communicating components.

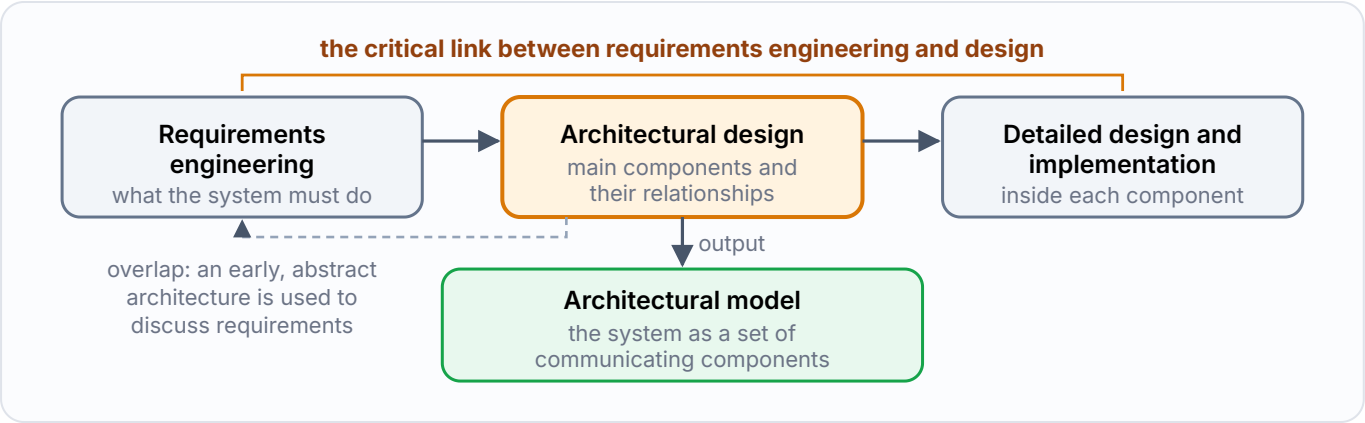


Fig 6.1 Where architectural design sits: between knowing *what* the system must do and designing the inside of each part. Drawn for this guide

Component	A part of the system with one well-defined job (e.g. the "vision system" of a robot). Large components are called sub-systems.
Stakeholder	Anyone with an interest in the system: users, customers, managers, engineers.
Functional / non-functional	Functional requirements say *what* the system does; non-functional requirements say *how well* (performance, security, safety, availability, maintainability).

Requirements and architecture overlap ★ READ ONCE

Ideally a specification contains no design, but this is unrealistic except for very small systems. In practice **requirements engineering and architectural design overlap**: you may propose an abstract architecture that groups system functions or features into large-scale components or sub-systems, and use it to discuss the requirements with stakeholders.

Why architecture matters ★★★ MUST KNOW

Software architecture affects the **performance, robustness, distributability and maintainability** of a system (Bosch). Individual components implement the functional requirements, but the dominant influence on the non-functional characteristics is the system's architecture. A study of "architecturally significant requirements" (Chen et al.) confirmed that non-functional requirements have the most significant effect on a system's architecture.

Agility and architecture ★★★ MUST KNOW

Agile methods build software in small increments with little design done up front. Even so:

- It is generally accepted that an **early stage** of agile processes is to design an overall system architecture. Incremental development of architectures is not usually successful.
- **Refactoring** (restructuring software to improve it without changing what it does) individual *components* in response to change is relatively easy.
- Refactoring the system *architecture* is usually **expensive because it affects so many components**: you may need to modify most system components to adapt them to the change.

> **IN SIMPLE WORDS**
> Repainting a room is cheap; moving the load-bearing walls means rebuilding most of the house. Components are the rooms; the architecture is the load-bearing walls.

6.2 Example: a packing robot control system ★★★ MUST KNOW

Sommerville's example is a **packing robot** that can pack different kinds of objects. It uses a vision component to pick out objects on a conveyor, identify the type of object and select the right kind of packaging. It then moves objects from the delivery conveyor to be packaged, and places packaged objects on another conveyor. Fig 6.2 is an **abstract model**: it shows only these components and the links between them.

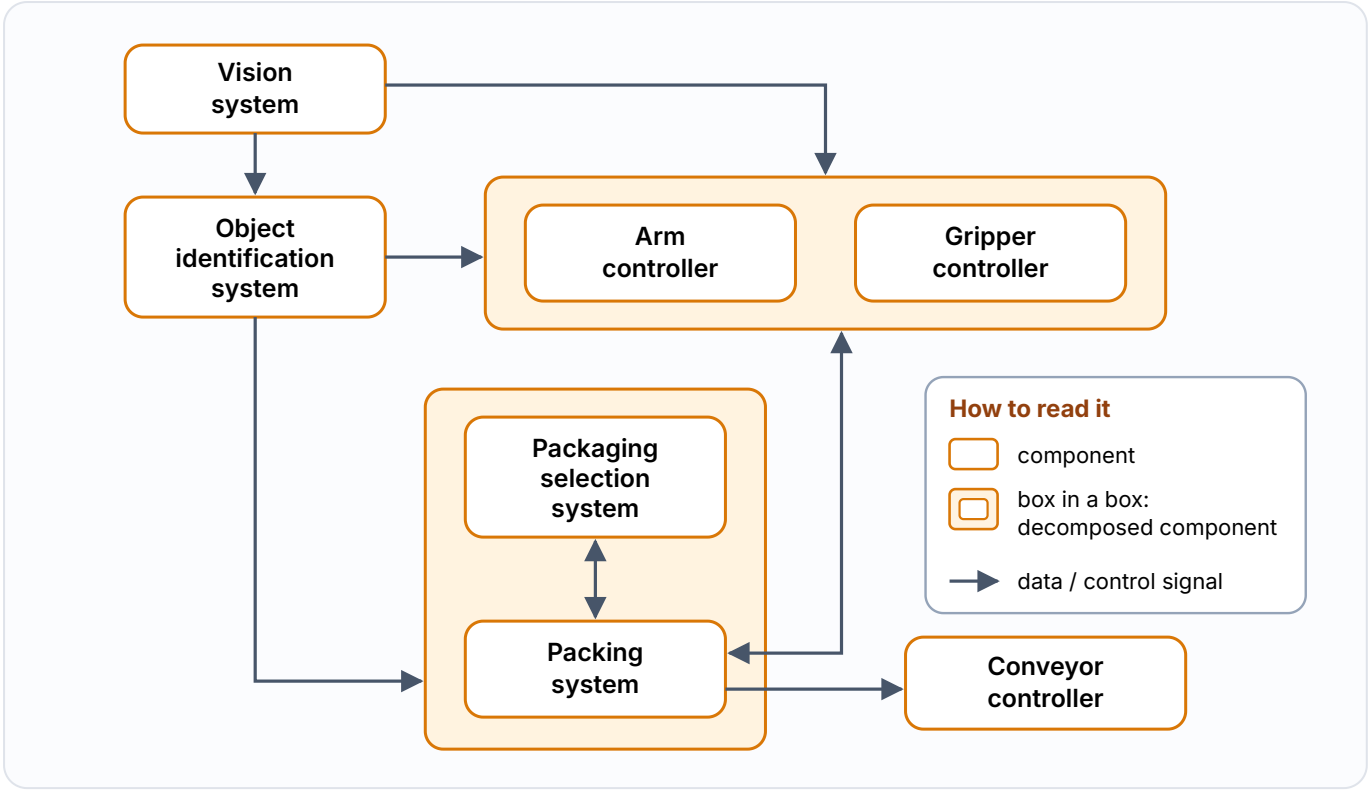


Fig 6.2 The architecture of a packing robot control system. Sommerville Fig 6.1 · Your slides, slide 5

How to read such a **block diagram** (also called a **box-and-line diagram**):

- Each **box** represents a component.
- **Boxes within boxes** mean the component has been decomposed into sub-components (Arm + Gripper controllers form one sub-system; Packaging selection + Packing form another).
- **Arrows** mean that data and/or control signals pass from component to component in the direction of the arrow. When you redraw Fig 6.2, keep the arrow directions and the two shaded groups.

6.3 Architectural abstraction: small and large ★★ IMPORTANT

Architectures can be designed at two levels of abstraction (amount of detail):

- **Architecture in the small** is concerned with the architecture of **individual programs**. At this level, we are concerned with the way that an individual program is **decomposed into components**. Sommerville's Chapter 6 is mostly about this level.
- **Architecture in the large** is concerned with the architecture of **complex enterprise systems** that include other systems, programs and program components. These enterprise systems may be **distributed over different computers**, which may be owned and managed by **different companies**.

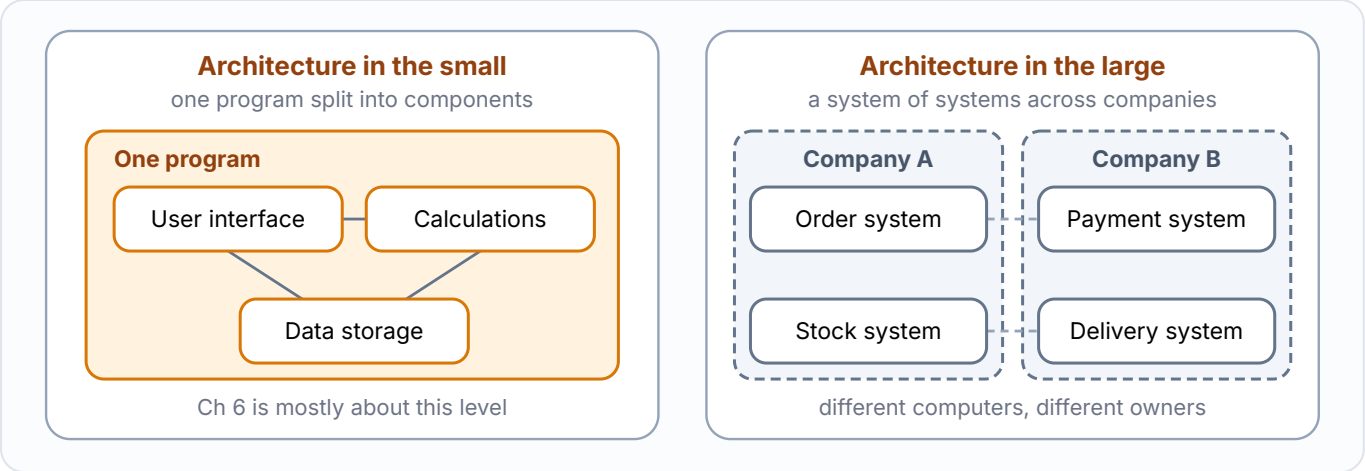


Fig 6.3 Two levels of architectural abstraction (component names are only illustrations). Drawn for this guide

6.4 Advantages of explicit architecture ★★★ MUST KNOW

"Explicit" means the architecture is deliberately designed and written down, not left to emerge by accident. Bass, Clements and Kazman suggest that explicitly designing and documenting a software architecture has **three advantages**:

Advantage	What it means
1. Stakeholder communication	The architecture is a high-level presentation of the system that may be used as a **focus of discussion** by a range of different system stakeholders.
2. System analysis	Making the architecture explicit at an early stage requires some analysis, so it becomes possible to analyse **whether the system can meet its non-functional requirements**. Architectural decisions have a profound effect on whether critical requirements such as performance, reliability and maintainability can be met.
3. Large-scale reuse	An architectural model is a compact, manageable description of how a system is organized and how its components interoperate. The architecture is often the same for systems with similar requirements, so it **may be reusable across a range of systems**. **Product-line architectures** may be developed: the same architecture is reused across a range of related systems.

MEMORY TRICK

Think **CAR**: **C**ommunication (with stakeholders), **A**nalysis (of the system), **R**euse (large-scale).

6.5 Architectural representations and their use ★★★ MUST KNOW

Box-and-line (block) diagrams ★★★ MUST KNOW

Simple, informal **block diagrams** showing entities and relationships (like Fig 6.2) are the **most frequently used** method for documenting software architectures. People from different disciplines can readily understand them. But they have been **criticised** (Bass et al.) because they lack semantics and are **very abstract**: they do not show the **types (nature) of relationships** between components, nor the **externally visible properties** of the entities or sub-systems. **However**, they are useful for **communication with stakeholders** and for **project planning**.

IN SIMPLE WORDS

Semantics = precise meaning. In Fig 6.2, is the arrow from *Vision system* to *Object identification* a function call, a message or shared data? The diagram does not say. Nor does it show what services each component offers (its "externally visible properties").

STRENGTHS

- ✓ Simple, quick and informal to draw
- ✓ Intuitive: domain experts and engineers can relate to them and join discussions
- ✓ Not cluttered with detail; show the whole system
- ✓ Managers find them helpful for project planning

WEAKNESSES

- ✗ Lack semantics (no precise meaning)
- ✗ Do not show the types of relationships between components
- ✗ Do not show externally visible properties of sub-systems
- ✗ Too abstract to be a complete design document

★ KEY POINT

Whether this criticism matters **depends on the use of architectural models**: the requirements for model semantics depend on how the models are used.

Use of architectural models ★★★ MUST KNOW

The apparent contradiction between theory ("block diagrams are poor") and industrial practice ("everyone uses them") arises because an architectural model is used in **two ways**:

	1. As a way of facilitating discussion about the system design	2. As a way of documenting an architecture that has been designed
Aim	A high-level architectural view, useful for communication with system stakeholders and project planning because it is **not cluttered with detail**.	A **complete system model** that shows the different components in a system, their **interfaces** and their **connections**.
Benefit	Stakeholders can relate to it and understand an abstract view of the system, then discuss the system as a whole without being confused by detail. It identifies the key components, so managers can start assigning people to plan their development.	A detailed description makes it easier to **understand and evolve** the system.
Notation	Informal block diagrams are enough; for many projects they are the only architectural description.	Ideally a more rigorous notation, e.g. an architectural description language (ADL, see 6.10).

A detailed, complete description leaves **less scope for misunderstanding** the relationships between components. However, it is **expensive and time-consuming** to develop, and it is practically impossible to know whether it is cost-effective, so this approach is **not widely used**.

6.6 Architectural design decisions ★★★ MUST KNOW

Architectural design is a **creative process** in which you design a system organization that will satisfy the functional and non-functional requirements of a system. There is **no formulaic** (fixed, step-by-step) architectural design process: it differs depending on the **type of system** being developed, the **background and experience** of the system architect, and the **specific requirements** for the system. So it is best to think of architectural design as a series of decisions to be made rather than a sequence of activities.

However, a number of **common decisions span all design processes**, and these decisions affect the **non-functional characteristics** of the system. Based on their knowledge and experience, architects consider the fundamental questions in Fig 6.4.

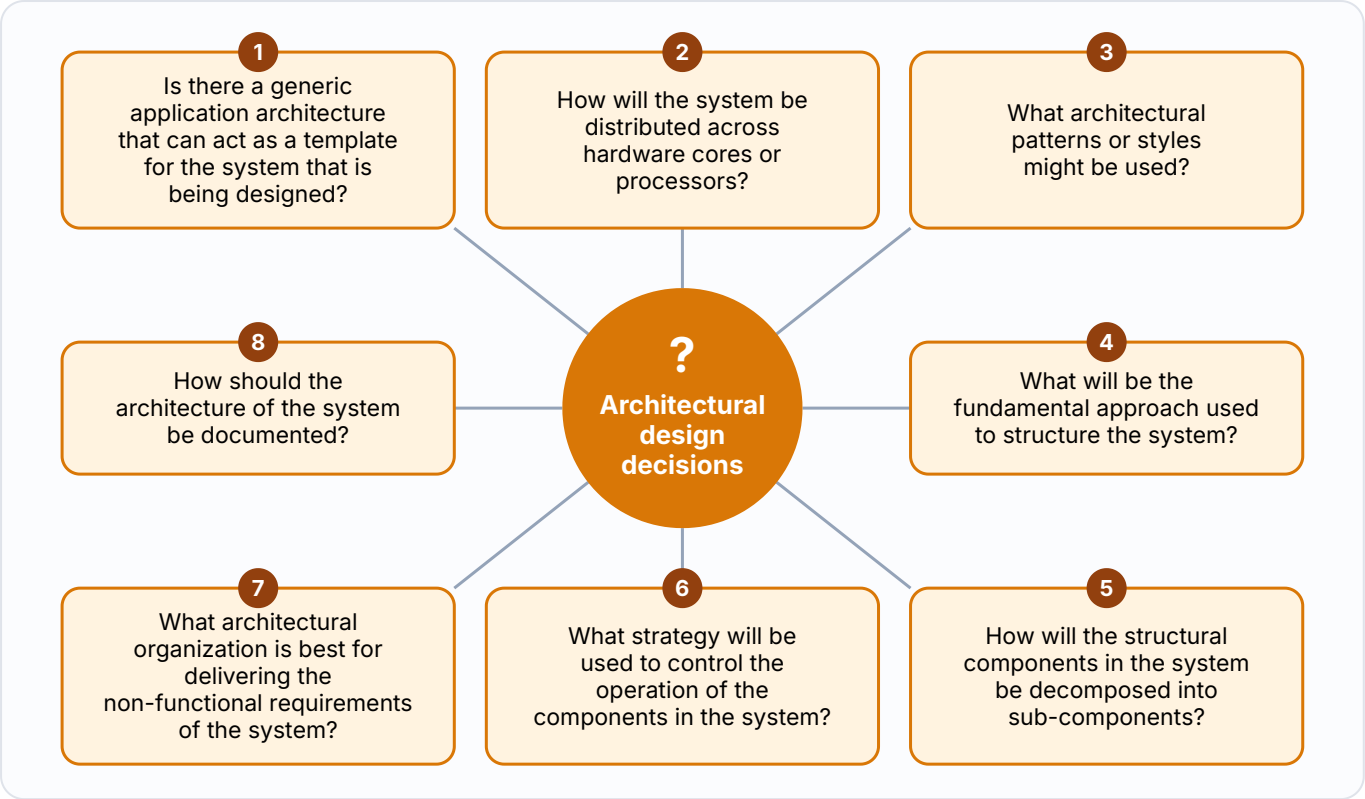


Fig 6.4 Architectural design decisions: the eight fundamental questions, numbered clockwise for revision. Sommerville Fig 6.2 · Your slides, slide 13

Notes that turn the questions into an answer:

- **Q2 (distribution)** — embedded systems and apps for PCs and mobile devices need not be distributed, but most large systems are distributed over many computers. The choice of distribution architecture is a **key decision that affects performance and reliability**.
- **Q4–Q6 (structure, decomposition, control)** — Garlan and Shaw's idea of an **architectural style** covers these three: choose the most appropriate structure (e.g. client–server or layered); decide a strategy for decomposing components into sub-components; and, in **control modelling**, develop a general model of the control relationships between the parts, i.e. how the execution of components is controlled.
- **Q1 and Q3** are about reuse (6.7); **Q7** is explained in 6.8; **Q8** (views and notations) in 6.9–6.10.
- **Evaluating** the design is difficult: the true test is how well the system meets its requirements **when in use**, but you can compare it with **reference architectures** or generic patterns.

MEMORY TRICK

Generic · **D**istribution · **P**atterns · **S**tructure · **D**ecomposition · **C**ontrol · **N**on-functional · **D**ocumentation: "**G**ood **D**evelopers **P**refer **S**imple **D**esigns; **C**lear **N**on-functional **D**ocs".

6.7 Architecture reuse ★★ IMPORTANT

Each system is unique, but architectures are often reused. When designing, you decide what your system has in common with broader classes of applications and how much knowledge from those architectures you can reuse. Three forms:

Same domain

Systems in the same application domain often have **similar architectures** that reflect domain concepts.

Product lines

Application product lines are built around a **core architecture** with **variants** that satisfy particular customer requirements.

Patterns or styles

The architecture may be designed around one or more architectural **patterns** or "styles", which capture the essence of an architecture and can be instantiated in different ways.

DEFINITION — LEARN THIS LINE

Architectural pattern (style) — a description of a system organization, such as a client–server organization or a layered architecture. It captures the essence of an architecture that has been used in different software systems. ("Pattern" and "style" now mean the same thing.)

6.8 Architecture and system characteristics ★★★ MUST KNOW

Because non-functional characteristics depend so closely on the architecture, the **choice of architectural style and structure should depend on the non-functional requirements**. For each critical requirement there is a matching architectural strategy:

If this is critical…	…use this architecture (slide wording)	How and why (textbook detail)
Performance	Localise critical operations and minimise communications. Use **large** rather than fine-grain components.	Keep critical operations in a few components on the **same computer**, not across the network. Large components reduce communications, as most interactions happen inside a component. Replicating the system on different processors may also help.
Security	Use a **layered** architecture with critical assets in the inner layers.	The most critical assets are protected in the innermost layers, with a high level of security validation applied to these layers.
Safety	**Localise** safety-critical features in a small number of sub-systems.	Co-locating safety-related operations reduces the cost of safety validation and may allow a protection system that safely shuts the system down on failure.
Availability	Include **redundant** components and mechanisms for **fault tolerance**.	Spare (redundant) components make it possible to replace and update components without stopping the system. Fault tolerance = continuing to work when a part fails.

If this is critical...	...use this architecture (slide wording)	How and why (textbook detail)
Maintainability	Use **fine-grain**, replaceable components.	Small, self-contained components can readily be changed. Separate producers of data from consumers; avoid shared data structures.

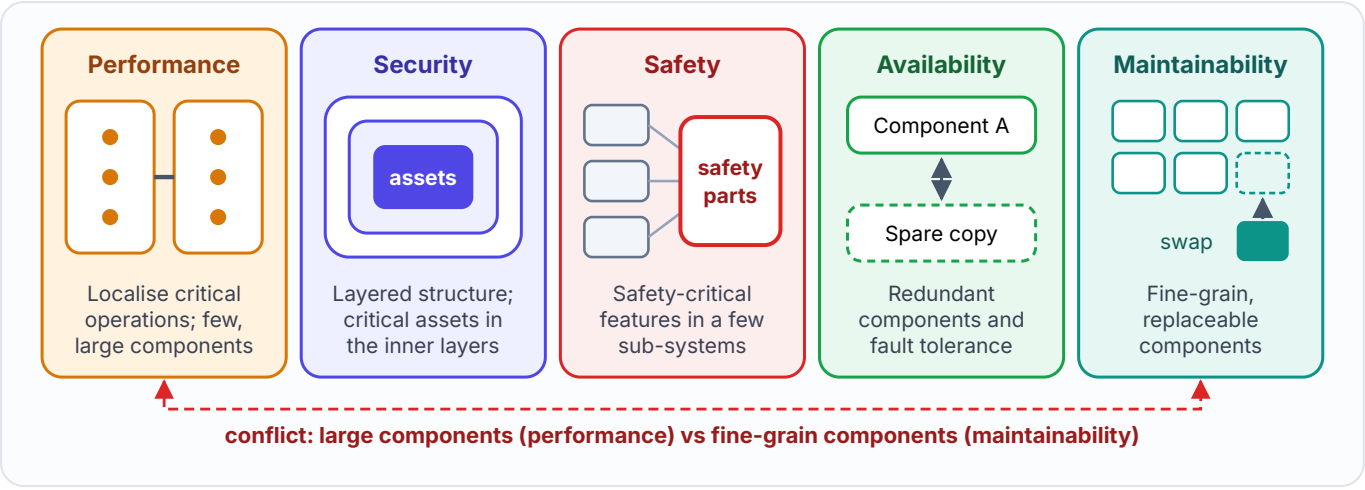


Fig 6.5 The five architectural strategies at a glance, and the textbook's example of a conflict. Drawn for this guide from Your slides, slide 15

Conflicts and trade-offs ★★★ MUST KNOW

There is potential **conflict** between some of these architectures, so a compromise must be found:

Requirements in conflict	Why they pull apart	Compromise
Performance vs maintainability (textbook example)	Performance wants a few **large** components; maintainability wants many **small, fine-grain** components.	Use **different architectural patterns or styles for separate parts** of the system.
Security vs the rest (textbook point)	Security is now almost always a critical requirement, so it cannot simply be traded away.	Design an architecture that maintains security while also satisfying the other non-functional requirements.
Security vs performance (follows from the strategies)	Layers with security validation add extra steps between components; performance wants communication minimised.	Keep time-critical operations together within one layer; apply the heaviest checks only to the inner layers.

> **EXAMPLE**
>
> In **Mentcare** (the clinics' patient information system), patient records are the critical asset, so they would sit in the innermost layer. In the **insulin pump**, the safety-critical dose computation and checks would be co-located in one small component that can be validated thoroughly.

6.9 Architectural views ★★★ MUST KNOW

Whether a model is used to focus discussion or to document a design, two questions arise (slide 17):

1. **What views or perspectives** are useful when designing and documenting a system's architecture?
2. **What notations** should be used for describing architectural models? (see 6.10)

No single diagram can show everything: **each architectural model only shows one view or perspective** of the system. It might show how a system is **decomposed into modules**, how the **run-time processes interact**, or the different ways in which components are **distributed across a network**. All of these are useful at different times, so for both design and documentation you usually need to present multiple views of the software architecture, just as a house needs a floor plan, a wiring plan and a plumbing plan.

Krutchen's 4+1 view model ★★★ MUST KNOW

In his well-known **4+1 view model** of software architecture, Krutchen suggests **four fundamental architectural views**, linked through common **use cases or scenarios** (the "+1"). A use case or scenario describes one typical interaction with the system, e.g. "a doctor records a consultation".

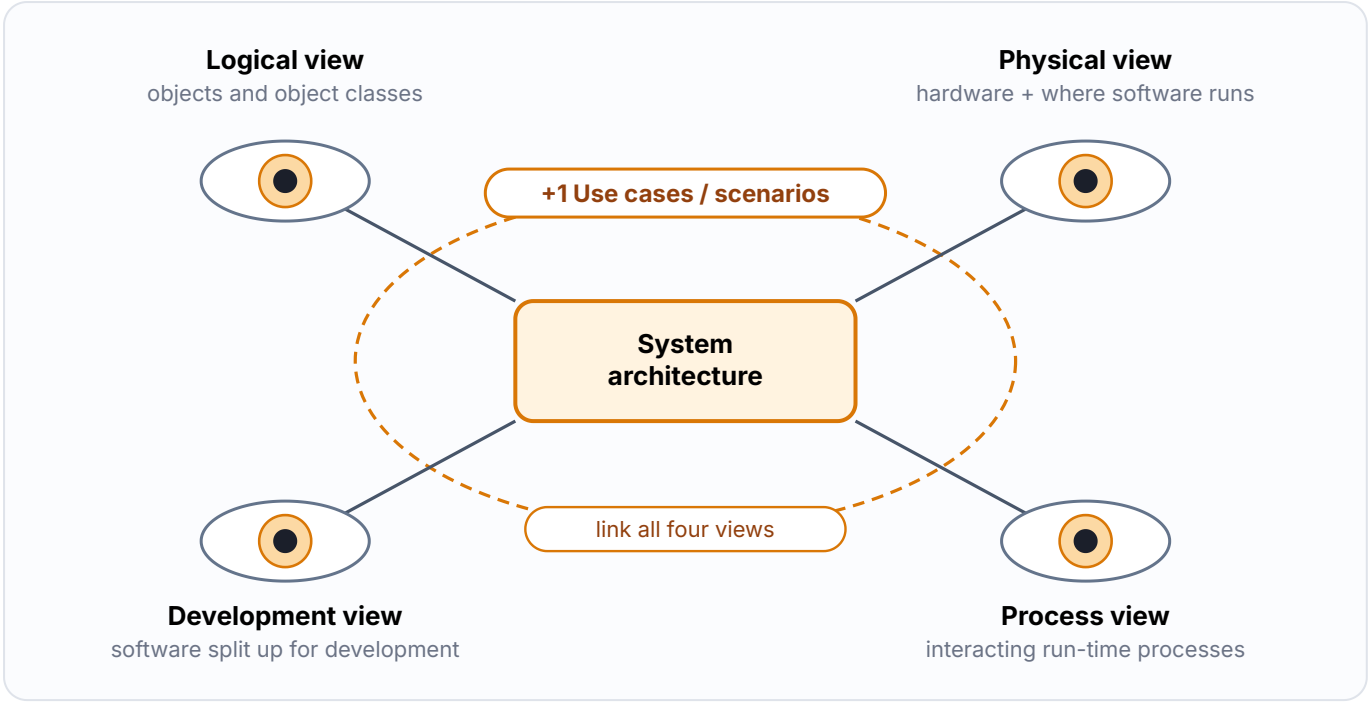


Fig 6.6 Krutchen's 4+1 view model: four views of one system architecture, linked by use cases or scenarios (+1). Sommerville Fig 6.3 · Your slides, slides 18–19 (the "+1" ring added)

View	What it shows (exam wording)	Useful for	Example: Mentcare (illustration)
Logical	The key abstractions in the system as **objects or object classes**. It should be possible to relate the system requirements to entities in this view.	Relating requirements to the design	Classes such as Patient, Consultation, Prescription, Clinic, Staff member.
Process	How, **at run-time**, the system is composed of **interacting processes**.	Judging performance, availability and other non-functional characteristics	A client process on each clinic PC talks to a server process that handles patient-record requests.
Development	How the software is **decomposed for development**: its breakdown into components implemented by a single developer or development team.	Software managers and programmers	Modules for patient records, appointments, prescribing and reporting, each given to one team.

View	What it shows (exam wording)	Useful for	Example: Mentcare (illustration)
Physical	The **system hardware** and how software components are **distributed across the processors** in the system.	Systems engineers planning a system deployment	Clinic PCs connected over the network to a central database server.
+1 Scenarios	Relate the four views to each other.	Linking and checking the views	"Doctor records a consultation" traced through all four views.

⚠ WATCH OUT

Process view (run-time processes) ≠ development view (code split among teams); physical view (hardware) ≠ logical view (classes). The "+1" links the **four** views; it is not a fifth view.

✎ EXAM TIP

10M: draw Fig 6.6 → one line per view (shows + useful for) → the +1 → a Mentcare example each.

The conceptual view, and how many views are needed ★★ IMPORTANT

Hofmeister et al. suggest similar views but add a **conceptual view**: an abstract view of the system that can (1) be the basis for decomposing high-level requirements into more detailed specifications, (2) help engineers make decisions about components that can be reused, and (3) represent a **product line** rather than a single system. **The packing robot architecture (Fig 6.2) is an example of a conceptual view.**

In practice, conceptual views are almost always developed during design, to explain the architecture to stakeholders and inform decision making. Other views may be developed as different aspects are discussed, but it is **rarely necessary to develop a complete description from all perspectives**. Architectural patterns can also be associated with the different views.

6.10 Representing architectural views ★★ IMPORTANT

Which **notation** should describe the views: the **UML** (Unified Modeling Language, the standard diagrams for object-oriented software) or **ADLs** (architectural description languages)?

- Some people argue that the **UML** is an appropriate notation for describing and documenting system architectures.
- **Sommerville disagrees** (he is the "I" on slide 20): the UML does not include abstractions appropriate for **high-level system description**; it was designed for object-oriented systems, and object classes are too close to the implementation. He prefers **informal notations**, quicker to write and easy to draw on a whiteboard. The UML is most valuable for **documenting an architecture in detail** or for **model-driven development** (where models are used to generate the system).
- **ADLs** have been developed but are **not widely used**. Their basic elements are **components and connectors**, with rules for well-formed architectures. They are precise, but specialist languages that domain experts find hard to use.

★ KEY POINT

Informal models and notations, such as the UML, will remain the most common. Agile users say detailed design documents are mostly unused; Sommerville largely agrees. **Except for critical systems**, do not develop a detailed description from all four of Krutchen's views: develop the views **useful for communication** and do not worry about completeness.

Chapter 6 in one page

Definitions

- **Architectural design**: understanding how a system should be organized and designing its overall structure. It is the critical link between requirements engineering and design (it identifies the main components and their relationships). **Output**: an architectural model showing the system as a set of communicating components.
- **In the small**: one program decomposed into components. **In the large**: enterprise systems of systems, programs and components on computers that may belong to different companies.
- **Pattern/style**: a description of a system organization (client–server, layered).
- **Product line**: a core architecture plus variants for particular customers.
- **Conceptual view**: an abstract view used to decompose requirements, decide on reuse and represent product lines (e.g. the packing robot).

Agility and architecture

Design the architecture early: refactoring components is easy, but refactoring the architecture is expensive because it affects most components.

Advantages of explicit architecture (CAR)

1. Stakeholder communication
2. System analysis: can the non-functional requirements be met?
3. Large-scale reuse, including product-line architectures

Box-and-line diagrams

+ simple, intuitive, good for stakeholders and planning. **–** lack semantics; do not show relationship types or visible properties.
Two uses of models: (1) facilitating discussion: high-level, uncluttered; (2) documenting a designed architecture: complete, with components, interfaces and connections.

8 design decisions (Good Developers Prefer Simple Designs; Clear Non-functional Docs)

1. Generic application architecture as a template?
2. Distribution across cores/processors?
3. Patterns or styles?
4. Fundamental approach to structure?
5. Decomposition into sub-components?
6. Control strategy?
7. Organization for non-functional requirements?
8. How to document?

Non-functional requirement → architecture

- **Performance**: localise critical operations; few, large components
- **Security**: layered, with critical assets in the inner layers
- **Safety**: safety features in a small number of sub-systems
- **Availability**: redundant components, fault tolerance
- **Maintainability**: fine-grain, replaceable components

Conflict: performance (large) vs maintainability (fine-grain). Compromise: use different styles for different parts.

4+1 views (Krutchen)

Logical: objects/classes · **Process**: run-time processes · **Development**: decomposition for teams · **Physical**: hardware and distribution · **+1**: use cases/scenarios link them.

Notations

UML: too low-level for high-level design (Sommerville); best for detailed documentation. ADLs: components + connectors; precise but not widely used. Informal notations are enough except for critical systems.

Diagrams to practise

Fig 6.2 packing robot · Fig 6.4 design decisions · Fig 6.6 4+1 views

Likely exam questions

2M **Define architectural design. Why is it called the critical link between design and requirements engineering?**

How to answer → Definition (6.1) → identifies the main components and their relationships → output: an architectural model.

4M **Explain the relationship between agile development and architecture.**

How to answer → Designed early even in agile; refactoring the architecture is expensive (affects most components).

4M **Differentiate architecture in the small and architecture in the large.**

How to answer → Two definitions (6.3), one example each; sketch Fig 6.3.

6M **What are the advantages of explicitly designing and documenting a software architecture?**

How to answer → CAR: stakeholder communication, system analysis, large-scale reuse (6.4).

8M **With a neat diagram, explain the architecture of a packing robot control system.**

How to answer → What the robot does → Fig 6.2 → boxes, boxes in boxes, arrows → a conceptual view.

6M **"Box-and-line diagrams are poor architectural representations." Discuss, and explain the two uses of architectural models.**

How to answer → Criticism → strengths → the two uses as a table → semantics needed depend on use (6.5).

10M **Explain architectural design decisions with a neat diagram.**

How to answer → Creative process (3 factors) → series of decisions → Fig 6.4, all 8 questions → notes on Q2 and Q4–Q6.

8M **How do non-functional requirements affect the architecture? Give the strategy for performance, security, safety, availability and maintainability.**

How to answer → Bosch point → 5-row table (6.8) → performance vs maintainability conflict and compromise.

4M **Explain architecture reuse.**

How to answer → Same domain; product lines (core + variants); patterns/styles with definition (6.7).

10M **Explain Krutchen's 4+1 view model with a neat diagram. Illustrate the views for a system such as Mentcare.**

How to answer → Why multiple views → Fig 6.6 → four views + "useful for" → +1 → Mentcare column.

5M **Discuss the notations used to represent architectural views. Is the UML suitable?**

How to answer → For vs against the UML → ADLs → informal notations enough except for critical systems (6.10).

2M **What is a conceptual view? Give an example.**

How to answer → Hofmeister et al.; three uses; the packing robot (Fig 6.2).

Architectural patterns (MVC, layered, repository, client–server, pipe and filter) and application architectures complete Chapter 6 and are studied after CIE-1.

EXAM DAY

Exam-Day Playbook

How to turn what you studied into marks: answer structure, command words, diagrams and the comparison questions to expect.

Writing answers that score

2 marks

- One-line **definition** in textbook words
- Plus **one** example or key feature
- 2–3 lines total. Don't write an essay.

4–6 marks

- Definition
- A small diagram or a table, if one exists
- 4–6 points, each with a bold keyword and one explaining line
- One example

8–10 marks

- Definition / introduction (2 lines)
- **Neat labelled diagram**, drawn first
- Explain each box or phase under its own sub-heading
- Advantages and disadvantages, or a comparison
- When to use it + a case-study example
- One-line conclusion

★ KEY POINT

Examiners mark **keywords**, **structure** and **diagrams**. Underline the keywords (the highlighted phrases in this guide), number your points, and leave space between them.

Diagram rules

1. **Draw first, write after.** The diagram tells you what to explain, and it scores even if time runs out.
2. **Label every box and arrow** using the textbook's names (for example "Requirements definition", not "Req").
3. **Give it a caption**: *Fig: The waterfall model.*
4. **Use boxes for activities, arrows for flow**, and dashed arrows for feedback or iteration.
5. **Keep it big and clean.** Half a page is fine for a 10-mark question.

Command words decoded

If the question says…	…it wants
Define / What is	The exact definition, plus one example. Short.
List / Enumerate	The complete list, numbered. One line each is enough.

If the question says…	…it wants
Explain / Describe	Definition + points with explanation + diagram if one exists + example.
With a neat diagram	The diagram carries a large share of the marks. Draw it carefully and explain every part.
Differentiate / Compare / Distinguish	A **table** with a "Basis" column and 5–7 rows. Add one line each of "use A when…, use B when…".
Discuss / Justify	Points for and against, then your conclusion with a reason.
Apply to / For the given case study	Name the system in every point. Generic answers lose marks here.

✎ **CASE-STUDY QUESTIONS**

When a question names a system (insulin pump, Mentcare, weather station, iLearn, or something new like a library or ATM system), connect every point to it: "*Because the insulin pump is safety-critical, …*". If the system is new, pick the textbook system it most resembles and reuse that reasoning.

Diagrams you must be able to draw

Every textbook or slide diagram in this guide. Tick each one when you can draw it from memory with all labels. Diagrams drawn only to help understanding are not listed.

Ch 1 · Introduction

☐ **Fig 1.3** The four essential attributes of good software

☐ **Fig 1.4** Software engineering is one part of system engineering and builds on computer science

☐ **Fig 1.5** Insulin pump hardware architecture

☐ **Fig 1.6** Activity model of the insulin pump (UML)

☐ **Fig 1.7** Organisation of the Mentcare system

☐ **Fig 1.8** The weather station's environment (UML packages with the «system» stereotype)

☐ **Fig 1.9** The iLearn architecture

Ch 2 · Software Processes

☐ **Fig 2.1** The waterfall model

☐ **Fig 2.2** Incremental development

☐ **Fig 2.3** Reuse-oriented software engineering

☐ **Fig 2.4** The V-model as drawn by Pressman

☐ **Fig 2.5** The prototyping paradigm

☐ **Fig 2.6** A typical spiral model

☐ **Fig 2.7** The requirements engineering process

☐ **Fig 2.8** A general model of the design process

☐ **Fig 2.9** Stages of testing (slide 28 labels the last stage "Acceptance testing")

☐ **Fig 2.10** Testing phases in a plan-driven software process (V-model)

☐ **Fig 2.11** System evolution

☐ **Fig 2.12** The process of prototype development

Ch 4 · Requirements Engineering

☐ **Fig 4.1** User and system requirements

☐ **Fig 4.2** Readers of different types of requirements specification

☐ **Fig 4.3** The eight stakeholder groups of the Mentcare system (the textbook also includes…

☐ **Fig 4.4** One non-functional requirement can generate new functional requirements and restrict…

☐ **Fig 4.5** Types of non-functional requirement

☐ **Fig 4.7** A spiral view of the requirements engineering process

☐ **Fig 4.8** The requirements elicitation and analysis process (step 4 is called "requirements…

☐ **Fig 4.9** Ethnography and prototyping for requirements analysis

☐ **Fig 4.10** Use cases for the Mentcare system

☐ **Fig 4.11** Users of a requirements document and how they use it

☐ **Fig 4.12** Requirements evolution

☐ **Fig 4.13** Requirements change management

☐ **Fig 4.14** What the insulin pump software does, with the textbook requirements that govern each step

Ch 5 · Design Concepts

☐ **Fig 5.1** Design is the last action of modeling

☐ **Fig 5.2** Translating the requirements model into the design model

☐ **Fig 2.13** Incremental delivery

☐ **Fig 2.14** The process improvement cycle

☐ **Fig 2.15** Capability maturity levels as a staircase (box names from slide 54; grey meanings from...

Ch 3 · Agile

☐ **Fig 3.1** Plan-driven and agile development

☐ **Fig 3.3** The XP release cycle

☐ **Fig 3.4** A "prescribing medication" story card

☐ **Fig 3.5** Examples of task cards for prescribing medication (Task 3 shown in full)

☐ **Fig 3.8** Test case description for dose checking

☐ **Fig 3.9** The Scrum sprint cycle

☐ **Fig 3.11** Distributed Scrum

☐ **Fig 3.12** Factors influencing the choice of plan-based or agile development

☐ **Fig 5.3** The textbook's door example

☐ **Fig 5.4** Modularity and software cost

☐ **Fig 5.8** A crosscutting concern (user validation) as scattered/tangled fragments (left) versus...

☐ **Fig 5.9** The textbook's refactoring case

☐ **Fig 5.10** Core OO ideas

☐ **Fig 5.11** The analysis class FloorPlan refined into a design class

☐ **Fig 5.13** Dimensions of the design model

☐ **Fig 5.14** Interface representation for ControlPanel

☐ **Fig 5.15** (a) UML component diagram

Ch 6 · Architectural Design

☐ **Fig 6.2** The architecture of a packing robot control system

☐ **Fig 6.4** Architectural design decisions

☐ **Fig 6.5** The five architectural strategies at a glance, and the textbook's example of a conflict

☐ **Fig 6.6** Krutchen's 4+1 view model